

UNIVERSIDAD DE CHILE
FACULTAD DE CIENCIAS FÍSICAS Y MATEMÁTICAS
DEPARTAMENTO DE INGENIERÍA ELÉCTRICA

MITIGATING CATASTROPHIC FORGETTING IN OBJECT CLASSIFICATION VIA GAUSSIAN CLASS REPRESENTATIONS

TESIS PARA OPTAR AL GRADO DE
MAGÍSTER EN CIENCIAS DE LA INGENIERÍA, MENCIÓN ELÉCTRICA

MEMORIA PARA OPTAR AL TÍTULO DE
INGENIERO CIVIL ELÉCTRICO

IGNACIO ROMERO ARAVENA

PROFESOR GUÍA:
JAVIER RUIZ DEL SOLAR

PROFESOR CO-GUÍA:
PATRICIO LONCOMILLA

MIEMBROS DE LA COMISIÓN:
JORGE SILVA
RODRIGO VERSCHAE

Este trabajo ha sido parcialmente financiado por:

FONDECYT 1251823

SANTIAGO DE CHILE
2026

RESUMEN DE LA TESIS PARA OPTAR AL
GRADO DE MAGÍSTER EN CIENCIAS DE LA
INGENIERÍA, MENCIÓN ELÉCTRICA,
MEMORIA PARA OPTAR AL TÍTULO DE
INGENIERO CIVIL ELÉCTRICO
POR: IGNACIO ROMERO ARAVENA
FECHA: 2026
PROF. GUÍA: JAVIER RUIZ DEL SOLAR
PROF. CO-GUÍA: PATRICIO LONCOMILLA

MITIGACIÓN DEL OLVIDO CATASTRÓFICO EN LA CLASIFICACIÓN DE OBJETOS MEDIANTE REPRESENTACIONES GAUSSIANAS DE CLASES

Los modelos tradicionales de aprendizaje profundo suelen fallar cuando trabajan con datos que cambian en el tiempo, ya que incorporar clases nuevas exige un reentrenamiento costoso o provoca que el modelo olvide lo aprendido. Para resolver esto, se propone un método de aprendizaje continuo libre de gradientes enfocado en el reconocimiento visual. En lugar de optimizar parámetros, el sistema aborda la tarea estimando densidades en forma cerrada a partir de las representaciones de un modelo fundacional como extractor de características congelado, modelando cada clase mediante una distribución gaussiana multivariada. Así, las clases nuevas se integran al instante, sin requerir retropropagación, almacenamiento de muestras, ni ajuste de parámetros. Además, como la información de las clases anteriores nunca se altera, se previene el olvido por diseño. Al evaluarse en benchmarks como Split CIFAR-100, Split ImageNet-R y Split Caltech-256, este enfoque logra un rendimiento competitivo frente a métodos que dependen de optimización de parámetros. Entre las configuraciones probadas, la variante MOG4-Shared —que combina una mezcla de gaussianas por clase con una covarianza compartida— logró el mejor desempeño. Finalmente, un análisis estadístico valida el supuesto gaussiano y muestra que la covarianza óptima depende de los datos disponibles por clase.

RESUMEN DE LA TESIS PARA OPTAR AL
GRADO DE MAGÍSTER EN CIENCIAS DE LA
INGENIERÍA, MENCIÓN ELÉCTRICA,
MEMORIA PARA OPTAR AL TÍTULO DE
INGENIERO CIVIL ELÉCTRICO
POR: IGNACIO ROMERO ARAVENA
FECHA: 2026
PROF. GUÍA: JAVIER RUIZ DEL SOLAR
PROF. CO-GUÍA: PATRICIO LONCOMILLA

MITIGATING CATASTROPHIC FORGETTING IN OBJECT CLASSIFICATION VIA GAUSSIAN CLASS REPRESENTATIONS

Traditional deep learning models struggle with non-stationary data, as adding new categories typically requires costly retraining or causes catastrophic forgetting. This thesis introduces a gradient-free, class-incremental learning framework that frames lifelong visual recognition as closed-form density estimation rather than parameter optimization. By pairing a frozen foundation model as backbone with per-class multivariate Gaussian models, new classes are integrated instantly without backpropagation, storing past samples, or fine-tuning. Because existing class statistics remain untouched, the framework eliminates forgetting by design. Evaluated across benchmarks such as Split CIFAR-100, Split ImageNet-R, and Split Caltech-256, this approach achieves competitive performance against optimization-heavy methods. Among the tested variants, MOG4-Shared—which combines a compact per-class mixture with a single shared covariance—provides the best trade-off between computational efficiency and accuracy. Finally, evaluating the feature space of the DINOv2 foundation model through normality tests, spectral analysis, and Bhattacharyya distances validates the Gaussian assumption for feature extraction and shows that the optimal covariance structure depends directly on sample availability per class.

A los completos con Coca Zero

Acknowledgments

A mi mamá y mi papá, por siempre confiar en mí y darme todas las herramientas para lograr lo que quiera en la vida. Son los mejores papás que he tenido, los amo un huevo.

A mi hermana, la persona que más amo en el mundo. Todo lo que soy, voy a ser, he logrado y voy a lograr, es para ti.

A Lili, Miguelín, Máximo Augusto. A la Loreto. A mi tata, Yaqui, mi mami y la tía Ketita. La familia no se escoge, pero yo los voy a escoger siempre.

Al Charly, el Tony, Conbotas, y la Princesa. Cuando me titule les compro un churu.

A la Isi y a la Amandita: el mejor regalo que me entregó la universidad, mis mejores amigas. Por ser luz y alegrar mis días.

Marian, Miriam, Marina, Morin. Nunca supe cómo era, pero gracias por tantos años de amistad.

A quienes se han ido sumando a lo largo de los años: Máximo, Meza, Sophia, Andrés, Cristóbal, Benja, Germán, Kavi, Sofía Amapola, Luna, Renatito, Javi, Nachita. Sin ustedes la vida no sería la misma, los quiero.

A mis rompehogares: Gonz, Matti, Sebita, Lucho, Sahian, Fer, Rona, Bruno, Felipe, Pipe. Por los chistecitos a la hora de almuerzo, el apañe académico, Mario Kart, cartitas pokémon y tantas otras ñoñerías que hemos pasado juntos.

Porque no existen los deportes individuales: a la rama de natación y Pace One. Pionxi, Oscar, Javis, Martín, Edgar, Seba, Jo, Pía, profe Moni, coaches Salazar, Cami y tantos compañeros y compañeras de entrenamiento. Son el mejor equipo que puedo pedir.

A mis profesores, por inspirarme más allá de lo que decía el libro. Irma Meza, Yarith Jorrat, Harold Leiva, Súa Vicencio, María Isabel Caqueo, Felipe Muñoz, Joaquín Álvarez. Siempre estaré muy agradecido de haber pasado por sus salas de clase.

Al profesor Javier y a Pato Loncomilla: gracias por confiar en mí.

Y finalmente a mi amigo Jesús, por acompañarme en el camino y enseñarme que la medida del amor, es amar sin medidas.

Declaración y uso de IA

Para la redacción, estructuración y refinamiento de esta tesis se empleó Claude, herramienta de inteligencia artificial generativa desarrollada por Anthropic, a través de la plataforma Claude Code. Específicamente, la herramienta asistió en la revisión gramatical y de estilo técnico, la reorganización y jerarquización lógica de secciones, la elaboración de borradores preliminares para el resumen ejecutivo y la redacción de descripciones para tablas e ilustraciones.

Las interacciones con la herramienta se basaron en instrucciones descriptivas y contextuales orientadas a la optimización del texto, tales como solicitudes para reescribir pasajes buscando mayor claridad y concisión, resumir apartados en párrafos breves, reorganizar la secuencia lógica de ciertas secciones o sintetizar borradores de resúmenes ejecutivos que integraran el problema, la arquitectura propuesta, las pruebas experimentales y las principales contribuciones del trabajo.

El autor revisó, contrastó y validó de forma crítica la totalidad del contenido antes de su incorporación definitiva en el documento. Cada afirmación técnica, ecuación y resultado se cotejó rigurosamente con los datos experimentales obtenidos y con la arquitectura efectivamente implementada. Asimismo, se garantizó la coherencia general del documento, la consistencia en la notación matemática y la continuidad de la línea argumental. Aquellos fragmentos que no se ajustaban al estilo o al nivel de detalle requerido fueron reescritos de manera independiente, siendo el autor quien incorporó de forma directa los datos cuantitativos, las referencias bibliográficas y los detalles de implementación, confirmando que todos los análisis y conclusiones corresponden estrictamente a su criterio profesional.

Finalmente, se declara que la herramienta de inteligencia artificial se utilizó de manera transparente y exclusiva como un insumo de apoyo al trabajo intelectual, por lo que no ostenta condición alguna de coautoría. La responsabilidad académica, ética y técnica sobre los resultados, análisis y conclusiones presentados en este trabajo corresponde de manera íntegra y exclusiva al autor.

Contents

List of Acronyms **xiv**

1 Introduction **1**

1.1 Motivation 1

1.2 Hypothesis 4

1.3 General Objective 4

1.4 Specific Objectives 4

1.5 Contributions 4

1.6 Thesis Structure 5

2 Continual Learning and Foundation Models **7**

2.1 Artificial Intelligence and Machine Learning 7

2.1.1 Deep Learning 8

2.1.2 Computer Vision 8

2.2 Continual Learning 9

2.2.1 Stability and Plasticity 10

2.2.2 Main Paradigms 11

2.2.3 Related Work in Continual Learning 11

2.3 Foundation Models 18

2.3.1 The Foundation Model Paradigm 18

2.3.2 Vision Transformers 19

3 Statistical and Probabilistic Foundations of the Feature Space 22

3.1 Foundations of Probabilistic Modeling 22

3.1.1 Bayes Classification 23

3.1.2 Likelihood-Based Classification 23

3.2 Gaussianity Assumptions and Analysis 24

3.2.1 Per-Dimension Gaussianity 25

3.2.2 Per-Class Gaussianity 29

3.3 Inter-Class Structural Separability Analysis 30

3.3.1 Pairwise Bhattacharyya Distance 30

3.3.2 Pairwise Mean Euclidean Distance 30

3.3.3 Stream Statistics and Trajectory Tracking 30

3.4 Covariance Estimation in High-Dimensional Spaces 31

3.4.1 The Curse of Dimensionality and Covariance Degeneracy 31

3.4.2 Theoretical Foundations of Regularization 32

3.5 Spectral Geometry of the Feature Space 32

3.5.1 Principal Component Analysis 33

3.5.2 Spectral Decay and Effective Dimensionality 33

3.5.3 PCA Dimension-Reduction Testing 33

4 Modelling Class Densities in a Frozen Feature Space 35

4.1 Proposed Architecture 35

4.1.1 Likelihood-Based Classification 37

4.2 Covariance Estimation 37

4.2.1 Diagonal Covariances 38

4.2.2 Full Covariances 38

4.2.3 Hybrid Covariance 39

4.2.4 OAS Covariance 40

4.2.5 Ledoit-Wolff 40

4.2.6 Shared Covariance 41
4.3 Mixture of Gaussians 41
4.3.1 Mixture of 2 Gaussians 41
4.3.2 Mixture of up-to-4 Gaussians 42

5 Experimental Results and Analysis **44**
5.1 Image Datasets and Benchmarks 45
5.1.1 Source Datasets 45
5.1.2 Continual Learning Splits and Protocol 45
5.1.3 Benchmark Properties Summary 46
5.2 Evaluation Metrics: Average Accuracy and Forgetting 46
5.3 Confidence and Calibration Metrics 47
5.3.1 Expected Calibration Error (ECE) 47
5.3.2 Confidence on Errors (CoE) 48
5.3.3 Total Confidence (TC) 48
5.3.4 Probability Margin (PM) 49
5.3.5 Average Predictive Entropy (H) 49
5.4 Covariance Estimator and Mixture-of-Gaussian Results 50
5.4.1 Comparison with State-of-the-Art Methods 50
5.4.2 Covariance Parameterisations and the Shape-Volume Axis 51
5.4.3 Linear Shrinkage versus Structural Decoupling 52
5.4.4 Shared Covariance in the Long-Horizon Joint Benchmark 53
5.4.5 Overall Best Configuration: MOG4-Shared 53
5.5 Calibration Results and the Cost of High Dimensionality 54
5.5.1 Overconfidence of the Full Covariance Model 55
5.5.2 Calibration of the Diagonal Model 55
5.5.3 Calibration of MOG4-Shared Model 55
5.6 Incremental Class Inclusion 56

5.7 Inter-Class Separability 57
5.7.1 Per-Class Accuracy Once All Classes Are Added 57
5.7.2 Class Overlap: Bhattacharyya and Euclidean Distances 59
5.7.3 Worst-Performing Classes and Failure Diagnostics 62
5.7.4 Synthesis of Inter-Class Confusion and Structural Bounds 65
5.8 Sensitivity to the Choice of Backbone 66
5.8.1 Interpreting the Backbone Comparison 67
5.9 Spectral Analysis of the DINOv2 Embedding Space 67
5.9.1 Spectral Properties of the Foundation Embeddings 68
5.9.2 Empirical Validation via Subspace Compression Tracking 69
5.9.3 Covariance Parameterisations and Subspace Interplay 69
5.10 Validating the DINOv2 Gaussianity Assumption 70
5.10.1 Per-Dimension Gaussianity 70
5.10.2 Per-Class Gaussianity 71
5.10.3 Empirical Validation of the Gaussian Assumption 72
5.11 Computational Cost and Practical Considerations 73
5.11.1 Computational Footprint and Practical Scalability 73
5.11.2 Limitations and Architectural Open Questions 74
5.12 Summary of Findings and Practical Recommendations 74

6 Conclusion **76**

6.1 Future Work 78

Bibliography **84**

List of Tables

5.1 Statistical characterisation of the four benchmarks. $\overline{n_c}$ denotes the average sample budget per class. 46

5.2 Summary of the probabilistic calibration and uncertainty metrics used in this work. "Better" indicates the direction of the preferred value; "context" means both high and low values are informative depending on whether predictions are correct. 50

5.3 Continual-learning performance on the three 10-task benchmarks. A_{10} and F_{10} are reported in percentage points. 51

5.4 Performance on the long-horizon **Split Joint Datasets** benchmark (30 tasks aggregated from CIFAR-100, ImageNet-R and Caltech-256). A_{30} and F_{30} are reported in percentage points. 52

5.5 Probabilistic calibration metrics for the Bayesian classifier at the last after-task checkpoint. 54

5.6 Bottom-5 accuracy classes under Diag-Cov. Conf.%: error rate to primary confusion target. D_B, d_E: target Bhattacharyya/Euclidean distances ($\overline{D_B}, \overline{d_E}$: dataset averages). 63

5.7 Bottom-5 accuracy classes under Full-Cov. Conf.%: error rate to primary confusion target. D_B, d_E: target Bhattacharyya/Euclidean distances ($\overline{D_B}, \overline{d_E}$: dataset averages). 63

5.8 Impact of backbone architecture on Diag-Cov, Full-Cov, and MOG4-Shared across three benchmarks. 67

5.9 Final average accuracy (%) of the Gaussian continual-learning classifier across dimensionality settings, comparing PCA projections ($d \in \{64, 256\}$) against the full feature space ($d = 768$). *Diag*: diagonal covariance; *Full*: full covariance with adaptive ridge regularisation. Bold indicates the highest accuracy within each dimensionality setting. Higher is better. 69

5.10 Per-dimension Gaussianity statistics over all $(C \times d)$ (class, dimension) pairs per dataset ($d = 768$). $|\text{Skew}|_\mu$ and $|\text{Kurt}|_\mu$ are mean absolute values; $\mathbb{P}[\cdot]$ is the fraction of pairs satisfying each cut-off. 70

5.11 Per-feature normality tests on the training split (fractions of $(C \times d)$ pairs). *normaltest*: D'Agostino–Pearson χ^2_2 at α=0.05. *relaxed*: |Skew| < 1 and |Kurt| < 1 simultaneously. 71

5.12 Summary results from computing the univariate Henze–Zirkler test statistic over all $(C \times d)$ class-dimension pairs on the training split. $\mathbb{E}_{H_0}[T_{\mathrm{HZ}}]$ is the theoretical expectation under perfect normality, as defined in Chapter 3, evaluated at the per-class sample size n_c. 71

5.13 Summary results from computing the diagonal HZ test statistic over all classes in the ambient foundation feature space ($d = 768$), using all available training samples. Values concentrated near $T_{\mathrm{HZ}} \approx 1.00$ indicate convergence toward empirical multivariate Gaussianity under the feature-independence assumption. 72

5.14 Inference cost of the Gaussian classification stage once every class has been added, in floating-point operations (FLOPs) per query, for the two dominant covariance parameterisations and for MOG4-Shared. For reference, a single forward pass of the DINOv2 ViT-B/14 feature extractor requires approximately 17.56 GFLOPs. 73

List of Figures

1.1 Overview of the continual-learning framework: a frozen foundation model extracts features $\vec{x}$, classes are modeled independently as Gaussians (G_k) to allow dynamic registration without retraining, and inference predicts via Bayesian posterior maximization. 3

2.1 Machine Learning and Deep Learning Strategies. Image taken from [1]. . . . 9

2.2 A sequential task arrival model for Continual Learning, depicting the transition from Task A to Task C over time, where each task represents a unique set of features or data distributions. 10

2.3 Standard Object Classifier comparison. The upper path shows standard inference, while the lower path illustrates the dilemma when a new class is introduced: the trade-off between catastrophic forgetting via fine-tuning and the infeasibility of retraining from scratch. 11

2.4 Class and Task Incremental Learning. Image taken from [2]. 12

2.5 Vision Transformer (ViT) architecture overview. The model processes an image as a sequence of patches through a standard Transformer encoder. Image taken from [3]. 20

3.1 Visual representation of kurtosis levels: leptokurtic (> 0), mesokurtic ($= 0$) and platykurtic (< 0) distributions [4]. 26

3.2 Comparison of negatively-skewed, symmetric and positively-skewed distributions [5]. 26

4.1 Overview of the proposed framework, reproduced from the peer-reviewed article that accompanies this thesis. A frozen foundation encoder maps each input image to a fixed embedding $\vec{x} \in \mathbb{R}^d$. Each class is represented by a Gaussian density (mean and covariance) that is updated incrementally without backpropagation. At inference, classification follows the MAP rule under the learned class-conditional densities. 36

5.1 Evolution of average accuracy across ten sequential tasks for CIFAR-100 (blue), ImageNet-R (green), and Caltech-256 (black), under (**a**) Diag-Cov, (**b**) Full-Cov, and (**c**) MOG4-Shared estimation. 57

5.2 Per-class accuracy once all classes have been added, for CIFAR-100 ($N = 100$ classes). Left: Diag-Cov. Right: Full-Cov. 58

5.3 Per-class accuracy once all classes have been added, for ImageNet-R ($N = 200$ classes). 58

5.4 Per-class accuracy once all classes have been added, for Caltech-256 ($N = 250$ classes). 58

5.5 Inter-class separability under the Diag-Cov Gaussian model as the class set grows from $k = 2$ to the full benchmark size. **(a)** Mean pairwise D_B with shaded $[\mathrm{P}_{10}, \mathrm{P}_{90}]$ band. **(b)** Minimum pairwise D_B; the dashed horizontal line marks $D_B = 20$. 60

5.6 Count of class pairs below three overlap thresholds ($D_B < 10$, $D_B < 50$, $D_B < 100$) as the class set grows. Over 99.8 % of all pairs exceed $D_B = 50$ on every benchmark. 60

5.7 Inter-class separability in terms of the plain Euclidean distance d_E between class means, as the class set grows from $k = 2$ to the full benchmark size. **(a)** Mean pairwise d_E with shaded $[\mathrm{P}_{10}, \mathrm{P}_{90}]$ band. **(b)** Minimum pairwise d_E. 61

5.8 Count of class pairs below three Euclidean-distance overlap thresholds as the class set grows, complementing the Bhattacharyya-based count of Figure 5.6. 61

5.9 Representative visual specimens of the primary low-accuracy classes and corresponding confusion sinks on the Split CIFAR-100 benchmark, mapped to the tracking trends outlined in Table 5.6 and Table 5.7. 64

5.10 Representative specimens from Split ImageNet-R mapped to the tracking trends outlined in Table 5.6 and Table 5.7, showing the non-photographic style of the dataset. 64

5.11 Representative visual entities comprising the minimum separability boundary on the Split Caltech-256 stream mapped to the tracking trends outlined in Table 5.6 and Table 5.7. 65

5.12 PCA spectral decay of the ℓ_2-normalised DINOv2 features. **(a)** Individual % variance explained per component: steep decay over the first ~ 50 components, near-zero contribution thereafter. **(b)** Cumulative % variance explained: dashed reference lines mark 50, 80, 90 and 95 %; annotated n values show how many components suffice to reach 90 % on each dataset. Despite the $d = 768$ embedding space, 90 % of the variance is captured by fewer than 130 components on every benchmark. 68

List of Acronyms

AI Artificial Intelligence

API Adaptive Plasticity Improvement

BIC Bayesian Information Criterion

CIL Class-Incremental Learning

CL Continual Learning

CNN Convolutional Neural Network

CoE Confidence on Errors

DER Dark Experience Replay

DINO self-DIstillation with NO labels

DINOv2 DINO, version 2

ECE Expected Calibration Error

EM Expectation-Maximization

EMA Exponential Moving Average

ER Experience Replay

EWC Elastic Weight Consolidation

FLOPs Floating-Point Operations

GDA Gaussian Discriminant Analysis

GFLOPs Giga Floating-Point Operations

GMM Gaussian Mixture Model

HDSS High-Dimension, Small-Sample-Size

HZ Henze–Zirkler

LW Ledoit–Wolf

MAP Maximum A Posteriori

MDP Markov Decision Process

ML Machine Learning

MOG Mixture Of Gaussians

MSE Mean Squared Error

NMC Nearest Mean Classifier

OAS Oracle Approximating Shrinkage

OOD Out-Of-Distribution

PCA Principal Component Analysis

PEFT Parameter-Efficient Fine-Tuning

PM Probability Margin

RPN Region Proposal Network

SAM Segment Anything Model

SCM Sample Covariance Matrix

SI Synaptic Intelligence

SOTA State-Of-The-Art

TC Total Confidence

ViT Vision Transformer

Chapter 1

Introduction

1.1 Motivation

State-of-the-art (SOTA) object classification methods rely on deep learning models that are either trained end-to-end on the target application or initially trained on one task and subsequently fine-tuned for a different one. In both cases the resulting models are *monolithic*: a single network is trained on a large dataset and performs feature extraction and classification jointly. Such models are highly effective when the set of object classes is fixed and known at training or fine-tuning time. When new object classes must be incorporated, however, the entire model must be retrained or fine-tuned. This procedure requires significant computational resources, demands access to the original training dataset and, in many cases, leads to *catastrophic forgetting* of previously learned classes [6].

A natural way to integrate new classes while avoiding catastrophic forgetting is to formulate the problem as continual or class-incremental classification, in which the algorithm must learn to distinguish an ever-growing number of categories [7]. In this paradigm, a *task* corresponds to a distinct portion of the training experience, presented sequentially to the model. Instead of having access to the entire dataset at once, as in conventional supervised learning, the model receives data in sequential chunks [8]. Each task typically corresponds to a set of new classes that must be recognised. The central challenge is that when the model is updated task by task, it tends to overwrite previously acquired knowledge: monolithic models are structurally ill-suited for this scenario, and an alternative paradigm is therefore required.

This limitation has motivated the SOTA in continual classification to focus on balancing *stability* and *plasticity*. Stability refers to a model's ability to preserve previously learned knowledge; plasticity describes its capacity to adapt and incorporate new tasks [9]. Traditional approaches rarely optimise both properties simultaneously: when stability dominates, the model struggles to integrate new knowledge; when plasticity dominates, it loses access to past information. This trade-off is compounded by the limited capacity of neural networks to integrate new representations without interfering with old ones and by the need to continually adjust the network's structure to match the growing number of object classes.

The dominant view in the literature therefore treats continual learning as a problem of *parameter interference*: learning new classes requires updating network weights, which may overwrite previously acquired knowledge, and most existing methods attempt to control this process through regularization, rehearsal buffers, or architectural adaptations [10]. Although effective, these strategies share a common premise that this thesis sets out to challenge: that incremental learning must be achieved through continuous, careful optimization of the parameters of the network responsible for feature extraction.

This premise diverges significantly from the organisational principles of biological vision. The human visual system relies on early cortical regions responsible for feature extraction that mature and stabilize during the initial postnatal weeks or months [11], providing highly stable feature representations that remain stationary throughout adulthood. However, despite this invariant sensory foundation, human lifelong learning persists. Higher-order cognitive systems seamlessly map these fixed perceptual representations onto newly encountered semantic categories, even if acquiring knowledge of a new object class can occasionally result in a slight, localized reduction in the ability to distinguish highly similar classes over time.

A directly analogous structural separation has recently become achievable in computer vision through large-scale visual foundation models. By leveraging pre-trained foundation models such as DINOv2 [12]—which are trained on massive, diverse datasets—downstream systems obtain highly discriminative representations that remain completely stable once frozen. In this stationary feature space, *representation drift*—the gradual shifting of a network's internal features during sequential task updates, which serves as the primary driver of catastrophic forgetting—is eliminated by construction: the coordinates describing each class no longer change when a new task arrives. Mirroring the biological paradigm, class-incremental learning can thus be conceptually reframed: rather than protecting network parameters from destructive interference, the challenge becomes one of accurately modeling the distribution of new classes within a robust, fixed embedding space. This perspective motivates the present work: once a sufficiently discriminative and stable feature space is secured, the core challenge of continual learning shifts from *preventing forgetting* to *estimating probability densities*.

Building on this observation, this thesis proposes a continual-learning object-recognition system organized into two complementary, decoupled components. As illustrated in Figure 1.1, the first component is a *feature extractor* derived from a frozen, pre-trained foundation model. This encoder maps an input image to a static, highly discriminative feature vector $\vec{x}$ in a shared embedding space. Crucially, these representations must be sufficiently general to characterize arbitrary object classes, including those unseen during the foundation model's pre-training. The second component is a *statistical classifier* composed of a collection of independent, class-conditional Gaussian models $(G_1, \ldots, G_N)$. When a new class arrives, a corresponding Gaussian model is estimated and integrated without altering or accessing previously learned class models. At inference time, classification is performed by computing the posterior probability $p(c_k|\vec{x})$ for each class and assigning the input to the class that yields the maximum value, in direct analogy to a generative Bayes classifier. This modular, two-stage framework is developed in full mathematical detail in Chapter 4.

Gaussian class representations offer two key advantages that make them particularly well suited to this reframed problem. First, each class is modelled independently from the rest, so incorporating a new class amounts to estimating a small number of sufficient statistics

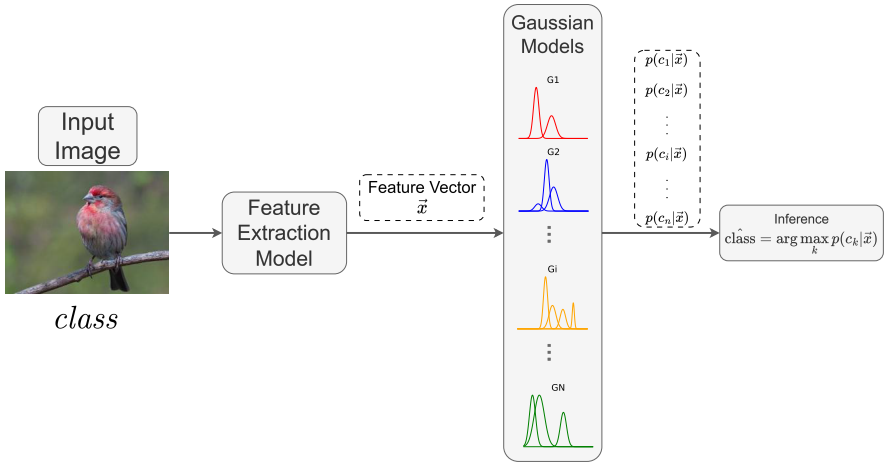


Figure 1.1: Overview of the continual-learning framework: a frozen foundation model extracts features $\vec{x}$, classes are modeled independently as Gaussians (G_k) to allow dynamic registration without retraining, and inference predicts via Bayesian posterior maximization.

(mean and covariance) and does not require any modification of the previously stored class models; this yields incremental updates that are exact and gradient-free, and that produce zero catastrophic forgetting by construction. Second, Gaussian models are compact: they encode each class in a few parameters that can be estimated from a modest sample budget and refined as new samples become available, without any storage of the underlying images. This second property, however, is far from trivial in practice: because the dimensionality d of foundation-model embeddings (typically in the hundreds) is frequently comparable to, or larger than, the number of training samples n_c available per class, the empirical covariance matrix is structurally rank-deficient whenever $n_c < d$. This thesis studies this small-sample regime in detail, evaluating a full spectrum of covariance parameterisations—from diagonal to full covariance matrices, including shared and shrinkage-based estimators.

Because the entire framework rests on the assumption that class-conditional features are approximately Gaussian, this assumption cannot be taken for granted: it must be empirically validated rather than merely postulated. This thesis therefore complements its architectural proposal with a rigorous statistical characterisation of the DINOv2 embedding space, combining moment-based diagnostics (skewness and excess kurtosis) with formal hypothesis tests—the D'Agostino–Pearson omnibus test [13, 14] for univariate normality and a diagonal adaptation of the Henze–Zirkler test [15] for multivariate normality under severe rank deficiency. In parallel, the *Bhattacharyya distance* is used to quantify inter-class separability and to establish a principled theoretical ceiling on the accuracy that any Gaussian classifier can achieve in a given feature space. This separability analysis is not merely descriptive: it is used throughout this thesis to identify, explain, and visualise the specific pairs of classes whose overlapping Gaussian distributions constitute the dominant source of classification error observed in this framework, as opposed to any degradation of previously learned representations.

1.2 Hypothesis

We hypothesize that the combination of an independent class-conditional Gaussian model and a frozen pre-trained feature extractor provides a robust framework for continual object classification. By enabling the addition of new classes without full-model retraining, this design eliminates catastrophic forgetting while maintaining a simple, compact architecture.

1.3 General Objective

The main objective of this thesis is to develop and to validate a continual learning framework for object classification using pre-trained foundation models and independent class-conditional Gaussian models as classifiers, with the purpose of enabling the incremental incorporation of new classes while preventing catastrophic forgetting

1.4 Specific Objectives

For the purpose of fulfilling the general objective and verifying the hypothesis of this work, the following specific objectives are defined:

- To assess the practical performance of the proposed framework across its own architectural variations—including different covariance-matrix estimators and Mixture-of-Gaussian variants—as well as against state-of-the-art baselines.
- To conduct an in-depth empirical analysis of the system's performance, measuring the effect of upstream backbone representations, progressive class expansion, confidence calibration metrics, and downstream computational cost.
- To statistically validate the underlying assumptions of the framework by analyzing the dataset representations within the feature extractor embedding space, specifically assessing the Gaussianity of class-conditional feature distributions, evaluating their spectral structure, and quantifying inter-class separability to identify the sources of residual classification error.

1.5 Contributions

The principal contributions of this thesis are the following:

1. A gradient-free class-incremental learning framework that pairs a frozen DINOv2 backbone with independent, closed-form class-conditional Gaussian models, eliminating

catastrophic forgetting by construction rather than by mitigation, together with a systematic evaluation of five covariance parameterisations (diagonal, full, hybrid, shared, and shrinkage-based) and their mixture-of-Gaussian extensions.

2. A rigorous, multi-layered statistical validation of the Gaussianity assumption underlying the framework—combining moment-based diagnostics, the D'Agostino–Pearson omnibus test, and a diagonal adaptation of the Henze–Zirkler test for both univariate and multivariate normality under severe rank deficiency—that had not previously been carried out for foundation-model embedding spaces at this level of detail.

3. A quantitative link between inter-class separability, measured via the pairwise Bhattacharyya distance, and the specific, named class pairs that account for the residual classification error of the proposed classifier, reframing that error as a property of the embedding space rather than of the classifier itself.

1.6 Thesis Structure

The remainder of the document is organised as follows.

Chapter 2 – Continual Learning and Foundation Models. Covers the background needed to situate this work: the basics of artificial intelligence, machine learning, and deep learning applied to object classification, then continual learning itself, including catastrophic forgetting, the stability–plasticity dilemma, and the main families of existing solutions (rehearsal-, regularisation-, architecture-, and prompt-based methods, plus the classifier-adaptation and class-prototype approaches closest to what is proposed here). It closes with the Vision Transformer and the DINOv2 backbone used throughout this thesis.

Chapter 3 – Statistical and Probabilistic Foundations of the Feature Space. Lays out the mathematical and statistical tools the framework depends on: Bayesian classification and the Maximum A Posteriori rule, the Mixture-of-Gaussians model, and covariance estimation in the high-dimensional, small-sample regime typical of foundation-model embeddings. It also introduces the spectral tools used to describe the intrinsic dimensionality of the feature space, the tests used later to check the Gaussianity assumption, the Bhattacharyya and Euclidean separability metrics, and the confidence and calibration metrics used to judge how trustworthy the predictions are.

Chapter 4 – Modelling Class Densities in a Frozen Feature Space. Presents the proposed architecture in detail: a frozen encoder paired with independent, class-conditional Gaussian models updated in closed form, followed by the full family of covariance estimators evaluated in this thesis (diagonal, full, hybrid, OAS, Ledoit-Wolf, and shared covariance) and their Mixture-of-Gaussians extensions.

Chapter 5 – Experimental Results and Analysis. Reports the experimental protocols and the resulting analysis: the datasets and evaluation protocol, calibration and covariance-estimator performance across benchmarks, accuracy as classes are added incrementally, inter-class separability and the specific class pairs behind the residual error, sensitivity to the choice of backbone, the spectral structure of the DINOv2 embedding space, and the empirical validation of the Gaussianity assumption. Across every one of these analyses, MOG4-Shared is evaluated alongside the rest of the estimators and identified as the strongest overall configuration once accuracy, calibration, and computational cost are all taken into account.

Chapter 6 – Conclusion. Draws together the central findings of the thesis, discusses the limitations of the current framework, and outlines directions for future work that follow directly from those limitations.

Chapter 2

Continual Learning and Foundation Models

Continual Learning (CL) is the central problem this thesis addresses: how can a visual recognition system keep learning new object categories over time without catastrophically forgetting the ones it already knows? Answering this question rigorously, however, first requires a shared vocabulary. Before discussing what makes forgetting happen, what has been tried to prevent it, and how this thesis proposes to avoid it altogether, we must establish the fundamental principles of Artificial Intelligence, Machine Learning, and Deep Learning on which every subsequent argument rests, closing this introductory background with Computer Vision and its Object Classification task—the specific setting, including a first, necessarily brief, encounter with the Transformer architecture, in which the Continual Learning problem studied in this thesis is framed.

With this vocabulary in place, we return to Continual Learning and examine it in the depth the central problem of this thesis demands: the challenge of catastrophic forgetting, the stability–plasticity dilemma it creates, the main paradigms used to formulate it, and a review of the state-of-the-art mitigation strategies that provide both a performance baseline and the motivation for this thesis. This review then gives way, fluidly, to the representation-learning backbone that the proposed framework builds upon: Vision Transformers and, in particular, the self-supervised DINOv2 architecture.

2.1 Artificial Intelligence and Machine Learning

Artificial Intelligence (AI) is a broad field of computer science dedicated to creating systems capable of performing tasks that typically require human intelligence, such as learning, reasoning, perception, and language understanding [16]. Within this domain, Machine Learning (ML) focuses on developing algorithms that allow computers to learn patterns from data without explicit programming. ML systems are generally categorized into three paradigms: Supervised Learning, Unsupervised Learning, and Reinforcement Learning [17].

First, in **Supervised Learning**, the model learns a mapping from input variables $\mathbf{x}$ to output variables y based on a labeled dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N}$. The objective is to minimize a loss function that penalizes the discrepancy between the predicted output and the ground truth. This paradigm is typically divided into Classification, where the output is a discrete category, and Regression, where the output is a continuous value.

Second, **Unsupervised Learning** deals with unlabeled data $\mathcal{D} = \{\mathbf{x}_i\}_{i=1}^{N}$. Instead of predicting a target, the goal is to discover the underlying structure or distribution of the data. Common tasks include Clustering, which groups similar data points (e.g., K-Means or Gaussian Mixture Models), and Dimensionality Reduction, which projects high-dimensional data into a lower-dimensional space while preserving essential information (e.g., PCA or t-SNE).

Finally, **Reinforcement Learning** is based on an agent interacting with an environment to maximize a cumulative reward. Unlike supervised learning, the agent is not told which actions to take but must discover them through trial and error. This process is often modeled as a Markov Decision Process (MDP) [18], where the agent transitions between states based on actions, receiving feedback in the form of numerical rewards. The focus is on finding an optimal policy that maximizes long-term return, a process that requires balancing exploration (trying new actions) and exploitation (capitalizing on known high-reward actions) during learning.

2.1.1 Deep Learning

In traditional Machine Learning, the feature extraction step is a manual process where a human expert must identify and define the most relevant characteristics (such as the shape or edges of the car) before passing them to a classification algorithm. In contrast, as shown in figure 2.1, Deep Learning utilizes neural networks to automate this process; the model performs both feature extraction and classification simultaneously, learning to identify complex patterns directly from the input data without human intervention.

This shift allows Deep Learning to handle much more complex, high-dimensional data, though it typically requires significantly larger datasets and more computational power to be effective. While traditional Machine Learning depends on the quality of hand-crafted features, Deep Learning architectures—like multi-layered neural networks—are designed to build hierarchical representations of data, progressively moving from low-level pixels to high-level object concepts.

2.1.2 Computer Vision

Computer Vision is the interdisciplinary field that enables computers to gain high-level understanding from digital images or videos, effectively automating tasks performed by the human visual system [19]. While Convolutional Neural Networks (CNNs) [20] historically dominated this field, the Transformer architecture—originally developed for Natural Language Processing—has revolutionized visual recognition through the self-attention mechanism [21].

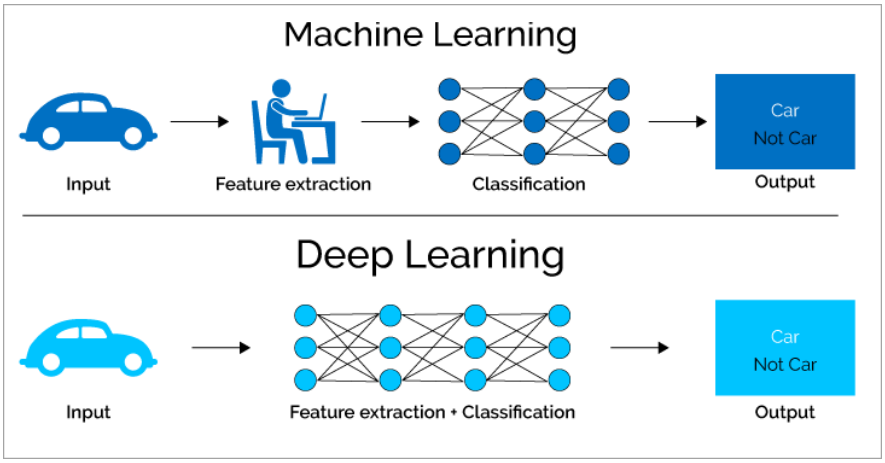


Figure 2.1: Machine Learning and Deep Learning Strategies. Image taken from [1].

Among the many tasks studied in Computer Vision, **Object Classification** is one of the most fundamental: it consists of assigning a single categorical label to an entire input image, and it serves as the primary benchmark for evaluating the representative power of visual models [22]. This task is the setting adopted throughout this thesis: the Continual Learning problem discussed next is studied specifically as a sequence of object classification tasks, in which the model must progressively learn to assign incoming images to a growing set of object categories.

Having established this general background in Artificial Intelligence, Machine Learning, Deep Learning, and Computer Vision—including a first glimpse of the Transformer architecture that will reappear in depth later in this chapter—we now turn to the specific paradigm around which this thesis is organized: Continual Learning, studied here within the object classification setting.

2.2 Continual Learning

Continual Learning (CL), also known as lifelong learning, addresses the problem of progressively acquiring knowledge from non-stationary data streams over time. In object classification, this setup presents data as a sequence of disjoint tasks, where each task introduces a new set of object categories. The primary challenge is enabling a model to learn these novel categories incrementally without degrading its classification performance on previously acquired ones. As illustrated in Figure 2.2, a **task** in this context a is defined as a specific learning session or split—such as Task A, B, or C—each introducing distinct class-conditional data distributions, symbolized by the varying shapes (e.g., circles vs. hearts) above the task labels.

Unlike conventional training approaches that rely on a fixed dataset and a stationary environment, CL emphasizes the arrival of these tasks at distinct intervals along the **time** axis. This sequential arrival requires the model to be inherently non-stationary, adapting its internal representations as environments evolve. The primary challenge in this paradigm

lies in the ongoing integration of new information from a current task (e.g., Task B) while simultaneously preserving the knowledge previously acquired from Task A.

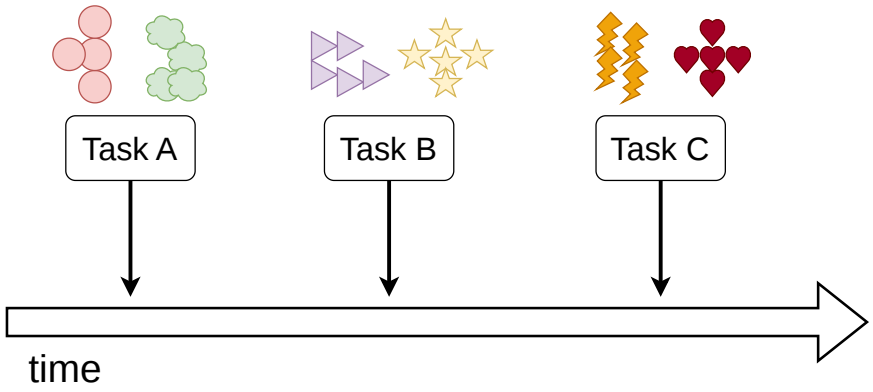


Figure 2.2: A sequential task arrival model for Continual Learning, depicting the transition from Task A to Task C over time, where each task represents a unique set of features or data distributions.

Without specific architectural or algorithmic safeguards, this sequential learning process inevitably triggers **catastrophic forgetting** [10]. This phenomenon occurs when the assimilation of new knowledge leads to the degradation or total loss of prior learning. As illustrated in Figure 2.3, the dilemma of forgetting becomes apparent when a *New Object Class* is introduced to an existing classifier, forcing a choice between two traditional but flawed strategies:

- **Fine-Tuning:** Updating the model using only the current task data is computationally efficient but leads directly to catastrophic forgetting [10]. Because the optimization process focuses solely on the new objective, the internal representations associated with previously learned classes are overwritten, causing the model to lose its original predictive capabilities.
- **Retraining from Scratch:** While joint training on the full historical dataset ($\mathcal{D}_{old} \cup \mathcal{D}_{new}$) technically prevents forgetting, it faces two significant hurdles. First is the issue of **Data Privacy and Availability**: in many real-world scenarios, the original data $\mathcal{D}_{old}$ is often deleted or inaccessible due to privacy regulations. Second is the **Computational Bottleneck**: as the system's lifespan increases, the cumulative dataset grows, making the energy and time required to retrain the model unsustainable in continuously evolving environments.

2.2.1 Stability and Plasticity

A central concept in continual learning is the balance between stability and plasticity. **Stability** emphasizes preserving the parameters and representations that are crucial for retaining old knowledge [9]. **Plasticity**, in contrast, refers to the ability of a model to adapt rapidly to

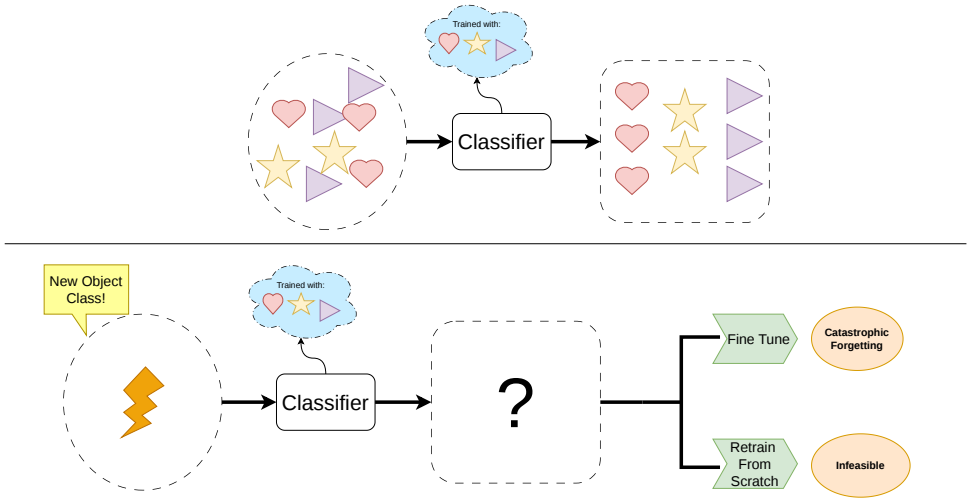


Figure 2.3: Standard Object Classifier comparison. The upper path shows standard inference, while the lower path illustrates the dilemma when a new class is introduced: the trade-off between catastrophic forgetting via fine-tuning and the infeasibility of retraining from scratch.

new information, ensuring that it can effectively learn novel tasks. Achieving an appropriate equilibrium between these two properties is known as the **stability–plasticity dilemma** [10].

2.2.2 Main Paradigms

As shown in Figure 2.4, two principal paradigms define how continual learning problems are formulated. **Task-incremental learning (Task-IL)** assumes that the task identity is available during inference, allowing for task-specific components to be isolated [23]. **Class-incremental learning (Class-IL)**, on the other hand, does not provide the task identity at test time, requiring the model to distinguish between all classes seen across all tasks simultaneously [2].

2.2.3 Related Work in Continual Learning

To address the stability–plasticity dilemma, we categorize continual learning strategies into five broad approaches: rehearsal, regularization, architecture-based methods, representation and prompt-based tuning, and classifier adaptation over frozen representations. Classifier adaptation—both its discriminative and analytic, prototype-based variants—is the approach most directly comparable to the framework proposed in this thesis, so it's the one we treat as the closest point of reference.

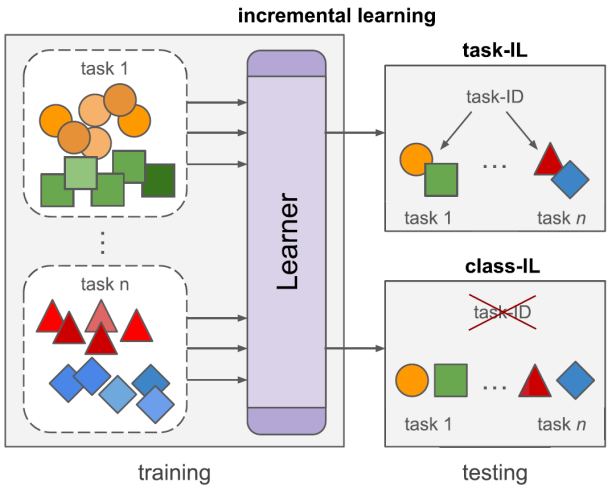


Figure 2.4: Class and Task Incremental Learning. Image taken from [2].

Rehearsal Methods

Rehearsal-based methods target the most direct cause of catastrophic forgetting: once training on a new task starts, there is no gradient signal left telling the model anything about what it already learned. So they mix a handful of past examples into the current task's training data—sometimes these are just the original images sitting in a small buffer, sometimes they are synthetic examples that a separate generator recreates on the fly instead of storing anything real—keeping the model reminded of the old classes even while it is busy learning the new ones. The real design question is what to keep and how to use it efficiently, since a memory buffer with no limits would defeat the whole point of a learner that has to run with limited memory and storage.

- **Experience Replay (ER)** [24] is the simplest instantiation of this idea: a small episodic memory buffer retains a random or reservoir-sampled subset of past examples, which are interleaved with the current task's mini-batches so that every gradient step still receives some supervision on old classes. Its simplicity makes it a common and surprisingly strong baseline, but its effectiveness scales with buffer size, and the fixed capacity inevitably biases the retained sample toward whichever selection heuristic is used.
- **Average Gradient Episodic Memory (A-GEM)** [25] reframes the buffer's role from a source of additional gradient signal to a constraint on the optimisation trajectory itself: rather than training jointly on old and new samples, it projects the current task's gradient so that it does not increase the loss computed on a reference batch drawn from the buffer, which is considerably cheaper than the original GEM formulation while retaining most of its forgetting-mitigation benefits.
- **Dark Experience Replay (DER)** [26] enriches the stored exemplars with the network's own past output logits rather than only their hard labels, and distils this "dark knowledge"—the full soft-label distribution encoding inter-class similarity structure learned at the time of storage—back into the current model. This provides a denser

training signal per replayed sample than label-only rehearsal, at the cost of needing to persist and periodically re-align these logits with a model that is continually changing.

- **Generative Replay** [27] sidesteps the need to store raw exemplars altogether by training a generative model (or, in feature-space variants, a feature synthesiser) to reproduce samples of previously seen classes on demand. This removes the raw-data privacy liability of exemplar buffers, but transfers the difficulty elsewhere: the generator itself must be continually updated without forgetting, and any drift in the quality of its synthetic samples directly corrupts the rehearsal signal for old classes.

Across all four variants, the shared limitation is structural rather than incidental: rehearsal methods require either persistent storage of raw or compressed past data, or a generative model that is itself subject to the same forgetting dynamics it is meant to prevent, and their forgetting-mitigation quality degrades gracefully but monotonically as the buffer-to-task-count ratio shrinks over a long task stream.

Regularization Methods

Rather than revisiting old data, regularization-based methods attempt to identify, at the parameter level, which weights of the network are responsible for previously learned behaviour, and then constrain how much those specific weights are allowed to move while training on new tasks. The stability–plasticity trade-off is therefore made explicit as a per-parameter penalty: important parameters are frozen or heavily penalized for movement, while the remaining capacity of the network is left free to absorb new information.

- **Elastic Weight Consolidation (EWC)** [28] estimates parameter importance using the diagonal of the Fisher Information Matrix computed on each task, then adds a quadratic penalty to the loss that discourages movement away from the previous task's optimum in proportion to that importance. The approach is elegant and grounded in a Bayesian interpretation of sequential posterior updates, but the diagonal Fisher approximation ignores parameter interactions, and the accumulated quadratic penalties from many tasks can over-constrain the network, progressively eroding its plasticity.
- **Synaptic Intelligence (SI)** [29] computes an online, per-synapse importance measure by accumulating each parameter's contribution to the reduction of the loss along the entire training trajectory of a task, avoiding the need for a separate importance-estimation pass after each task and making the method considerably cheaper than EWC in practice, though it inherits a similar long-horizon rigidity problem.
- **Learning Without Forgetting (LwF)** [30] regularises the network's function rather than its parameters directly: before training on a new task, the previous model's predictions on the new task's inputs are recorded, and a knowledge-distillation loss then penalises deviations of the updated model's outputs from these recorded soft targets. This makes LwF architecture-agnostic and exemplar-free, but its effectiveness depends on the new task's inputs being informative enough to anchor the old decision boundaries, which becomes unreliable under large domain shifts between tasks.

- **Adaptive Plasticity Improvement (API)** [9] takes a more geometric view, using Dual Gradient Projection Memory (Dual-GPM) to identify the subspaces of parameter (or gradient) space that were important for previous tasks and enforcing that new updates proceed in directions orthogonal to them. This directly targets the stability side of the dilemma by construction, but the number of protected subspaces necessarily grows with the number of tasks, gradually shrinking the remaining free subspace available for future plasticity.

The common thread across this family is that importance estimates—whether Fisher-based, synaptic, distillation-based, or subspace-based—are necessarily approximate and computed with respect to the tasks seen so far; as the task stream lengthens, the accumulated constraints compete for a fixed parameter budget, and all four methods must trade off increasingly aggressive stability against a shrinking capacity for plasticity.

Architecture-based Methods

A third family sidesteps the stability–plasticity trade-off at the parameter level altogether by modifying the network's structure itself, either by isolating the sub-networks responsible for each task or by physically growing the network's capacity as new tasks arrive.

- **Parameter Isolation** [31] identifies and freezes the specific parameters or sub-networks important for previous tasks—typically through pruning or learned gradient masks—so that only a designated, disjoint subset of weights remains trainable for the current task. This offers a strong, structural guarantee against forgetting for the frozen parameters, but the available free capacity shrinks monotonically as more tasks are isolated, and, in many instantiations, recovering which sub-network to use at inference time requires knowledge of the task identity.
- **Dynamic Expansion** methods take a more literal approach: instead of freezing parts of a fixed-size network, they let the network itself grow. DER [32] (Dynamically Expandable Representation, not to be confused with Dark Experience Replay above) keeps everything learned so far completely frozen and, for each new task, bolts on a brand-new trainable branch next to it; the outputs of all branches get concatenated into one large "super-feature", and a pruning step afterward trims away whatever redundant channels that new branch didn't really need. DEN [33] (Dynamically Expandable Networks) is more conservative: rather than always adding a new branch, it first tries to fit the new task using the capacity that's already there, and only grows the network—adding new units—when the training loss shows that existing capacity genuinely isn't enough. Both approaches guarantee that old sub-networks remain untouched and hence immune to forgetting by construction, but at the cost of a parameter count, and consequently a memory and computational footprint, that grows over the lifetime of the system—an increasingly serious limitation for long task streams.

Whether through isolation or expansion, architecture-based methods trade the risk of interference for an explicit, often unbounded, growth in model complexity, and frequently

still require task identity at inference to route each input to the correct sub-network—a requirement that is unavailable in the class-incremental setting studied throughout this thesis.

Representation and Prompt-based Methods

The rise of foundation models (e.g., ViTs, described in detail in Section 2.3) opened up a different strategy altogether: instead of touching the backbone's weights at all, these methods leave it completely frozen and confine all task-specific learning to a small set of extra parameters called *prompts*. Concretely, a prompt is a short sequence of learnable embedding vectors that gets prepended to, or otherwise inserted into, the sequence of patch tokens the Transformer already processes for a given image; the backbone treats these vectors exactly as it would treat any other token, so they never change a single one of the network's own weights, but they do change what the network attends to and, through that, the features it eventually outputs for that image. Since the backbone itself never changes, forgetting at the representation level cannot happen by construction; the real challenge shifts to designing prompts expressive enough to adapt the frozen features to each task, without ever telling the model which task it is currently looking at.

- **Learning to Prompt (L2P)** [34] keeps a pool of learnable prompts and, for each input, picks out and combines the ones that best match it: every prompt has an associated key vector, and the input's own embedding is compared against these keys to decide which prompts to use. Because this selection is done per input rather than per task, the same shared pool can serve every task without ever needing to know which one is currently active; the trade-off is that the whole approach now depends on that matching step working correctly, and a wrong match sends the input to prompts meant for entirely different content.
- **Dual Prompt** [35] builds on this by splitting the prompt pool into two clearly different roles: a set of fixed, shared *General* prompts that capture knowledge common to all tasks, and task-specific *Expert* prompts that specialize the representation for the classes belonging to one particular task. This division gives a better balance between remembering old tasks and learning new ones than L2P's single undifferentiated pool, though it still needs the same kind of key-based matching to pick out the right Expert prompt at inference time.
- **CODA-Prompt** [36] does away with that hard, similarity-based matching step entirely. Instead of picking a discrete subset of prompts, it combines all of them through an attention-weighted average, so the whole prompting mechanism can be trained end-to-end together with the classifier, rather than depending on a separately tuned matching heuristic.
- **Advanced Prompting** extensions carry this same idea into more efficient and distributed settings: **InFLoRA** [37] and **DOLFIN** [38] apply low-rank adaptation and distributed training, respectively, to cut down the parameter cost of maintaining a prompt pool, while **APT** [39] skips prompt selection altogether by injecting key-value tokens directly into the transformer's attention layers, which also reduces the computational cost at inference time.

Across this family, freezing the backbone rules out representation-level forgetting almost by definition, but everything then depends on how well the prompt-selection mechanism works. That mechanism brings its own extra parameters and its own training difficulties, and because prompts are usually learned by backpropagating through the frozen backbone, it also adds a computational cost per task that the purely analytic methods discussed next manage to avoid entirely.

Classifier Adaptation on Frozen Representations

A complementary line of work also keeps the feature extractor frozen, just like prompt-based methods do, but skips learnable prompts entirely: the frozen embedding is simply taken as-is, and all the adaptation happens in a discriminative or metric-based classifier built directly on top of it, with no backpropagation through the backbone at all.

- **SLCA** [40] proposes a slow learner with classifier alignment for continual learning on pre-trained models: the backbone is fine-tuned with a substantially reduced learning rate to preserve its pre-trained generality, while a separate classifier head is periodically re-aligned to correct for the resulting representation drift, enabling efficient incremental learning without the full instability of end-to-end fine-tuning.
- **ACIL** [41] treats incremental learning analytically: following an initial base phase where the CNN backbone is trained via backpropagation and subsequently frozen, it projects the extracted embeddings into a higher-dimensional space via a randomized feature expansion layer with non-linear activations. A linear classifier is then updated over these expanded features using recursive regularized least-squares, which provably reproduces the joint-training closed-form solution over all observed classes without revisiting past data, ensuring absolute memorization and data privacy without exemplar storage.
- **FeCAM** [42] exploits the heterogeneity of class distributions in this exemplar-free setting by replacing the Euclidean or purely linear decision rules of the methods above with a Mahalanobis-distance-based classifier, explicitly accounting for the fact that different classes occupy differently shaped regions of the frozen embedding space.

Because the backbone is never updated, none of these methods suffers representation-level forgetting, and—unlike prompt-based methods—none of them requires backpropagation at all during incremental updates; the price is that their classifiers rely on comparatively simple statistical or geometric assumptions about the embedding space, which the next family of methods begins to make explicit.

Class Prototype and Analytic Classifier Methods

A closely related family of methods represents each class through distances to class prototypes (first-order information), sometimes complemented by second-order statistics, and is the family most directly comparable to the framework proposed in this thesis.

- **SimpleCIL** [43] performs first-session adaptation by applying parameter-efficient fine-tuning (PEFT) to the backbone only during the initial task, after which the backbone is frozen for the remainder of the stream and class prototypes are computed simply as the mean embedding of the samples belonging to each class. Its simplicity and strong empirical performance on foundation-model embeddings established that even a purely first-order, distance-to-mean classifier can be a remarkably strong baseline once the backbone is sufficiently general.
- **RanPAC** [44] improves upon this prototype-based scheme by first applying a fixed random projection followed by a non-linear activation, lifting the features into a higher-dimensional space in which classes become more linearly separable by construction (a random-feature approximation of a kernel expansion), and then accumulating second-order information through a shared, incrementally updated Gram matrix across all classes rather than per-class covariances.
- **FeTrIL** [45] stores only the centroid representation of each past class rather than any raw exemplars or full covariance matrices. When a new task arrives, features of the new classes are geometrically shifted toward past class regions using the difference between past and new class centroids; the resulting pseudo-features are then used alongside real new-class features to train a linear classifier incrementally.
- **FeCAM** [42], already introduced above, is more naturally placed in this family for its second-order treatment: it explores Mahalanobis distances instead of typical Euclidean distances, applying a Tukey power-ladder transformation to correct feature skewness before normalising each class covariance so that its diagonal entries equal one, making Mahalanobis distances comparable across classes despite never validating that the underlying feature distributions justify a Gaussian, Mahalanobis-based treatment in the first place.

Across this family, representing a class through its prototype brings real efficiency, but that efficiency comes at a representational cost: first-order methods such as SimpleCIL and FeTrIL reduce a class to a single point in the embedding space, discarding the covariance information that describes how the class's features actually spread around that point, while RanPAC and FeCAM recover richer, second-order structure without a principled probabilistic model to justify the specific distance or normalization they apply, and neither checks whether treating class covariance this way is consistent with how the class distributions actually look.

Summary of Limitations. Across all five families, the same pattern recurs: each avoids one part of the stability–plasticity trade-off only by pushing its cost elsewhere—onto a growing memory or generator, a hardening set of importance constraints, an unbounded architecture, a fragile prompt-selection mechanism, or a classifier built on first-order or heuristic statistics rather than a proper probabilistic model. None of them checks whether the Gaussian assumption behind their distance metrics actually holds in the embedding space, or ties inter-class separability directly to the accuracy that can realistically be expected.

These limitations motivate a framework that (i) avoids exemplar storage, generator training, growing architectures, and backpropagation altogether by keeping the backbone entirely

frozen and updating only closed-form sufficient statistics; (ii) models each class with a full, properly normalized multivariate Gaussian density rather than a heuristic distance metric, restoring a direct probabilistic and generative interpretation; and (iii) explicitly and statistically validates the Gaussian assumption on which this generative model rests, rather than simply asserting it.

2.3 Foundation Models

Having introduced Continual Learning as the central topic of this thesis, we now turn to a different piece of background this framework depends on: foundation models, the broader category of models the representation-learning backbone belongs to, and in particular Vision Transformers and the DINOv2 architecture used as the frozen feature extractor throughout this work. Before getting into the details of these two architectures, though, it is worth stepping back and asking what a foundation model actually is, and why a model of that kind is exactly what a frozen, exemplar-free continual learner needs.

2.3.1 The Foundation Model Paradigm

A **foundation model** is best understood by the role it plays rather than by any single architecture: it is a large-scale model pre-trained on vast quantities of data and subsequently adaptable to a wide range of downstream tasks with comparatively little task-specific effort. The term itself was coined by researchers at the Stanford Institute for Human-Centered AI precisely to capture this role: these models are not built for one job, but act as a common technological base on top of which many specialised applications are eventually constructed [46].

What really changes compared to a traditional machine learning pipeline is where the effort goes. Instead of training a task-specific architecture from scratch on a carefully generated by hand dataset, foundation models split the work into two clearly separated phases. The first, *pre-training at scale*, exposes the network to huge amounts of data—web text, image collections, or multimodal data, depending on the domain—often, though not always, using self-supervised objectives such as predicting masked tokens or learning contrastive or self-distilled representations, which keep manual annotation to a minimum. The network just picks up rich, general-purpose structure about whatever domain it is trained on. The second phase, *downstream adaptation*, takes this pre-trained network—frozen entirely or updated only slightly—and repurposes it for a specific target task through comparatively lightweight means: linear probing, parameter-efficient fine-tuning, or even zero-shot prompting, without ever retraining the backbone from scratch.

This two-phase recipe works across pretty much any type of data, largely because most foundation models, whatever they are trained on, share the same underlying architecture: the Transformer. In language, this gives rise to models such as GPT, Llama, and Gemini, which learn how words and phrases relate to each other well enough to support reasoning, translation, and text generation. In vision, the same recipe produces Vision Transformers

and segmentation models such as the Segment Anything Model (SAM) [47], which build detailed spatial and visual representations that carry over to detection, segmentation, and classification tasks. And across different types of data at once, systems such as CLIP or GPT-4o map images, text, and audio into a single shared space, letting them reason across all of them together.

2.3.2 Vision Transformers

Within this landscape, Vision Transformers are the specific architecture this thesis builds on. As introduced in Section 2.1.2, the Transformer—originally developed for Natural Language Processing—changed visual recognition by relying on self-attention [21], a mechanism that lets every element of the input attend directly to every other element, and this challenged the long-standing dominance of Convolutional Neural Networks (CNNs) [20] in Computer Vision.

A Vision Transformer (ViT) adapts this architecture to images by treating them as a sequence of discrete tokens, the same kind of input a Transformer would otherwise expect from a sentence of words. As illustrated in Figure 2.5, this is done through a specific pipeline:

- **Patch Partitioning:** The input image is divided into a grid of fixed-size patches, each of which becomes one token in the sequence.
- **Linear Projection:** Each patch is flattened into a vector and projected into an embedding space, giving every token a numerical representation the network can operate on.
- **Positional Encoding:** Since self-attention treats the sequence as an unordered set, it has no inherent notion of where each patch sat in the original image; a learnable positional embedding is therefore added to each patch embedding to encode its location in the grid.
- **Transformer Encoder:** The resulting sequence of position-aware patch tokens is processed by several stacked layers of multi-head self-attention and feed-forward blocks, which allow the model to relate tokens from anywhere in the image to one another and build up a global representation.

Object Classification and Representative Power

The original ViT was designed and evaluated primarily on Object Classification, the task introduced in Section 2.1.2 that also serves as the setting for the Continual Learning problem studied in this thesis. By attending over the full sequence of patch tokens rather than a local receptive field, ViTs have shown they can match or exceed CNN performance on this task, particularly when pre-trained on the kind of large-scale datasets described below.

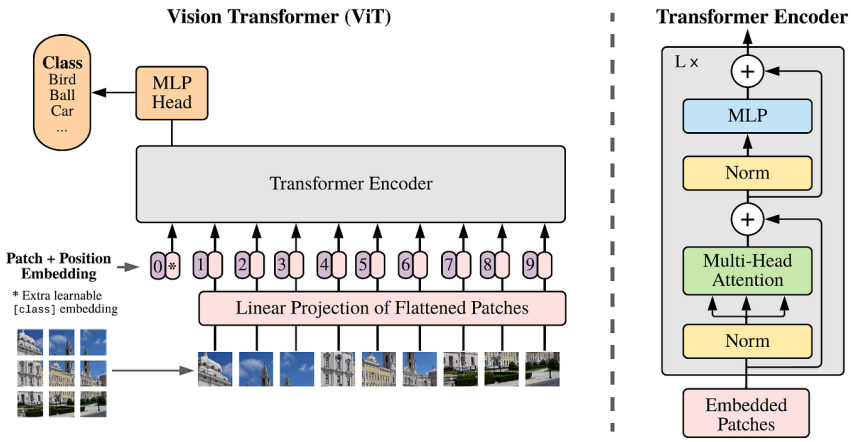


Figure 2.5: Vision Transformer (ViT) architecture overview. The model processes an image as a sequence of patches through a standard Transformer encoder. Image taken from [3].

Self-Supervised Advancements: DINOv2

Most vision datasets are still built around labels assigned by hand, but recent work has shifted toward foundation models trained via self-supervised learning instead, following the same general pre-training paradigm described earlier in this chapter: no human-provided labels are used at all. A leading example of this approach is **DINOv2** [12], which produces visual features that are robust and general enough to perform well "out of the box," both at the level of whole-image descriptions and at the level of individual patch tokens.

DINOv2 is built around a **Student-Teacher self-distillation** framework, extending earlier work on DINO [48] and iBOT [49]. Two networks of identical architecture are trained in parallel: a *Student* network g_{θ_s}, whose weights are updated by gradient descent as usual, and a *Teacher* network g_{θ_t}, which is never trained directly and instead tracks the student as an Exponential Moving Average (EMA) of its weights θ_s:

$$\theta_t \leftarrow \lambda\theta_t + (1-\lambda)\theta_s \tag{2.1}$$

where $\lambda \in [0, 1]$ is a smoothing parameter controlling how slowly the teacher follows the student. During training, the framework optimizes two complementary objectives to learn both global and fine-grained spatial representations:

- **Image-Level Loss (DINO):** The student is fed both global and local crops of the same image, while the teacher only ever processes global views. The student is trained to match the teacher's output distribution over a set of learned prototype vectors, an online *Sinkhorn-Knopp centering* step keeping this matching balanced so that the representations do not collapse to a single trivial solution.
- **Patch-Level Loss (iBOT):** To force the model to capture dense, local structure rather than only a global summary, a subset of the student's input patch tokens are masked out. The student must then reconstruct the teacher's corresponding patch-level features, computed from the same image without any masking.

- **KoLeo Regularization:** To keep the resulting embeddings spread out across the feature space rather than collapsing into a narrow region, DINOv2 adds a term based on the Kozachenko-Leonenko (KoLeo) differential entropy estimator, which penalizes feature vectors for lying too close to their nearest neighbor within a training batch:

$$\mathcal{L}_{\text{KoLeo}} = -\frac{1}{n}\sum_{i=1}^{n}\log\left(\min_{j\neq i}\|\vec{x}_i - \vec{x}_j\|_2\right) \tag{2.2}$$

  where $\vec{x}_i$ is the ℓ_2-normalized feature vector of the i-th image in a batch of size n.

Chapter 3

Statistical and Probabilistic Foundations of the Feature Space

The framework proposed in this thesis rests on a single working hypothesis: that the class-conditional feature distributions produced by a frozen visual foundation model are well approximated by multivariate Gaussians, and that the resulting decision surfaces can be described in terms of the geometry of the embedding space they live in. This chapter puts together, in one self-contained treatment, the mathematical and statistical tools used throughout this thesis to build and check that hypothesis. We start with the probabilistic modelling framework itself—Bayesian inference, the Maximum A Posteriori decision rule, and its Mixture-of-Gaussians extension. We then turn directly to the Gaussianity assumption these models depend on, presenting the moment-based diagnostics and the formal hypothesis tests, univariate first and then multivariate, including the Henze–Zirkler statistic and its diagonal adaptation. With that assumption in place, we introduce the inter-class separability metrics—the probabilistic Bhattacharyya distance and the geometric Euclidean distance—used to quantify how classes crowd the embedding space as new tasks arrive. We then explain why estimating a covariance matrix in this setting is difficult in the first place, a difficulty commonly known as the curse of dimensionality, and how it can be addressed through regularization; this naturally leads into the spectral tools—Principal Component Analysis and the associated eigenvalue-decay diagnostics—used to describe the intrinsic dimensionality of the feature space and, in turn, to understand why that regularization is needed.

3.1 Foundations of Probabilistic Modeling

Representing each class as a Gaussian density only makes sense if there is a solid decision rule behind it that says how to actually use that density to classify a new observation. This section builds that rule from the ground up: starting from Bayes' theorem, we arrive at the Maximum A Posteriori (MAP) criterion, which tells us how to pick the most likely class given the observed features, and then extend it to mixtures of Gaussians for classes whose features are better described by more than one cluster.

3.1.1 Bayes Classification

At its core, Bayes classification is rooted in **Bayes' Theorem**, which provides a mathematical framework for updating the probability of a hypothesis based on observed evidence. For a classification task, we seek the posterior probability $p(y = c \mid \vec{x})$—the probability that an observation belongs to class c given its observed feature vector $\vec{x}$.

Bayes' Theorem is defined as:

$$p(y = c \mid \vec{x}) = \frac{p(\vec{x} \mid y = c)\, p(y = c)}{p(\vec{x})}, \tag{3.1}$$

where

- $p(y = c \mid \vec{x})$ is the **posterior** probability.
- $p(\vec{x} \mid y = c)$ is the **likelihood** of the data given the class.
- $p(y = c)$ is the **prior** probability of the class.
- $p(\vec{x})$ is the **evidence** or marginal likelihood, acting as a normalization constant.

The objective of a Bayes classifier is to minimize the Bayes risk; in the simplest case, this corresponds to assigning the observation to the class that maximizes the posterior probability.

3.1.2 Likelihood-Based Classification

Likelihood inference provides a coherent framework for updating the probability of a hypothesis as more evidence or information becomes available. In contrast to frequentist approaches that treat parameters as fixed but unknown constants, the Bayesian paradigm treats parameters as random variables characterized by probability distributions.

The connection between Bayes' Theorem and likelihood-based classification lies in the **Maximum A Posteriori (MAP)** decision rule. Since the evidence $p(\vec{x})$ is constant across all classes for a given observation, the classification decision depends solely on the product of the likelihood and the prior:

$$p(y = c \mid \vec{x}) \propto p(\vec{x} \mid y = c)\, p(y = c). \tag{3.2}$$

A generative modeling framework can be adopted and class-conditional Gaussian feature distributions can be assumed:

$$p(\vec{x} \mid y = c) = \mathcal{N}(\vec{x} \mid \boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c), \tag{3.3}$$

where $\vec{x} \in \mathbb{R}^d$ represents the feature vector, $\boldsymbol{\mu}_c$ is the class mean vector, and $\boldsymbol{\Sigma}_c \succ 0$ is the covariance matrix.

Classification can be performed via MAP inference by maximizing the log-posterior:

$$\hat{y} = \arg\max_{c \in \mathcal{Y}} \left[\log p(\vec{x} \mid y = c) + \log \pi_c \right], \tag{3.4}$$

where $\pi_c = p(y = c)$ denotes the class prior.

Substituting the multivariate Gaussian density into the MAP rule, the decision rule becomes equivalent to:

$$\hat{y} = \arg\min_{c} \left[(\vec{x} - \boldsymbol{\mu}_c)^T \boldsymbol{\Sigma}_c^{-1} (\vec{x} - \boldsymbol{\mu}_c) + \log |\boldsymbol{\Sigma}_c| - 2 \log \pi_c \right]. \tag{3.5}$$

Mixture of Gaussians (MoG)

A Mixture of Gaussians models a complex probability density as a convex combination of K distinct Gaussian components. This approach is fundamental for clustering and density estimation in multi-modal distributions. Therefore, to capture multimodal feature distributions, the class-conditional model can be extended to a mixture:

$$p(\vec{x} \mid y = c) = \sum_{k=1}^{K_c} \alpha_{c,k} \mathcal{N}(\vec{x} \mid \boldsymbol{\mu}_{c,k}, \boldsymbol{\Sigma}_{c,k}), \tag{3.6}$$

where $\alpha_{c,k} \geq 0$ and $\sum_k \alpha_{c,k} = 1$.

The MAP decision rule becomes

$$\hat{y} = \arg\max_{c} \left[\log \left(\sum_{k=1}^{K_c} \alpha_{c,k} \mathcal{N}(\vec{x} \mid \boldsymbol{\mu}_{c,k}, \boldsymbol{\Sigma}_{c,k}) \right) + \log \pi_c \right]. \tag{3.7}$$

This whole classification framework—from the plain Bayes rule to its Gaussian and mixture-of-Gaussians instantiations—is only as good as the assumption it is built on: that class-conditional features are actually well described by Gaussians in the first place. The remainder of this chapter puts that assumption to work. Section 3.2 tests it directly through moment-based diagnostics and formal hypothesis tests; Section 3.3 studies how the resulting Gaussians relate to one another as more classes are added; Section 3.4 addresses the practical difficulty of estimating a covariance matrix $\boldsymbol{\Sigma}_c$ in a high-dimensional, small-sample regime; and Section 3.5 closes the chapter by examining the geometric structure of the space these covariances live in.

3.2 Gaussianity Assumptions and Analysis

Gaussianity, in simple terms, is just how well a set of data points can be described by a normal distribution: a bell-shaped curve that, in one dimension, is fully defined by a mean and a variance, or, in many dimensions at once, by a mean vector and a covariance matrix. The

classifier proposed in this thesis works by fitting one such Gaussian to each class and then, for a new observation, comparing how likely that observation is under each of these class-specific Gaussians. This only makes sense if the features actually produced for each class really do look Gaussian. If they do not, the mean and covariance we estimate stop being a good description of the data, and the classifier can end up making confident, well-defined decisions that are simply wrong, even though the underlying math is correct. Checking Gaussianity before trusting the classifier is therefore not a formality: it tells us whether modelling each class as a single Gaussian is actually a reasonable choice for this particular feature space, rather than just a convenient one.

We check this assumption at two levels. We first look at *per-dimension* (univariate) Gaussianity: whether each individual coordinate of the feature vector, taken on its own, looks Gaussian. We then look at *per-class* (multivariate) Gaussianity: whether the full, high-dimensional feature vector of each class follows a joint Gaussian distribution.

3.2.1 Per-Dimension Gaussianity

We start at the simplest level: looking at each coordinate of the feature vector on its own, one at a time, and asking how close its distribution is to a normal curve. To verify Gaussianity at this per-dimension level, we combine two descriptive shape diagnostics with a formal statistical test, applying all of them to every (class c, dimension j) pair.

Shape moments.

Skewness and kurtosis are both examples of what statisticians call *moments*: numerical summaries computed from the data that describe the shape of its distribution, in the same way that the mean and variance are moments that describe its location and spread. Concretely, kurtosis and skewness describe the shape of a distribution:

The **kurtosis** measures the "tailedness" and peakedness of a probability distribution relative to a normal distribution [4]. As illustrated in Figure 3.1, a perfectly Gaussian distribution has an excess kurtosis of zero (mesokurtic). Distributions with high kurtosis (leptokurtic, $Kurtosis > 0$) exhibit sharper peaks and heavier tails, indicating a higher concentration of data around the mean and a greater likelihood of extreme outliers. Conversely, low kurtosis (platykurtic, $Kurtosis < 0$) indicates a flatter peak and thinner tails.

The **skewness** quantifies the asymmetry of a distribution around its mean. As shown in Figure 3.2, a symmetric distribution (center) is a fundamental assumption of the Gaussian model, implying that the mean $\boldsymbol{\mu}_c$, median, and mode coincide, providing an adequate representation of the class prototype within the embedding space. A positive skew (right) indicates a distribution with a long tail extending toward higher values, while a negative skew (left) indicates a long tail extending toward lower values. Significant skewness would suggest that the class-conditional mean is no longer a central representation of the feature distribution.

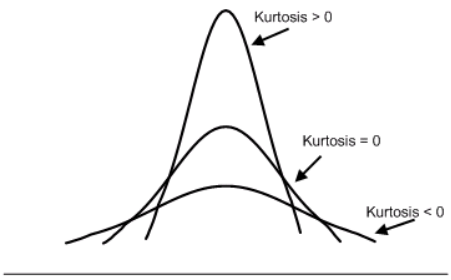


Figure 3.1: Visual representation of kurtosis levels: leptokurtic (> 0), mesokurtic ($= 0$) and platykurtic (< 0) distributions [4].

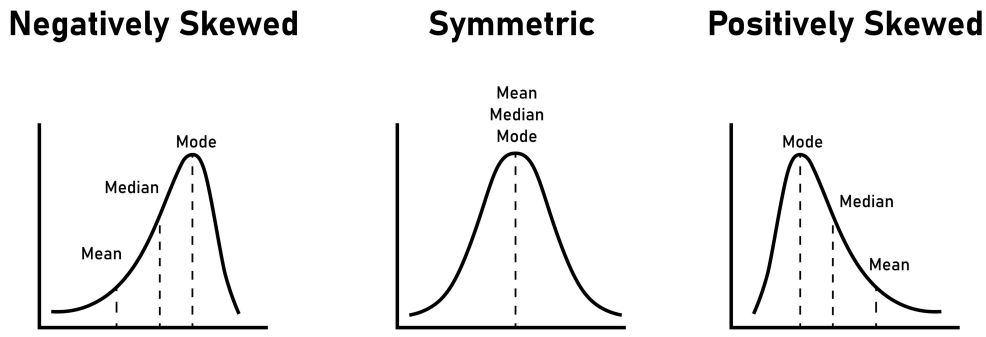


Figure 3.2: Comparison of negatively-skewed, symmetric and positively-skewed distributions [5].

Following the normality diagnostics of Pearson [5] and D'Agostino and Pearson [13, 14], we compute the sample skewness and excess kurtosis for each class c along each of the d coordinates:

$$\widehat{\mathrm{Skew}}_{c,j} \;=\; \frac{\frac{1}{n_c}\sum_i (x_{i,j}-\mu_{c,j})^3}{\hat{\sigma}^3_{c,j}}, \qquad \widehat{\mathrm{Kurt}}_{c,j} \;=\; \frac{\frac{1}{n_c}\sum_i (x_{i,j}-\mu_{c,j})^4}{\hat{\sigma}^4_{c,j}} - 3, \tag{3.8}$$

where both quantities are computed exactly, unlike the qualitative reading of Figures 3.1 and 3.2 above, and vanish exactly for a theoretical Gaussian distribution [5, 4].

Empirical evidence suggests that for many real-world datasets, both kurtosis and skewness typically fall within the range $[-1, 1]$, with values in that range indicating distributions close to symmetric and free of excessively heavy tails [50].

D'Agostino–Pearson test

This range is useful, but it is not, by itself, a formal test: looking at a single kurtosis or skewness value cannot tell us how confident we should be that a departure from zero is a real deviation from normality rather than plain sampling noise. The D'Agostino–Pearson test closes this gap by turning skewness and kurtosis into a single, joint hypothesis test [13, 14], applied to every (class, dimension) pair. Concretely, it converts each moment into a standardized score (Z_1 for skewness, Z_2 for kurtosis) and adds their squares into one combined statistic:

$$K^2 \;=\; Z_1\Big(\widehat{\mathrm{Skew}}\Big)^2 + Z_2\Big(\widehat{\mathrm{Kurt}}\Big)^2 \;\sim\; \chi^2_2 \quad \text{under } H_0, \tag{3.9}$$

where H_0 is the hypothesis that the coordinate's data is normally distributed. Under H_0, K^2 follows a known chi-squared distribution with two degrees of freedom, which lets us compute an exact probability (a p-value) for how extreme the observed K^2 is; we reject H_0, and conclude the coordinate is not Gaussian, whenever this probability falls below the significance level $\alpha = 0.05$.

Relaxed criterion.

The D'Agostino–Pearson test becomes more sensitive as the sample size n_c grows: with enough samples, even small, practically harmless departures from normality become statistically significant, causing the test to reject H_0 more often than is practically useful. To compensate for this, we add a second, size-independent check based directly on the reference range introduced earlier in this section: a dimension is classified as near-Gaussian if

$$|\mathrm{Skew}| < 1 \quad \text{and} \quad |\mathrm{Kurt}| < 1 \tag{3.10}$$

hold simultaneously.

Henze–Zirkler test per dimension.

The tests above only look at the shape of each coordinate's distribution, through its skewness and kurtosis. As a stricter check, we also apply the Henze–Zirkler (HZ) test [15] to every coordinate on its own: instead of summarising the data with a couple of shape numbers, it directly compares how the data is distributed against how a true Gaussian would be. The HZ statistic is defined for a feature vector of any dimension d; here we use its simplest form, one coordinate at a time ($d = 1$), applied independently to every (class c, dimension j) pair. The same statistic, now with d equal to the full embedding dimension, is used again in Section 3.2.2.

Given n_c centred samples $z_i = x_i - \hat{\mu}_{c,j}$ and a bandwidth $b > 0$, the statistic is built from three simple terms, using the squared, variance-normalised distance $d^2_M(\cdot,\cdot)$ between two values:

$$A = \frac{1}{n_c^2} \sum_{j=1}^{n_c} \sum_{k=1}^{n_c} e^{-\frac{b^2}{2} d_M^2(z_j, z_k)}, \tag{3.11}$$

$$B = \frac{2}{n_c} (1 + b^2)^{-1/2} \sum_{j=1}^{n_c} e^{-\frac{b^2}{2(1+b^2)} d_M^2(z_j, 0)}, \tag{3.12}$$

$$C = (1 + 2b^2)^{-1/2}. \tag{3.13}$$

In plain terms, A checks how similar the samples are to one another, B checks how quickly the data thins out around the mean, and C is simply the value B would take for a perfect Gaussian. Put together, the final statistic is:

$$T_{n,b} = n_c \cdot [A - B + C]\,. \tag{3.14}$$

$T_{n,b}$ stays close to a fixed reference value when the data really is Gaussian, and moves away from it when the data is more tightly clustered or more heavy-tailed than a Gaussian would be.

Bandwidth. For $d = 1$, the bandwidth reduces to a simple function of the sample size alone:

$$b^2 \;=\; \frac{1}{2} \left(\frac{4}{3} \right)^{2/5} n_c^{-2/5}. \tag{3.15}$$

Fast computation. Computing A directly means comparing all n_c^2 pairs of samples, which becomes costly once this has to be repeated for every coordinate. We avoid this by expanding the exponential in A as a short Taylor series:

$$A_m \;=\; \frac{1}{n_c^2} \sum_{j,k} e^{-\frac{b^2}{2}(z_{j,m} - z_{k,m})^2} \;=\; \frac{1}{n_c^2} \sum_{\ell=0}^{L} \frac{b^{2\ell}}{\ell!}\, S_\ell^2 \;+\; \mathcal{O}\big((b^2)^{L+1}\big)\,, \tag{3.16}$$

where $S_\ell = \sum_j v_{j,m}\, z_{j,m}^\ell$ and $v_{j,m} = \exp(-b^2 z_{j,m}^2/2)$. Using $L = 8$ terms keeps the approximation error below 10^{-6}, while letting us compute all d coordinates at once instead of one at a time.

What to expect under H_0. If a coordinate is truly Gaussian (H_0), $T_{n,b}$ should fluctuate around a known reference value rather than around zero. For a general dimension d, this reference is approximately

$$\mathbb{E}_{H_0}[T_{\mathrm{HZ}}] \;=\; 1 - (1 + 2b^2)^{-d/2}, \tag{3.17}$$

a formula we reuse, with d equal to the full embedding dimension, in Section 3.2.2. For the univariate case treated here ($d = 1$), this approximation is not accurate enough, so we use the exact expectation instead:

$$\mathbb{E}_{H_0}[T_{\mathrm{HZ}}] \;=\; 1 - a^{-1/2} \left(1 + \frac{b^2}{a} + \frac{3b^4}{4a^2} \right), \qquad a = 1 + 2b^2, \tag{3.18}$$

with b^2 as given above. For $n_c \in [60, 400]$, this gives a small bandwidth ($b^2 \approx 0.04$) and a near-zero baseline ($\mathbb{E}_{H_0}[T_{\mathrm{HZ}}] \approx 0.04$). In other words, a genuinely Gaussian coordinate should give a T_{HZ} value fluctuating tightly around this near-zero baseline.

3.2.2 Per-Class Gaussianity

We now move from testing one coordinate at a time to testing each class's full feature vector at once, across all d dimensions together. This uses the same HZ statistic $T_{n,b}$ from Section 3.2.1, just with d set to the full embedding dimension instead of $d = 1$. This joint test is needed because per-coordinate normality is not enough on its own: a set of coordinates can each look Gaussian by itself while still not being jointly Gaussian, if the way they relate to each other does not match what a multivariate Gaussian would look like.

Applying the test this way runs into a problem, though. Both A and B need the Mahalanobis distance, and computing it requires inverting the class covariance matrix $\mathbf{\Sigma}_c$. Whenever a class has fewer samples than feature dimensions ($n_c < d$), which is the case throughout this thesis, the estimated covariance matrix cannot be inverted, so the standard version of the test simply cannot be computed.

To work around this without dropping any dimensions, we use a simplified, diagonal version of $\mathbf{\Sigma}_c$: instead of inverting the full matrix, we only invert its d individual per-coordinate variances, treating the coordinates as unrelated for this purpose. A small constant ($\varepsilon = 10^{-10}$) is added to these variances so that coordinates with almost no spread do not cause a division by a near-zero number. This simplified version can always be computed, no matter how small n_c is, which is what makes the test usable in this setting.

One thing changes when we switch to this high-dimensional, diagonal version of the test: the reference value T_{HZ} should be close to under H_0. The general formula is the same one given earlier,

$$\mathbb{E}_{H_0}[T_{\mathrm{HZ}}] \;=\; 1 - (1 + 2b^2)^{-d/2}, \tag{3.19}$$

but here d is large, so the term $(1 + 2b^2)^{-d/2}$ becomes essentially zero even for a modest bandwidth b, and the whole expression becomes close to 1 instead of the near-0 value used in the univariate case. In practice, this means that a class whose features really are Gaussian should give a T_{HZ} value close to 1, not close to 0.

It is important to clarify explicitly that, because our modified HZ test enforces a diagonal covariance matrix to handle this small-sample regime, it does not mathematically guarantee full, arbitrary multivariate Gaussianity, where the covariances between every pair of dimensions would be fully unconstrained. However, given how severe the data limitation is for each class relative to the embedding dimension ($n_c < d$), this evaluation serves as a highly robust and reliable approximation for that stronger, unconstrained notion of Gaussianity.

3.3 Inter-Class Structural Separability Analysis

To understand how the Bayesian decision surface scales and characterize the geometric crowding of the embedding space as new tasks are incorporated, we evaluate inter-class relationships using two complementary metrics: the probabilistic Bhattacharyya distance and the geometric Euclidean distance.

3.3.1 Pairwise Bhattacharyya Distance

The *inter-class* pairwise Bhattacharyya distance measures how distinguishable any two class-conditional Gaussians are from one another. The Bhattacharyya distance is symmetric and provides a direct connection to the Bayes error rate via the Bhattacharyya coefficient $\rho_{ij} = e^{-D_B(i,j)}$. Under the diagonal-Gaussian parameterisation, the Bhattacharyya distance between classes i and j is defined as:

$$D_B(i,j) = \frac{1}{8}(\boldsymbol{\mu}_i - \boldsymbol{\mu}_j)^\top \bar{\Sigma}^{-1}(\boldsymbol{\mu}_i - \boldsymbol{\mu}_j) + \frac{1}{2}\ln\left(\frac{\det \bar{\Sigma}}{\sqrt{\det \Sigma_i \, \det \Sigma_j}}\right), \tag{3.20}$$

where $\bar{\Sigma} = (\Sigma_i + \Sigma_j)/2$. Because both Σ_i and $\bar{\Sigma}$ are diagonal, all matrix operations reduce to element-wise arithmetic: determinants become products of the diagonal entries (or sums of logs), and the matrix inverse reduces to element-wise reciprocals. A value of $D_B = 0$ indicates complete distributional overlap; larger values correspond to greater separability and a lower Bhattacharyya coefficient $\rho = e^{-D_B}$.

3.3.2 Pairwise Mean Euclidean Distance

While the Bhattacharyya distance jointly accounts for shifts in both first-order (means) and second-order (covariances) statistics, it can mask whether class texturing or raw prototype drift is the dominant factor in class confusion. Therefore, we isolate first-order geometric separation by calculating the plain pairwise Euclidean distance between the estimated class mean vectors $\boldsymbol{\mu}_i$ and $\boldsymbol{\mu}_j$:

$$d_E(i,j) = \|\boldsymbol{\mu}_i - \boldsymbol{\mu}_j\|_2 = \sqrt{\sum_{k=1}^{d}(\mu_{i,k} - \mu_{j,k})^2}, \tag{3.21}$$

where d is the nominal dimensionality of the visual foundation embedding space.

3.3.3 Stream Statistics and Trajectory Tracking

As the total number of observed classes k scales throughout the continual learning stream, we track three distinct statistics for both D_B and d_E across the expanding class-pair pool:

(a) The **macro-averaged pairwise distance** ($\overline{D_B}$ and $\overline{d_E}$) flanked by a $[\mathrm{P}_{10}, \mathrm{P}_{90}]$ percentile band to observe global drift.

(b) The **minimum pairwise distance** ($\min D_B$ and $\min d_E$), which explicitly identifies the hardest, most confusable class pair in the system.

(c) The **cumulative count of overlapping pairs** falling below critical proximity thresholds (e.g., $D_B \in \{10, 50, 100\}$ and $d_E \in \{0.3, 0.5, 0.7\}$) to capture the heavy-tailed growth of localized confusion sinks.

Together, these metrics bridge empirical classification failure modes with the underlying statistical architecture of the learned representations.

3.4 Covariance Estimation in High-Dimensional Spaces

A fundamental challenge in multivariate statistical analysis and pattern recognition is the estimation of the covariance matrix $\mathbf{\Sigma} \in \mathbb{R}^{d \times d}$ from a finite set of n observations. In classical low-dimensional regimes, where the sample size n is much larger than the feature dimension d ($n \gg d$), the empirical Sample Covariance Matrix (SCM) acts as a reliable, unbiased estimator. However, modern engineering applications—such as analyzing high-dimensional deep feature representations—frequently operate in the "high-dimension, small-sample-size" (HDSS) regime, where d is of the order of hundreds or thousands, while n remains strictly limited.

Under these conditions, classical multivariate analysis suffers severely from the *curse of dimensionality*, leading to critical theoretical and numerical breakdowns in statistical modeling.

3.4.1 The Curse of Dimensionality and Covariance Degeneracy

When the dimensionality d scales while the sample size n remains fixed or grows at a slower rate, several geometric and algebraic anomalies emerge:

- **Singularity and Rank Deficiency:** Structurally, the SCM is calculated as the sum of n outer products of centered data vectors. Consequently, the maximum rank of the SCM is bounded by $\min(n-1, d)$. In the typical HDSS scenario where $n < d$, the SCM has at least $d - n + 1$ zero eigenvalues. The resulting matrix is singular (rank-deficient) and cannot be inverted.
- **Loss of Negative Log-Likelihood and Distance Metrics:** Many fundamental statistical classifiers and density estimators—such as Gaussian Discriminant Analysis (GDA) or Mahalanobis distance formulations—depend heavily on the precision matrix (the inverse covariance, $\mathbf{\Sigma}^{-1}$) and the log-determinant $\ln|\mathbf{\Sigma}|$. When the estimated covariance is singular, the log-determinant diverges to $-\infty$, and the precision matrix is undefined, rendering density estimation mathematically impossible.

- **Eigenvalue Distortion (Spreading Phenomenon):** Even in moderately high dimensions where $n \geq d$ and the SCM is technically invertible, the sample eigenvalues do not converge to their true counterparts. As proved in random matrix theory (e.g., the Marčenko-Pastur law [51]), high-dimensional sample eigenvalues exhibit severe bias: the largest eigenvalues are systematically overestimated, while the smallest eigenvalues are underestimated. This artificial dispersion distorts the perceived geometric structure of the data.

3.4.2 Theoretical Foundations of Regularization

The problems described above all trace back to the same cause: using the plain SCM in a regime where it cannot be trusted. The standard fix is *regularization*: we stop trying to estimate the covariance matrix as precisely as possible, and instead accept a small, controlled amount of bias in exchange for a much more stable estimate.

In practice, regularization is implemented through two main strategies:

1. **Spectral regularization and conditioning** [52]: this strategy works directly on the eigenvalues of the covariance matrix. Since the problem is that some eigenvalues are zero or too close to zero, the fix is to push them up a bit, either by setting a minimum allowed eigenvalue or, more simply, by adding a small constant to the diagonal of the matrix (often called diagonal loading). This keeps every eigenvalue safely above zero, so the matrix stays invertible and its inverse does not blow up.
2. **Structured shrinkage** [53, 54]: this strategy takes a different route. Instead of correcting individual eigenvalues, it blends the unstable sample covariance matrix with a much simpler, well-behaved "target" matrix (for example, the identity matrix, scaled appropriately, or a diagonal matrix). The final estimate is a weighted average of the two, and by shrinking it toward this stable target, the distorted, spread-out eigenvalues of the sample covariance are pulled back closer to their true values, reducing the mean squared error of the estimate.

3.5 Spectral Geometry of the Feature Space

We now look at how variance is spread across the d coordinates of the embedding—that is, whether the data varies a lot along every one of its d dimensions, or whether most of that variation is actually concentrated in just a few of them, with the rest being close to constant. This matters because it directly affects both how trustworthy our Gaussian modelling is and how well the covariance estimators of the previous section can be expected to behave. We use Principal Component Analysis (PCA) to look at this spread, then use what it shows us, called *spectral decay*, to ask a simple question: is the feature space really as high-dimensional as it looks, or can it be described well with far fewer dimensions?

3.5.1 Principal Component Analysis

PCA is a standard and widely used technique [55]: given a centred data matrix $\mathbf{X} \in \mathbb{R}^{N \times d}$, PCA diagonalises its empirical covariance matrix, $\mathbf{S} = \mathbf{P}\mathbf{\Lambda}\mathbf{P}^T$, where the columns of $\mathbf{P}$ are orthonormal directions in the feature space (the *principal components*), and the diagonal of $\mathbf{\Lambda}$ holds their associated eigenvalues $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_d \geq 0$. Each eigenvalue simply tells us how much variance in the data lies along its corresponding direction, so together they describe exactly how the total variance of the embedding is distributed across the d dimensions.

3.5.2 Spectral Decay and Effective Dimensionality

We call the sequence of eigenvalues $\lambda_1, \lambda_2, \ldots, \lambda_d$, sorted from largest to smallest, the *spectrum* of the feature space, and how quickly it shrinks is its *decay*. A fast decay means that a handful of directions already explain most of the variance, and the remaining directions add very little; a slow, flat decay means variance is spread out roughly evenly across most of the d dimensions. Concretely, the fraction of total variance explained by the leading k components is

$$R(k) = \frac{\sum_{i=1}^{k} \lambda_i}{\sum_{i=1}^{d} \lambda_i}. \tag{3.22}$$

This matters for two practical reasons. First, if $R(k)$ is already close to 1 for some $k \ll d$, the data effectively lives in a k-dimensional subspace, even though it is represented using d numbers; we call this its *effective dimensionality*, and it is often much smaller than d. Second, this tells us whether the feature space could be safely compressed: if most of the discriminative information is packed into the first k components, reducing the embedding to just those k dimensions before classification should barely hurt accuracy, while making the small-sample covariance-estimation problem of the previous section considerably easier, since it is now a k-dimensional problem rather than a d-dimensional one.

To measure this decay in practice, we compute the empirical eigenspectrum of the ℓ_2-normalised embeddings, pooled across an entire benchmark's training split rather than split by class. Sorting the resulting d eigenvalues from largest to smallest lets us read off, directly, both the variance explained by each individual component and the cumulative variance explained as more components are added.

A steep, fast decay tells us the visual signal is concentrated in a small, active subspace, with the remaining directions being largely redundant; a slow, flat decay instead tells us the signal is spread thinly across most of the ambient space.

3.5.3 PCA Dimension-Reduction Testing

The analysis above tells us how concentrated the variance is, but not whether that concentration can actually be exploited to help with the small-sample rank deficiency ($n_c < d$). To

check this directly, we introduce a test in which we deliberately compress the features before classification and see how the framework performs.

Rather than fitting PCA separately on each target stream, which would break the class-incremental setting by letting the projection change over time, we use a single, fixed projection matrix $\mathbf{P}_k \in \mathbb{R}^{d \times k}$, pre-trained once on ImageNet-21k and never updated afterwards.

Comparing performance across these reduced dimensions tells us two things at once: whether performance holds up under compression, which would confirm that the discarded directions really were mostly redundant; and whether the gap between models narrows as k shrinks, which would confirm that the compression is also helping with the small-sample covariance problem, rather than just discarding useful information.

Chapter 4

Modelling Class Densities in a Frozen Feature Space

This chapter presents, in full detail, the framework this thesis proposes: a continual classifier built around a frozen visual foundation model, paired with a family of class-conditional Gaussian classifiers built on top of it. We start with the overall architecture and the likelihood-based decision rule it rests on. From there, we go through the different ways we estimate each class's covariance matrix, since this is where most of the difficulty of the small-sample, high-dimensional setting studied in this thesis actually shows up. The chapter closes by extending this same idea to mixtures of Gaussians, for classes whose features are not well described by a single Gaussian component.

4.1 Proposed Architecture

As shown in Figure 4.1, our CIL framework is built upon a simple yet structurally motivated decomposition: (i) a frozen feature extractor based on a visual foundation model, and (ii) a probabilistic classification layer that models each object category through an independent Gaussian distribution in the embedding space.

The choice of a visual foundation model as the feature extractor is central to this decomposition, and it is worth spelling out why. As discussed in Chapter 2, foundation models are pre-trained once, at scale, on broad data, precisely so that the resulting representation is general enough to be reused across many downstream tasks without having to be retrained for each one. For a continual classifier, this property is exactly what is needed: a backbone whose embeddings are already discriminative enough to describe object classes it has never explicitly been trained to separate, including ones that will only be introduced long after the backbone itself was trained. This is what allows us to treat the feature extractor as entirely frozen throughout this thesis—no fine-tuning, no gradient updates, not even at the moment a new class is introduced—and still expect the resulting embeddings to be informative enough for classification.

Freezing the backbone is also what makes the two components of the architecture genuinely modular. The feature extractor and the classification layer do not exchange gradients and do not need to be trained jointly: any frozen encoder that maps an input image to a fixed-dimensional embedding can be dropped in without changing anything about the classifier built on top of it, and, symmetrically, the classification layer can be modified, extended, or replaced without ever touching the backbone.

Given an input image, the feature extractor maps it to a fixed high-dimensional representation. For each class, we maintain a class-conditional multivariate Gaussian characterized by its mean vector and covariance matrix, estimated incrementally from observed samples. Classification is performed according to the maximum likelihood principle, assigning the input to the class whose Gaussian model yields the highest likelihood under the stationary embedding representation. This modular separation between representation learning and statistical modeling enables incremental updates without backpropagation or parameter adaptation: adding a new class requires only estimating its distribution in the fixed feature space, while previously learned classes remain unaffected at the representation level.

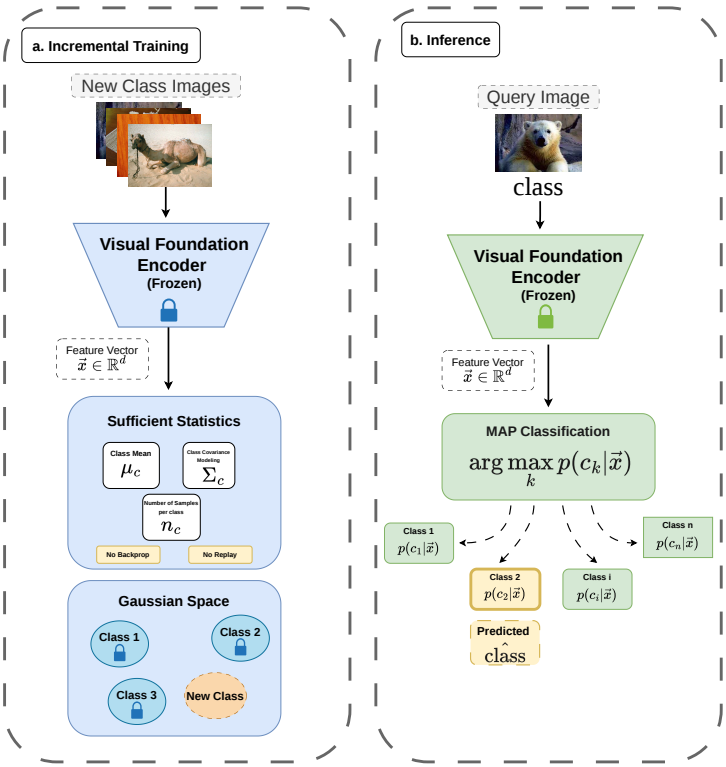


Figure 4.1: Overview of the proposed framework, reproduced from the peer-reviewed article that accompanies this thesis. A frozen foundation encoder maps each input image to a fixed embedding $\vec{x} \in \mathbb{R}^d$. Each class is represented by a Gaussian density (mean and covariance) that is updated incrementally without backpropagation. At inference, classification follows the MAP rule under the learned class-conditional densities.

4.1.1 Likelihood-Based Classification

Given an input image $x \in \mathcal{X}$, the frozen foundation model $\mathcal{F}$ produces a fixed-dimensional embedding

$$\vec{x} = \mathcal{F}(x) \in \mathbb{R}^d \tag{4.1}$$

and all learning is performed directly in the stationary feature space $\mathbb{R}^d$. We adopt a generative modelling framework and assume class-conditional Gaussian feature distributions:

$$p(\vec{x} \mid y = c) = \mathcal{N}(\vec{x} \mid \boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c), \tag{4.2}$$

where $\boldsymbol{\mu}_c \in \mathbb{R}^d$ and $\boldsymbol{\Sigma}_c \succ 0$ are the class-specific mean and covariance matrix, estimated incrementally from observed samples. Classification is performed via Maximum A Posteriori (MAP) inference:

$$\hat{y} = \arg\max_{c \in \mathcal{Y}} \left[\log p(\vec{x} \mid y = c) + \log \pi_c \right], \tag{4.3}$$

where $\pi_c = p(y = c)$ is the class prior. Under a uniform prior and the Gaussian density model, the decision rule is equivalent to the quadratic discriminant:

$$\hat{y} = \arg\min_{c \in \mathcal{Y}} \left[(\vec{x} - \boldsymbol{\mu}_c)^T \boldsymbol{\Sigma}_c^{-1} (\vec{x} - \boldsymbol{\mu}_c) + \log |\boldsymbol{\Sigma}_c| \right]. \tag{4.4}$$

This quadratic decision function is the core of the proposed framework. The Mahalanobis term $(\vec{x} - \boldsymbol{\mu}_c)^T \boldsymbol{\Sigma}_c^{-1} (\vec{x} - \boldsymbol{\mu}_c)$ encodes the *shape* of the class distribution while $\log |\boldsymbol{\Sigma}_c|$ encodes its *volume*. The covariance parameterisations introduced in Section 4.2 are specialisations of Equation (4.4) that differ in the structure imposed on $\boldsymbol{\Sigma}_c$; the interplay between the shape and volume components is the central design axis of the covariance estimation study and is examined in detail in the analysis of results.

In practice, adding a new class c to the system means passing its training images once through the frozen feature extractor to obtain their embeddings, and then computing the sample mean $\boldsymbol{\mu}_c$ and covariance $\boldsymbol{\Sigma}_c$ of those embeddings directly, in closed form, following whichever covariance parameterisation is in use. These two quantities are exactly the sufficient statistics of a Gaussian: once they are computed, the individual training samples themselves can be discarded, since the class-conditional density in Equation (4.2) depends on them only through $\boldsymbol{\mu}_c$ and $\boldsymbol{\Sigma}_c$. The new class is then registered simply by adding this pair to the set of class-conditional Gaussians the classifier maintains, extending the arg max in Equation (4.3) over one more class; nothing about the mean or covariance of any previously learned class is touched in this process. This formulation therefore enables genuinely incremental updates without backpropagation, replay buffers, or exemplar storage: the only thing that changes as the class set grows is which Gaussians are being compared at inference time.

4.2 Covariance Estimation

Due to the high dimensionality of the extracted feature ($\vec{x} \in \mathbb{R}^d$), the empirical sample covariance matrix (SCM) is a poor estimator for each class: its eigenvalues are biased, and

the matrix is singular whenever $n_c \leq d$. In our setting $d = 768$ while n_c ranges from approximately 40 to 400 depending on the benchmark, so the matrix always has at most $n_c - 1$ non-zero eigenvalues out of $d = 768$, leaving $d - (n_c - 1)$ eigenvalues that are structurally zero before any regularisation is applied. We focus on how to obtain a well-conditioned covariance estimate under the Gaussian modeling assumption. Our benchmarks are tested using different covariance estimation methods:

4.2.1 Diagonal Covariances

In this configuration, we assume that the features within each class c are independent, leading to a diagonal covariance matrix $\mathbf{\Sigma}_c = \text{diag}(\sigma_{c,1}^2, \sigma_{c,2}^2, \ldots, \sigma_{c,d}^2)$. This simplification significantly reduces the number of parameters to estimate from $O(d^2)$ to $O(d)$, which is particularly advantageous when the dimensionality $d = 768$ is large relative to the number of samples per class. For each class $c \in \{1, \ldots, C\}$, the mean vector $\boldsymbol{\mu}_c$ and the diagonal elements of $\mathbf{\Sigma}_c$ are computed as:

$$\boldsymbol{\mu}_c = \frac{1}{n_c} \sum_{i=1}^{n_c} \vec{x}_i, \quad \sigma_{c,j}^2 = \frac{1}{n_c - 1} \sum_{i=1}^{n_c} (x_{i,j} - \mu_{c,j})^2 + \varepsilon \tag{4.5}$$

where $\varepsilon = 10^{-6}$ is a small constant added to ensure numerical stability and strictly positive variances. The log-likelihood of a test sample $\vec{x}$ belonging to class c is determined by the discriminant function:

$$g_c(\vec{x}) = \ln P(\vec{x}|c) + \ln P(c) \tag{4.6}$$

Under the assumption of a uniform prior $P(c)$ and Gaussian distribution, this simplifies to:

$$g_c(\vec{x}) = -\frac{1}{2} \sum_{j=1}^{d} \left(\ln(2\pi\sigma_{c,j}^2) + \frac{(x_j - \mu_{c,j})^2}{\sigma_{c,j}^2} \right) \tag{4.7}$$

4.2.2 Full Covariances

To account for the dependencies between different dimensions of the feature vector, we estimate a full covariance matrix $\mathbf{\Sigma}_c \in \mathbb{R}^{d \times d}$ for each class c. While this provides a richer representation of the class distribution, the estimation of d^2 parameters is prone to ill-conditioning when the number of samples n_c is limited.The mean vector $\boldsymbol{\mu}_c$ and the empirical covariance $\mathbf{\Sigma}_c$ are calculated as:

$$\boldsymbol{\mu}_c = \frac{1}{n_c} \sum_{i=1}^{n_c} \vec{x}_i, \quad \mathbf{\Sigma}_c = \frac{1}{n_c - 1} \sum_{i=1}^{n_c} (\vec{x}_i - \boldsymbol{\mu}_c)(\vec{x}_i - \boldsymbol{\mu}_c)^T \tag{4.8}$$

To ensure that the matrix is invertible and to improve numerical stability, a regularization is applied by adding a small identity term to the diagonal: $\hat{\mathbf{\Sigma}}_c = \mathbf{\Sigma}_c + \varepsilon I$, with $\varepsilon = 10^{-6}$.

For obtaining the precision matrix Q_c (the inverse of the covariance matrix $\hat{\mathbf{\Sigma}}_c$), an approach based on Cholesky decompositions is used. The covariance $\hat{\mathbf{\Sigma}}_c$ is decomposed as

$$\hat{\mathbf{\Sigma}}_c = C_c C_c^T, \qquad C_c L_c = I, \tag{4.9}$$

with C_c lower triangular and I the identity, so that $L_c = C_c^{-1}$ is the Cholesky factor of the precision matrix $Q_c = L_c^T L_c$.

Then, the squared Mahalanobis distance and the log-determinant are computed from $\boldsymbol{\mu}_c$ and L_c as:

$$d_M(\vec{x}, \boldsymbol{\mu}_c, L_c) = (\vec{x}L_c - \boldsymbol{\mu}_c L_c)^T (\vec{x}L_c - \boldsymbol{\mu}_c L_c) \tag{4.10}$$

$$\ln |\hat{\mathbf{\Sigma}}_c| = 2 \sum_{i=1}^{d} \ln(L_{c\,i,i}) \tag{4.11}$$

4.2.3 Hybrid Covariance

The hybrid covariance estimator decouples the squared Mahalanobis distance from its associated normalisation constant. For each class c, the full covariance matrix $\hat{\mathbf{\Sigma}}_c$ is estimated and regularised exactly as in Section 4.2.2, and the squared Mahalanobis distance $d_M(\vec{x}, \mu_c, L_c)$ is computed using the same Cholesky-based procedure of Equations (4.9)–(4.10), so that the *shape* of the discriminant retains the full off-diagonal structure of the empirical covariance. However, the log-determinant term—which represents the *volume* of the distribution—is computed using only the diagonal elements (variances) of the covariance matrix:

$$\ln |\hat{\mathbf{\Sigma}}_c|_{\text{diag}} = \sum_{j=1}^{d} \ln(\sigma_{c,j}^2), \tag{4.12}$$

where $\sigma_{c,j}^2$ denotes the j-th diagonal entry of $\hat{\mathbf{\Sigma}}_c$. The resulting discriminant function is defined as:

$$g_c(\vec{x}) = -\frac{1}{2} \left(d \ln(2\pi) + \sum_{j=1}^{d} \ln(\sigma_{c,j}^2) + d_M(\vec{x}, \boldsymbol{\mu}_c, L_c) \right). \tag{4.13}$$

This formulation avoids the numerical issues that arise when near-zero eigenvalues of $\hat{\mathbf{\Sigma}}_c$ artificially inflate or destabilise the exact log-determinant, at the cost of using a diagonal approximation for the volume term only.

4.2.4 OAS Covariance

Oracle Approximating Shrinkage (OAS) is a shrinkage technique that computes an structurally optimized covariance matrix by finding an optimal compromise between the sample covariance matrix and a structured target (the identity matrix scaled by the average trace). The OAS estimator provides a more accurate estimation of the covariance $\mathbf{\Sigma}_c$ by minimizing the Mean Squared Error (MSE) under the assumption of Gaussianity. The estimated covariance is defined as:

$$\hat{\mathbf{\Sigma}}_{\text{OAS},c} = (1 - \rho_c)\hat{\mathbf{\Sigma}}_{\text{SCM},c} + \rho_c \hat{F}_c \tag{4.14}$$

where ρ_c is the shrinkage coefficient calculated specifically for class c, $\hat{\mathbf{\Sigma}}_{\text{SCM},c}$ is the sample covariance matrix, and $\hat{F}_c = \frac{\text{tr}(\hat{\mathbf{\Sigma}}_{\text{SCM},c})}{d} I$ is the shrinkage target. In our implementation, we compute the precision matrix $\Psi_c = \hat{\mathbf{\Sigma}}_{\text{OAS},c}^{-1}$ directly via the OAS algorithm. This precision matrix is then used to calculate the Mahalanobis distance in the log-likelihood discriminant function:

$$g_c(\vec{x}) = \ln A_c - \frac{1}{2}(\vec{x} - \boldsymbol{\mu}_c)^T \Psi_c (\vec{x} - \boldsymbol{\mu}_c) \tag{4.15}$$

where $\ln A_c$ is the normalization constant derived from the log-determinant of the OAS-estimated covariance matrix, evaluated numerically via the stable log-determinant of the eigendecomposition. This approach ensures that the model captures class-specific correlations while remaining robust to the "curse of dimensionality."

4.2.5 Ledoit-Wolff

The Ledoit-Wolf Diagonal Anchor estimator addresses the instability of the sample covariance matrix S by shrinking it toward a target matrix T.

The selection of a target matrix T represents a prior assumption regarding the feature distribution. Both methods stabilize the estimator by pulling the noisy off-diagonal elements of the sample covariance matrix S toward zero, but they differ in how they treat the diagonal elements (the variances).

Diagonal Target

The diagonal anchor is defined as $T_{\text{diag}} = \text{diag}(s_{11}, s_{22}, \ldots, s_{dd})$. This approach assumes that the features are independent but allows for heteroscedasticity, meaning each dimension can have a different variance. Shrinkage toward this target preserves the scale of each individual feature as observed in the sample data while suppressing only the correlations. This is particularly useful when certain dimensions of the DINOv2 embeddings are naturally more informative or volatile than others.

Identity Target

The identity anchor is more restrictive, defined as $T_{\text{ident}} = \mu I$, where $\mu = \frac{1}{d}\text{tr}(S)$ is the average variance across all dimensions. This target assumes that the features are not only independent but also share a common variance (homoscedasticity). The resulting distribution is spherical. Because the embeddings used in this work are L2-normalized, the variances across dimensions tend to be similar, making the identity anchor a highly robust regularizer that prevents any single dimension from disproportionately influencing the distance metric.

Structural Summary

The fundamental difference between these two estimators lies in the flexibility of the variance components:

- **Diagonal Anchor:** $\hat{\Sigma}_{ij} = (1-\gamma)s_{ij}$ for $i \neq j$, and $\hat{\Sigma}_{ii} = s_{ii}$. It yields an axis-aligned ellipsoidal distribution.
- **Identity Anchor:** $\hat{\Sigma}_{ij} = (1-\gamma)s_{ij}$ for $i \neq j$, and $\hat{\Sigma}_{ii} = (1-\gamma)s_{ii} + \gamma\mu$. It yields a distribution that is pulled toward a perfectly spherical geometry.

4.2.6 Shared Covariance

The Shared Covariance framework assumes that the internal noise or feature variability is homoscedastic across all classes. In this paradigm, all classes share a single covariance matrix $\mathbf{\Sigma}$, which captures the global structure of the feature space.

A key aspect of this framework in the continual learning setting is that $\mathbf{\Sigma}$ is re-estimated at the end of each task t using only the classes observed so far, $\mathcal{C}_t = \{1, \ldots, C_t\}$.

4.3 Mixture of Gaussians

In order to capture potential multi-modality within the feature space of a single class, we employ Gaussian Mixture Models (GMM). We implemented the following models using the previously defined diagonal, full, hybrid and shared covariances approaches.

4.3.1 Mixture of 2 Gaussians

Forcing each class to be defined as a GMM with $K = 2$ components, each class c is modeled by a probability density function:

$$p(\vec{x}|c) = \sum_{k=1}^{K} \pi_{c,k} \mathcal{N}(\vec{x}|\boldsymbol{\mu}_{c,k}, \boldsymbol{\Sigma}_{c,k}) \tag{4.16}$$

where $\pi_{c,k}$ represents the mixing weight of the k-th component for class c, subject to $\sum_{k=1}^{K} \pi_{c,k} = 1$. The parameters $\{\pi_{c,k}, \boldsymbol{\mu}_{c,k}, \boldsymbol{\Sigma}_{c,k}\}$ are estimated using the Expectation-Maximization (EM) algorithm. Depending on the evaluated variant (*MOG2-D*, *MOG2-F*, or *MOG2-H*), the component covariance matrices $\boldsymbol{\Sigma}_{c,k}$ adopt diagonal, regularized full, or hybrid structures. For numerical stability, a small constant is added to the diagonal entries.

The log-likelihood of a test sample $\vec{x}$ is computed stably via the *LogSumExp* (or `logaddexp`) operator:

$$\ln p(\vec{x}|c) = \mathrm{LogSumExp}_{k=1}^{2} \left(\ln \pi_{c,k} + \ln \mathcal{N}(\vec{x}|\boldsymbol{\mu}_{c,k}, \boldsymbol{\Sigma}_{c,k}) \right) \tag{4.17}$$

which expands in implementation to:

$$\ln p(\vec{x}|c) = \ln \left(\sum_{k=1}^{2} \exp \left(\ln \pi_{c,k} + \ln A_{c,k} - \frac{1}{2} d_M(\vec{x}, \boldsymbol{\mu}_{c,k}, \boldsymbol{\Sigma}_{c,k}) \right) \right) \tag{4.18}$$

where $d_M(\vec{x}, \boldsymbol{\mu}_{c,k}, \boldsymbol{\Sigma}_{c,k}) = (\vec{x} - \boldsymbol{\mu}_{c,k})^T \boldsymbol{\Sigma}_{c,k}^{-1} (\vec{x} - \boldsymbol{\mu}_{c,k})$ is the squared Mahalanobis distance to the k-th component—evaluated coordinate-wise for the diagonal model or via Cholesky factorisation for full and hybrid configurations—and $\ln A_{c,k} = -\frac{1}{2}(d \ln(2\pi) + \ln |\boldsymbol{\Sigma}_{c,k}|)$ represents the corresponding log-normalisation term.

4.3.2 Mixture of up-to-4 Gaussians

To adapt to the varying complexity of different classes across benchmarks, we implement a flexible Gaussian Mixture Model where the number of components K_c for each class c is not fixed. Instead, the model selects the most appropriate number of components $K_c \in \{1, 2, 3, 4\}$ by minimizing the Bayesian Information Criterion (BIC):

$$\mathrm{BIC} = k_p \ln(n_c) - 2 \ln(\hat{L}) \tag{4.19}$$

where n_c is the number of available samples for class c, $\hat{L}$ is the maximized likelihood of the fitted mixture, and k_p represents the number of free parameters. Importantly, model selection via BIC is performed using diagonal covariance matrices to maintain computational tractability, prevent numerical degeneration in the small-sample regime ($n_c < d$), and keep the parameter penalty lightweight ($k_p = K_c(2d + 1) - 1$).

Once the optimal number of components K_c and their sample assignments are determined, the component covariance matrices $\boldsymbol{\Sigma}_{c,k}$ can be instantiated according to the desired

parameterisation: diagonal (MOG4-D), regularized full (MOG4-F), hybrid (MOG4-H), or paired with a globally pooled covariance matrix (MOG4-Shared).

At inference time, the classification log-likelihood of an input embedding $\vec{x}$ across all active classes is computed stably via the LogSumExp (LSE) operator:

$$\ln p(\vec{x}|c) = \mathrm{LSE}_{k=1}^{K_c}\left(\ln \pi_{c,k} + \ln A_{c,k} - \frac{1}{2} d_M(\vec{x}, \boldsymbol{\mu}_{c,k}, \boldsymbol{\Sigma}_{c,k})\right) \tag{4.20}$$

where $d_M(\vec{x}, \boldsymbol{\mu}_{c,k}, \boldsymbol{\Sigma}_{c,k})$ denotes the squared Mahalanobis distance to the k-th component—evaluated coordinate-wise under diagonal covariances or via Cholesky factorisation for full-rank and hybrid parameterisations—and $\ln A_{c,k}$ represents the corresponding log-normalisation term. This decoupling ensures that the complexity of the cluster structure is selected robustly, while still allowing the final classifier to leverage richer second-order geometry.

Chapter 5

Experimental Results and Analysis

This chapter presents the quantitative results of the experiments carried out in this thesis, together with their interpretation, topic by topic. It opens by presenting the benchmarks used throughout the thesis and formally defining the metrics computed over them, both the average accuracy and forgetting scores and the complementary confidence and calibration metrics. From there, the covariance estimators and mixture-of-Gaussian variants results are presented, then how well-calibrated their predictions actually are, then how the system behaves as more classes are added over time, and how separable classes are from one another in the embedding space. It then looks at the feature space itself—its sensitivity to the choice of backbone, its spectral structure, and how well it satisfies the Gaussianity assumption the whole framework rests on—and closes by weighing the computational cost of the framework before summarising the practical conclusions that follow from all of this evidence taken together.

Before going further, it is worth fixing the short names used repeatedly in the tables and discussion below. **Diag-Cov** and **Full-Cov** are the two basic covariance parameterisations introduced in Section 4.2: a diagonal covariance matrix, which treats feature dimensions as independent of one another, and a full covariance matrix, which also captures the correlations between them. **Hybrid-Cov** is the intermediate estimator that keeps the full covariance structure for the Mahalanobis, shape-related term of the discriminant, while using only its diagonal for the log-determinant, volume-related term. **OAS** and **LW-I**/**LW-D** are the two shrinkage-based estimators introduced in the same section: Oracle Approximating Shrinkage, and Ledoit–Wolf shrinkage toward an Identity or a Diagonal target, respectively. The prefix **Shared-** (as in Shared-Diag-Cov or Shared-Full-Cov) means that the corresponding covariance is estimated once, globally, and reused across every class, rather than independently per class. Finally, **MOG2** and **MOG4** denote mixture-of-Gaussian variants with exactly two, or with up to four, components per class, respectively; the trailing letter—**D**, **F**, or **H**—again indicates whether each individual component uses a diagonal, full, or hybrid covariance, and **MOG4-Shared** combines the multi-component MOG4 formulation with a single, globally shared full-covariance matrix.

For every benchmark we report the average accuracy after the last task, A_T, and the forgetting score F_T, both formally defined below. Whenever possible we report mean and

variance over three independent runs (different task-order seeds). Since the proposed Gaussian classifiers maintain only sufficient statistics ($\boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c$) and do not involve parameter optimisation during incremental updates, the resulting classifier is independent of the specific class-presentation order; this explains the ± 0.00 standard deviation reported for our methods throughout. State-of-the-art prompt-based competitors are reproduced from the published values reported in their respective papers. Methods proposed in this thesis are highlighted in grey. The best result per column is shown in bold.

5.1 Image Datasets and Benchmarks

Before reporting any empirical result, this section presents the four benchmarks evaluated throughout this thesis. We first introduce the underlying image datasets and what makes each one relevant to this work, and then explain how each one is turned into a class-incremental stream.

5.1.1 Source Datasets

Three source datasets are used to build the benchmarks of this thesis, chosen so that, together, they vary in image resolution, visual style, and category granularity. **CIFAR-100** [56] contains 60,000 low-resolution (32×32) natural images, organised into 100 "fine" classes grouped under 20 broader "coarse" categories. **Caltech-256** [57] contains 30,607 higher-resolution images spread across 256 categories, with more images per class and more visual diversity than CIFAR-100; in this thesis, we use its first 250 categories, for reasons explained below. **ImageNet-R** [58] takes 200 of the classes from the original ImageNet [59] and replaces their photographs with 30,000 non-photographic renditions instead, such as paintings, cartoons, and sketches, which makes it well suited for testing robustness to this kind of stylistic domain shift. A fourth, much larger dataset, **ImageNet-21k** [60], is not itself turned into a benchmark in this section, but is used elsewhere in this thesis as pre-training data, both for the backbones compared in Section 5.8 and for the fixed projection matrix used in the PCA probe of Section 5.9.

5.1.2 Continual Learning Splits and Protocol

Each of the three source datasets above is turned into a class-incremental benchmark by splitting its classes into a fixed sequence of disjoint tasks, following the standard protocol described in Chapter 4: the model sees one task at a time, and average accuracy and forgetting, formally defined in Section 5.2, are computed over the classes seen so far. A fourth benchmark then combines all three streams into a single, longer one. Concretely:

- **Split CIFAR-100** divides the 100 classes into 10 disjoint tasks of 10 classes each.
- **Split ImageNet-R** divides its 200 classes, randomly, into 10 tasks of 20 classes each; because its images are stylistic renditions rather than plain photographs, this split also

introduces a deliberate domain-shift challenge on top of the usual class-incremental one.

- **Split Caltech-256** keeps only the first 250 of the original 256 categories, so that they divide evenly into 10 balanced tasks of 25 classes each (the remaining six categories are simply discarded). The resulting stream has around 28 100 images and an average of $\overline{n_c} \approx 112$ training samples per class, the smallest per-class budget of the three datasets, which makes it the one that puts the most pressure on the high-dimensional covariance estimators of Section 4.2.
- **Split Joint** concatenates the 30 tasks of the three benchmarks above into a single, heterogeneous, 30-task stream, presented to the model in a randomised order. This forces the classifier to move between natural, artistic, and high-resolution images within the same run, and is meant to stress-test the framework over a much longer and more varied stream than any of the three individual benchmarks alone.

5.1.3 Benchmark Properties Summary

The four resulting benchmarks span a wide spectrum of statistical difficulty for a Gaussian classifier. Their salient properties are summarised in Table 5.1.

Table 5.1: Statistical characterisation of the four benchmarks. $\overline{n_c}$ denotes the average sample budget per class.

Benchmark	**# Classes**	**# Tasks**	**Image source**	$\overline{n_c}$
Split CIFAR-100	100	10	Natural, low-resolution (32×32)	500
Split ImageNet-R	200	10	Renditions: art, sketch, cartoon	150
Split Caltech-256	250	10	Natural, high-resolution, variable scale	112
Split Joint	550	30	Heterogeneous concatenation of the above	mixed

The three primary datasets differ along two largely orthogonal axes: visual complexity (which determines how heavy-tailed the embedding coordinates are) and sample budget (which determines how reliably the per-class covariance can be estimated). CIFAR-100 occupies the favourable corner of both axes, ImageNet-R adds a deliberate input-distribution shift, and Caltech-256 combines high visual diversity with a small per-class budget. The Joint benchmark inherits the worst feature of each of the three constituent datasets and is therefore the most informative regime for separating method behaviour. With the benchmarks now fully characterised, we turn to the evaluation metrics computed over them.

5.2 Evaluation Metrics: Average Accuracy and Forgetting

As is common in the continual learning literature [34], we quantify performance using two standard metrics derived from the test accuracy scores obtained after each task: the average

accuracy and the average forgetting.

On the one hand, **Average Accuracy (A_t)**, presented on equation (5.1), measures overall performance after task t. It is the mean test accuracy across all tasks learned up to that point.

$$A_t = \frac{1}{t} \sum_{\tau=1}^{t} S_{t,\tau} \tag{5.1}$$

On the other hand, **Forgetting (F_t)**, presented on equation (5.2), quantifies how much the model has forgotten earlier tasks. It computes, for each past task, how much accuracy has dropped compared to its best historical score, and averages those drops.

$$F_t = \frac{1}{t-1} \sum_{\tau=1}^{t-1} \left(\max_{k \leq t-1} S_{k,\tau} - S_{t,\tau} \right) \tag{5.2}$$

Regarding forgetting, the proposed framework does not exhibit forgetting in the conventional sense, as previously learned class statistics are never modified when new classes are introduced. The measured forgetting values therefore primarily reflect the increased classification difficulty as the number of competing classes grows, rather than degradation of previously learned representations.

5.3 Confidence and Calibration Metrics

Accuracy and forgetting measure whether the model's predictions are correct, but they say nothing about whether the model's own confidence in those predictions can be trusted—a distinction that matters in particular for Class-Incremental Learning, where a classifier can look accurate while still being systematically over- or under-confident. To capture this, we introduce five complementary metrics that jointly characterise the probabilistic behaviour of the classifier: the Expected Calibration Error (ECE) measures the global alignment between confidence and accuracy; the Confidence on Errors (CoE) quantifies how dangerously overconfident the model is when it fails; Total Confidence (TC) and Probability Margin (PM) both capture the decisiveness of the posterior distribution from two complementary angles; and the Average Predictive Entropy (H) measures the spread of probability mass across all classes. These definitions are used directly in Section 5.5, where the resulting values are reported and interpreted.

5.3.1 Expected Calibration Error (ECE)

This metric quantifies the mismatch between predictive confidence and empirical accuracy. We partition the predictions into M equally spaced bins based on their confidence scores.

The ECE is defined as the weighted average of the absolute difference between the accuracy and confidence of each bin:

$$ECE_t = \sum_{m=1}^{M} \frac{|B_m|}{N} \left| acc(B_m) - conf(B_m) \right| \tag{5.3}$$

where N is the total number of samples, B_m is the set of indices of samples whose prediction confidence falls into the m-th interval, and $acc(B_m)$ and $conf(B_m)$ represent the mean accuracy and mean confidence within that bin, respectively.

5.3.2 Confidence on Errors (CoE)

To specifically analyze the model's behavior when it fails, we measure the average confidence assigned to the winning (but incorrect) class. High CoE indicates a model that is "dangerously wrong", which matters in safety-oriented applications. It is computed for a task t as:

$$CoE_t = \frac{1}{|\mathcal{E}_t|} \sum_{i \in \mathcal{E}_t} \max(\hat{p}_i) \tag{5.4}$$

where $\mathcal{E}_t$ is the set of misclassified samples and $\hat{p}_i$ is the predicted probability distribution for sample i.

5.3.3 Total Confidence (TC)

Total Confidence measures how certain the model is on average about its top-1 prediction, regardless of whether that prediction is correct:

$$TC_t = \frac{1}{N} \sum_{i=1}^{N} \max_c \, p_{i,c} \tag{5.5}$$

A value of $TC = 1$ means the model assigns all probability mass to a single class for every sample—it is always maximally decisive. A value close to $1/C$ would indicate near-uniform posteriors. TC is a measure of *decisiveness*, not correctness: a well-calibrated model should have $TC \approx A_T$ (confident when right, uncertain when wrong), whereas a badly overconfident model can reach $TC \approx 1$ even at low accuracy.

5.3.4 Probability Margin (PM)

The Probability Margin is the natural, non-logarithmic measure of how decisively the model separates its top-1 prediction from its second-best alternative:

$$PM_t = \frac{1}{N}\sum_{i=1}^{N}\Big(p_{i,(1)} - p_{i,(2)}\Big) \tag{5.6}$$

where $p_{i,(1)} \geq p_{i,(2)}$ are the two highest posterior probabilities for sample i. A margin near 1 means the model almost entirely ignores its second candidate; a margin near 0 means the model is nearly tied between its top-two guesses and one should treat the prediction with caution. PM is bounded in $[0, 1]$ and directly interpretable: a value of 0.7 means the winning class is, on average, assigned 70% more probability than the runner-up.

5.3.5 Average Predictive Entropy (H)

We utilize Shannon entropy to measure the uncertainty of the model's predictions. A higher entropy suggests that the model is distributing its probability mass across multiple classes, indicating lower certainty. For task t, the average entropy is calculated as:

$$H_t = \frac{1}{N}\sum_{i=1}^{N}\left(-\sum_{c=1}^{C} p_{i,c}\log(p_{i,c})\right) \tag{5.7}$$

where $p_{i,c}$ is the probability assigned to class c for sample i.

Summary of calibration and uncertainty metrics

Table 5.2 collects all six metrics (accuracy and the five calibration/uncertainty indicators), their range, the direction of improvement, and the key question each one answers. Together they provide a multi-dimensional view of the model's reliability throughout continual learning: accuracy tells us *how often* the model is right; ECE tells us *how well-calibrated* its probabilities are; CoE tells us *how dangerous* its mistakes are; TC tells us *how decisive* it is overall; PM tells us *how clearly* it separates its top-two candidates; and entropy tells us *how spread* its uncertainty is across all classes.

With accuracy, forgetting, and the confidence and calibration metrics now all formally established, we turn to the empirical results themselves, beginning with the covariance estimators and mixture-of-Gaussian variants.

Table 5.2: Summary of the probabilistic calibration and uncertainty metrics used in this work. "Better" indicates the direction of the preferred value; "context" means both high and low values are informative depending on whether predictions are correct.

Metric	**Symbol**	**Range**	**Better**	**Key question**
Accuracy	A_T	$[0, 1]$	↑	What fraction of predictions are correct?
Expected Calibration Error	ECE	$[0, 1]$	↓	How well does confidence match actual accuracy?
Confidence on Errors	CoE	$[0, 1]$	↓	How confident is the model when it is *wrong*?
Total Confidence	TC	$[0, 1]$	context	How decisive is the model on its top-1 choice?
Probability Margin	PM	$[0, 1]$	context	How much more likely is top-1 than top-2?
Predictive Entropy	H	$[0, \log C]$	↓	How spread is probability mass over all C classes?

5.4 Covariance Estimator and Mixture-of-Gaussian Results

Table 5.3 reports the average accuracy and the forgetting score for every covariance estimator (diagonal, full, hybrid, Ledoit–Wolf identity/diagonal target [53], OAS [54]) and for the mixture-of-Gaussian variants MOG-k with $k \in \{2, 4\}$ components and three sub-variants (D, F, H for diagonal, full and hybrid covariance respectively). To ensure statistical robustness, all proposed configurations are evaluated across three independent experimental runs using distinct randomized task-presentation seeds, with performance reported as the sample mean alongside its corresponding standard deviation. For the state-of-the-art baselines, these variance metrics are included when available from their original literature sources. The table is organised in three blocks: prompt-based state-of-the-art baselines reproduced from the literature (L2P, DualPrompt, APT); analytic, feature-based baselines (SimpleCIL, FeCAM, RanPAC); and, at the bottom, the proposed Gaussian estimators. Table 5.4 reports the same family on the 30-task Joint stream.

The discussion that follows uses the results of Tables 5.3 and 5.4 to compare the covariance parameterisations (Diag, Full, Hybrid, OAS, Ledoit–Wolf, Shared) against the mixture-of-Gaussian variants.

5.4.1 Comparison with State-of-the-Art Methods

Before looking at the covariance estimators among themselves, it is worth comparing our best results directly against the state-of-the-art baselines reported in Table 5.3. On Split CIFAR-100, MOG4-F reaches 88.87 %, matching ConvPrompt exactly, and staying within about one point of the strongest prompt-based methods, RainbowPrompt and APT. On Split ImageNet-R, MOG4-Shared reaches 80.19 %, which is actually higher than every prompt-based method in the table, including APT (79.40 %). The one clear exception is RanPAC, which stays ahead of all our variants on CIFAR-100 (92.20 % against our best of 88.98 %); we are not hiding this result, since our framework is meant to be competitive, not dominant on every benchmark.

Table 5.3: Continual-learning performance on the three 10-task benchmarks. A_{10} and F_{10} are reported in percentage points.

Method	**Split CIFAR-100**		**Split ImageNet-R**		**Split Caltech-256**	
	$A_{10}(\uparrow)$	$F_{10}(\downarrow)$	$A_{10}(\uparrow)$	$F_{10}(\downarrow)$	$A_{10}(\uparrow)$	$F_{10}(\downarrow)$
L2P	83.86 ± 1.17	7.35 ± 0.39	61.57 ± 0.40	9.73 ± 0.42	92.11 ± 0.32	2.94 ± 0.26
DualPrompt	86.51 ± 0.49	5.16 ± 0.20	68.13 ± 0.52	4.68 ± 0.18	92.50 ± 0.16	2.50 ± 0.16
DualPrompt-PGP	86.92 ± 0.05	5.35 ± 0.19	69.34 ± 0.05	4.53 ± 0.04	—	—
CODA-Prompt	87.00 ± 0.38	4.78 ± 0.24	74.24 ± 0.56	4.92 ± 0.21	—	—
CPrompt	87.83 ± 0.37	4.88 ± 0.79	76.71 ± 0.61	4.66 ± 0.93	—	—
ConvPrompt	88.87 ± 0.33	4.75 ± 0.15	77.86 ± 0.25	4.33 ± 0.24	—	—
RainbowPrompt	89.86 ± 0.11	3.44 ± 0.26	79.09 ± 0.13	3.90 ± 0.23	—	—
APT	88.88 ± 0.65	3.47 ± 0.55	79.40 ± 0.47	3.47 ± 0.55	—	—
SimpleCIL (NMC) [43]	83.70	—	55.56	—	—	—
FeCAM [42]	85.70	—	63.70	—	—	—
RanPAC [44]	**92.20**	—	77.90	—	—	—
Diag-Cov (ours)	86.07 ± 0.00	4.30 ± 0.12	77.12 ± 0.00	7.44 ± 0.17	90.06 ± 0.00	1.69 ± 0.36
Full-Cov (ours)	88.38 ± 0.00	4.12 ± 0.00	74.71 ± 0.00	5.16 ± 0.02	94.93 ± 0.01	1.74 ± 0.05
Hybrid-Cov (ours)	88.38 ± 0.00	4.18 ± 0.00	74.71 ± 0.00	5.03 ± 0.04	94.93 ± 0.00	2.09 ± 0.08
Shared-Diag-Cov (ours)	86.87 ± 0.00	4.72 ± 0.17	75.40 ± 0.00	5.17 ± 0.38	92.97 ± 0.00	2.06 ± 0.02
Shared-Full-Cov (ours)	88.72 ± 0.00	4.68 ± 0.16	77.99 ± 0.00	5.87 ± 1.44	93.62 ± 0.00	2.39 ± 0.09
MOG2-D (ours)	86.96 ± 0.00	4.24 ± 0.02	70.15 ± 0.00	4.93 ± 0.02	91.41 ± 0.00	1.67 ± 0.30
MOG2-F (ours)	85.01 ± 0.00	5.13 ± 0.17	73.42 ± 0.00	5.49 ± 0.26	93.95 ± 0.00	2.14 ± 0.11
MOG2-H (ours)	84.68 ± 0.00	5.26 ± 0.22	73.52 ± 0.00	5.47 ± 0.23	93.93 ± 0.00	2.13 ± 0.10
MOG4-D (ours)	87.80 ± 0.00	4.02 ± 0.06	72.52 ± 0.01	5.06 ± 0.15	91.12 ± 0.00	1.63 ± 0.31
MOG4-F (ours)	88.87 ± 0.00	3.99 ± 0.00	75.07 ± 0.00	5.07 ± 0.05	94.39 ± 0.00	2.12 ± 0.08
MOG4-H (ours)	88.88 ± 0.00	4.04 ± 0.00	75.01 ± 0.00	5.12 ± 0.04	94.36 ± 0.00	2.09 ± 0.08
MOG4-Shared (ours)	88.98 ± 0.00	4.03 ± 0.09	$\mathbf{80.19 \pm 0.00}$	4.44 ± 0.12	$\mathbf{94.61 \pm 0.00}$	1.88 ± 0.09
OAS (ours)	84.38 ± 0.00	3.90 ± 0.07	71.47 ± 0.00	4.57 ± 0.29	86.70 ± 0.00	1.90 ± 1.77
LW-I (ours)	84.78 ± 0.00	3.93 ± 0.09	71.91 ± 0.00	4.54 ± 0.43	89.89 ± 0.00	1.50 ± 0.08
LW-D (ours)	82.85 ± 0.00	3.94 ± 0.28	70.35 ± 0.00	4.62 ± 0.95	90.04 ± 0.00	1.66 ± 0.30

These numbers are close, but they are reached in a very different way. L2P, DualPrompt, CODA-Prompt, and the other prompt-based methods all rely on backpropagation to learn a pool of prompts, which means gradient updates, several training epochs per task, and a non-trivial amount of hyperparameter tuning. Our framework reaches comparable, and in some cases better, accuracy by only computing a mean and a covariance matrix per class, in closed form, without ever touching a gradient. This is also why our methods are reported with a standard deviation of exactly ± 0.00 across different task-presentation orders in Table 5.3: since each class is estimated independently from the others and nothing is optimised, the order in which classes are presented does not affect the final result, unlike methods that learn through gradient descent. Section 5.11 quantifies this cost advantage directly.

5.4.2 Covariance Parameterisations and the Shape-Volume Axis

The performance hierarchy observed across the benchmarks is mostly governed by the ratio of the per-class sample budget n_c to the feature dimensionality $d = 768$. This reflects a trade-off between how much structure the estimator can capture and how numerically stable it remains.

In the relatively high-sample regime of Split CIFAR-100 ($n_c = 400$), estimators that capture full off-diagonal correlations perform best. Both Full-Cov and MOG4-F match the state-of-the-art prompt-based method ConvPrompt at 88.87% without relying on any train-

Table 5.4: Performance on the long-horizon **Split Joint Datasets** benchmark (30 tasks aggregated from CIFAR-100, ImageNet-R and Caltech-256). A_{30} and F_{30} are reported in percentage points.

Method	**Split Joint Datasets**	
	$A_{30}(\uparrow)$	$F_{30}(\downarrow)$
Diag-Cov	78.32 ± 0.21	6.05 ± 0.12
Full-Cov	59.35 ± 0.00	4.69 ± 0.02
Hybrid-Cov	59.35 ± 0.00	3.86 ± 0.04
Shared-Diag-Cov	80.11 ± 0.00	4.89 ± 0.05
Shared-Full-Cov	81.49 ± 0.00	5.49 ± 0.05
MOG2-D	73.91 ± 0.02	5.69 ± 0.02
MOG2-F	78.41 ± 0.02	5.85 ± 0.04
MOG2-H	80.27 ± 0.02	5.04 ± 0.04
MOG4-D	80.28 ± 0.01	4.67 ± 0.00
MOG4-F	59.68 ± 0.25	5.88 ± 0.16
MOG4-H	58.01 ± 0.30	5.97 ± 0.09
MOG4-Shared	$\mathbf{84.49 \pm 0.00}$	3.79 ± 0.12
OAS	77.46 ± 0.00	4.60 ± 0.10
LW-I	76.41 ± 0.01	5.83 ± 0.31
LW-D	77.42 ± 0.02	5.41 ± 0.38

able parameters or upstream gradient optimization. This shows that when the sample budget is large enough to reliably estimate the d^2 free parameters of a covariance matrix, modeling the actual geometric *shape* of the class manifold gives a useful inductive bias that Euclidean or simple prototype-based models cannot capture.

The limits of unconstrained second-order modeling become clear in under-sampled or imbalanced regimes. On Split ImageNet-R ($n_c = 40$), the empirical sample covariance matrix becomes nearly singular, so near-zero eigenvalues corrupt the log-determinant volume term during Cholesky inversion. Under these conditions, the Hybrid-Cov estimator works as a reliable fallback. By using the full empirical structure only for the Mahalanobis distance (the shape term) while replacing the sensitive log-determinant calculation with a well-conditioned diagonal volume approximation, it keeps cross-feature correlations without the numerical breakdown that affects standard full-covariance formulations.

5.4.3 Linear Shrinkage versus Structural Decoupling

One clear finding of this evaluation is that the classical linear shrinkage estimators, OAS and Ledoit–Wolf (LW), consistently underperform: they get some of the lowest accuracy among all the Gaussian-based methods, on every constituent dataset.

Both estimators regularise the sample covariance matrix $\hat{\mathbf{\Sigma}}$ the same way: they replace it with a weighted average $(1-\alpha)\hat{\mathbf{\Sigma}} + \alpha\mathbf{T}$, where $\mathbf{T}$ is a simple, well-conditioned target (the identity matrix, scaled to match the average variance) and $\alpha \in [0, 1]$ is a shrinkage intensity chosen automatically from the data. This fixes rank-deficiency, since $\mathbf{T}$ is always full-rank, but it comes at a cost: the same α is applied to every eigenvalue of $\hat{\mathbf{\Sigma}}$, indiscriminately. The small eigenvalues, which are mostly noise, get pulled toward $\mathbf{T}$, which is what we want. But the large eigenvalues, which correspond to the directions along which classes are actually separated, get pulled toward $\mathbf{T}$ by the same amount, and lose part of the signal that made

them useful in the first place.

This is why Hybrid-Cov and Diag-Cov, described above, come out ahead despite being simpler models: neither one blends the discriminative directions with noise the way OAS and LW do, so the signal that separates classes stays intact instead of being reduced.

5.4.4 Shared Covariance in the Long-Horizon Joint Benchmark

The long-horizon 30-task Joint Dataset Benchmark exposes a clear weakness of independent per-class density estimation, and it shows why it can help to give every class the same covariance matrix instead of estimating one per class. As shown in Table 5.4, both Full-Cov and Hybrid-Cov suffer a sharp performance drop, falling to an identical 59.35% accuracy.

This drop comes from how unevenly the classes are sampled once the three datasets are combined. Because individual classes come from CIFAR-100 ($n_c = 400$), ImageNet-R ($n_c = 40$), and Caltech-256 ($n_c = 64$), their per-class covariance matrices end up built from very different sample sizes. This means the log-determinant and Mahalanobis terms of each class live on different numerical scales, so comparing posteriors across classes stops being reliable.

Shared-Full-Cov fixes this by using one covariance matrix, Σ_{shared}, for every class instead of a separate one each, reaching 81.49% accuracy on the Joint benchmark. Because this single matrix is estimated once from the pooled data of all active classes, every class is scored on the same numerical scale, and the log-determinant term stops depending on how many samples that particular class happened to have.

MOG4-Shared takes this one step further, and it is what performs best of all on the Joint benchmark, at 84.49% (Table 5.4). It keeps the same shared covariance Σ_{shared}, so it stays just as numerically stable as Shared-Full-Cov, but it no longer forces each class to be a single Gaussian: the Bayesian Information Criterion (BIC) is allowed to assign up to four centroids per class instead of one. This matters here because the Joint stream mixes very different kinds of images per class, low-resolution natural photos (CIFAR-100), artistic sketches (ImageNet-R), and clean product photography (Caltech-256), so a single centroid is often not enough to describe a class well.

This is why, when classes come from very different, unevenly sampled sources, using one shared covariance, together with enough per-class centroids to describe their visual variety, works better than trying to estimate a separate, unconstrained covariance for each class.

5.4.5 Overall Best Configuration: MOG4-Shared

Looking at all four benchmarks together, MOG4-Shared is not the single best method in every column of Table 5.3, but it is the only one that stays close to the top on all of them at once, which is why we treat it as the best overall configuration.

On Split CIFAR-100, MOG4-Shared ties with Full-Cov and MOG4-F at 88.87–88.98%, a

little behind RanPAC (92.20%). On Split ImageNet-R, it is the best method in the whole table, 80.19%, ahead of every other Gaussian estimator and every prompt-based baseline, including APT (79.40%). On Split Caltech-256, it stays within one point of the best result (94.61% vs. 94.93% for Full-Cov). On the long, 30-task Joint benchmark, where every other full-covariance variant either drops sharply or needs to be shared to stay stable, MOG4-Shared is clearly the best of all, at 84.49%, ahead of Shared-Full-Cov (81.49%) and far ahead of the unshared full-covariance mixtures (59–60%).

No other configuration in this chapter manages that combination: the estimators that do well on CIFAR-100 and Caltech-256 (Full-Cov, Hybrid-Cov) are exactly the ones that collapse on the Joint benchmark, while the ones built to survive the Joint benchmark (Shared-Diag-Cov, Shared-Full-Cov) give up some accuracy on the easier, single-dataset benchmarks. MOG4-Shared avoids this trade-off because it combines the two ideas developed in the previous subsection: a single shared covariance Σ_{shared} keeps it numerically stable even when classes come from very different, unevenly sampled datasets (Section 5.4.4), while still letting each class use up to four centroids to capture visual variety that a single Gaussian would miss. This is why MOG4-Shared is used as the main reference configuration for the rest of this chapter, whenever a single method needs to stand in for the proposed framework.

This accuracy does not come for free, though. Fitting up to four centroids per class and scoring every one of them against the shared covariance costs more, at both training and inference time, than a single-Gaussian estimator does. Section 5.11 looks at this cost directly and puts it in perspective against the rest of the pipeline.

5.5 Calibration Results and the Cost of High Dimensionality

Table 5.5 reports six metrics that characterize the probabilistic behavior and reliability of the Bayesian classifier at the final after-task checkpoint. While raw accuracy scores show a model's predictive power, checking whether its confidence matches its actual correctness matters for safety-critical or open-world deployment.

Table 5.5: Probabilistic calibration metrics for the Bayesian classifier at the last after-task checkpoint.

Metric	**CIFAR-100**			**ImageNet-R**			**Caltech-256**			**Joint**		
	Diag	Full	MOG4-Shared	Diag	Full	MOG4-Shared	Diag	Full	MOG4-Shared	Diag	Full	MOG4-Shared
Accuracy (%)	86.07	88.38	88.98	77.12	74.71	80.19	90.06	94.93	94.61	78.32	59.35	84.49
ECE (%)	3.72	11.68	8.96	13.72	37.44	5.37	7.05	26.03	4.27	11.46	29.05	10.41
Confidence on Errors	0.467	1.000	0.372	0.656	1.000	0.464	0.805	1.000	0.821	0.742	1.000	0.730
Total Confidence	0.836	1.000	0.800	0.883	1.000	0.824	0.974	1.000	0.984	0.931	1.000	0.925
Probability Margin	0.774	1.000	0.749	0.820	1.000	0.770	0.952	1.000	0.973	0.880	1.000	0.881
Entropy	0.583	0.000	0.856	0.327	0.001	0.680	0.054	0.000	0.043	0.114	0.000	0.232

The results in Table 5.5 show a clear, systematic difference between the confidence profiles of the independent Diagonal, Full and MOG4-Shared covariance models, and how it changes with high-dimensional scaling.

5.5.1 Overconfidence of the Full Covariance Model

The most notable feature of Table 5.5 is that the confidence profiles saturate on every single benchmark under the Full covariance parameterisation. For CIFAR-100, ImageNet-R, Caltech-256, and the Joint stream alike, the Full covariance model yields a Confidence on Errors (CoE) of exactly 1.000, a Total Confidence (TC) of 1.000, a Probability Margin (PM) of 1.000, and a predictive Entropy that drops to 0.000 (with a small numerical artifact of 0.001 on ImageNet-R). This means the model is badly overconfident: whenever it makes a prediction, it places 100% of its softmax probability mass onto the winning class, ignoring every alternative.

This is not an accidental property of the dataset distributions, but a direct, predictable outcome of the high-dimensional small-sample rank-deficiency problem described earlier. When a full covariance matrix $\mathbf{\Sigma}_c \in \mathbb{R}^{d\times d}$ is estimated from a restricted sample pool ($n_c < d = 768$), it is rank-deficient by construction. Bounding its near-zero eigenvalues with a fixed diagonal ridge regularization floor ($\varepsilon = 10^{-6}\mathbf{I}$) allows for Cholesky inversion, but it also introduces a serious artifact into the log-determinant volume calculation and the Mahalanobis quadratic form.

The regularisation floor ε causes this saturation. Because $n_c < d = 768$, most feature directions carry almost no real variance for a given class, so their eigenvalues end up pinned near ε after regularisation. Because the Mahalanobis distance divides by these eigenvalues, any small deviation along one of these near-empty directions gets scaled up by $1/\varepsilon$, a very large number. A test point that is not an almost perfect match to a class's shape is punished heavily for it.

5.5.2 Calibration of the Diagonal Model

In clear contrast to the Full model's blind overconfidence, the Diagonal covariance model shows well-calibrated probabilistic behavior. Restricting $\mathbf{\Sigma}_c$ to its diagonal keeps it well-conditioned even under data scarcity, and this shows directly in the numbers: Total Confidence tracks accuracy closely on every benchmark (e.g. 0.836 TC for 86.07% accuracy on CIFAR-100), which is why ECE stays low throughout (3.72% to 13.72%). The model also keeps a meaningful runner-up probability instead of betting everything on one class, seen in the steady 0.02–0.06 gap between Total Confidence and Probability Margin. This gap narrows on Caltech-256 and the Joint stream, where the classes are easier to separate and the model can afford to be more decisive without becoming overconfident.

5.5.3 Calibration of MOG4-Shared Model

MOG4-Shared avoids both failure modes above: its shared covariance Σ_{shared} stays well-conditioned, so it never saturates like Full-Cov, and its ECE stays reasonably low throughout. Compared to Diag-Cov it does even better on ImageNet-R, Caltech-256, and Joint (ECE of 5.37%, 4.27%, and 10.41% against 13.72%, 7.05%, and 11.46%), but worse on CIFAR-100

(8.96% against 3.72%), even though its accuracy there is slightly higher. This reflects its higher predictive entropy: using multiple centroids per class with a shared covariance spreads probabilities more evenly across competing classes, preventing overconfident predictions. On CIFAR-100 this extra spread is not matched by a higher true error rate, so calibration suffers a little; elsewhere it works in MOG4-Shared's favor, keeping it from being as overconfident as Diag-Cov on the mistakes it does make.

These metrics point to a practical trade-off for generative lifelong learning: unconstrained Full covariance matrices can extract fine-grained correlational shape details to maximize raw accuracy in balanced domains, but they do so at the cost of poor calibration. MOG4-Shared reaches comparable or better raw accuracy without collapsing into that same overconfidence, and its calibration is competitive with, or better than, Diag-Cov on three of the four benchmarks. Still, for applications where meaningful uncertainty estimates matter most, out-of-distribution detection or task-free deployment among them, Diag-Cov remains the safer default, since it is the only parameterisation whose calibration error stays low and consistent across every benchmark studied here.

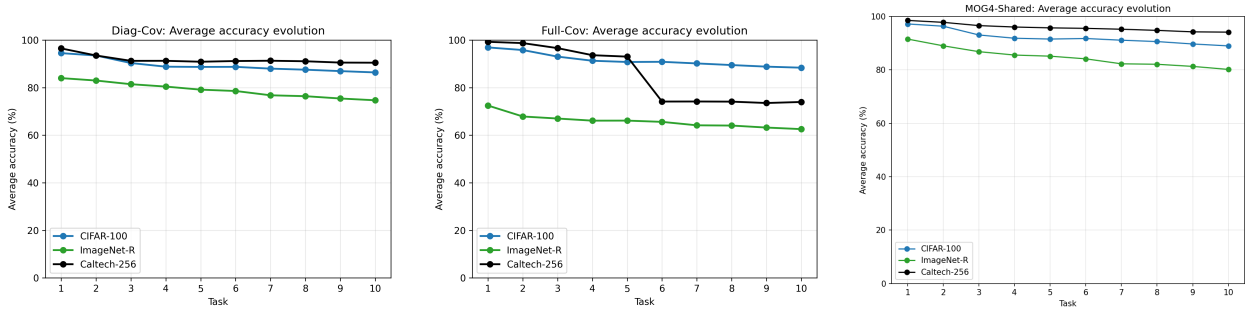
5.6 Incremental Class Inclusion

We design a class-incremental evaluation protocol to check how each closed-form estimator scales as the classification task grows harder. Each class-conditional Gaussian is estimated independently and frozen once its class is added, so this is not the usual test for catastrophic forgetting. Instead, by sequentially expanding the set of active classes and tracking how average accuracy evolves as more categories are added, we isolate a different effect: how a growing label space makes the decision surface itself harder, simply because there are more prototypes to tell apart.

Figure 5.1 shows the average accuracy across ten sequential tasks for the Diag-Cov (**a**), Full-Cov (**b**), and MOG4-Shared (**c**) estimators, on the three benchmarks (CIFAR-100, ImageNet-R, and Caltech-256). At each task index t, the average accuracy is computed over all classes seen so far, so the curves track how well the system keeps recognising the growing set of classes as new ones are added.

On all three benchmarks and for all three estimators, average accuracy declines as more tasks are added, typically starting in the mid-to-high 90 % range after the first task and settling somewhere in the 60–90 % range by the tenth. Full-Cov on Caltech-256 shows the steepest single-step drop, falling from roughly 93 % to 74 % between tasks 5 and 6, after which it stabilises. Diag-Cov degrades more smoothly on every benchmark. MOG4-Shared behaves closer to Diag-Cov than to Full-Cov: it also declines smoothly, without sharp single-task drops, and it ends the sequence with the highest accuracy of the three estimators on CIFAR-100 and close to the top on Caltech-256. Across all three estimators, ImageNet-R shows the most gradual decline of the three benchmarks.

This decline should not be read as forgetting in the usual sense. Every class-conditional Gaussian is estimated independently and frozen the moment its class is added, so no parameters from previously learned classes are ever overwritten: each class keeps, unchanged,



(a) Diag-Cov (b) Full-Cov (c) MOG4-Shared

Figure 5.1: Evolution of average accuracy across ten sequential tasks for CIFAR-100 (blue), ImageNet-R (green), and Caltech-256 (black), under (**a**) Diag-Cov, (**b**) Full-Cov, and (**c**) MOG4-Shared estimation.

the exact representation it was given when it was first learned. What grows with t is not error on old classes caused by new ones, but the difficulty of the classification problem itself, since every extra task adds more competing prototypes for the Bayesian decision rule to tell apart. Figure 5.5 quantifies this same mechanism through the decay of the minimum pairwise Bhattacharyya distance: as more classes are added, some pairs of prototypes inevitably move closer together in feature space, which raises the chance of confusion even though none of the individual class models has changed. The accuracy drop shown here is therefore a harder decision surface, not lost knowledge, and it matches the low forgetting values already reported in Table 5.3. The next section develops this separability analysis in full.

5.7 Inter-Class Separability

Before asking why some classes are harder than others, it helps to look at how accuracy is actually spread across classes once every one of them has been added to the stream. This section starts from that per-class picture, and then uses the geometry of the embedding space, through the Bhattacharyya and Euclidean distances between class prototypes, to explain why some classes end up harder to tell apart than others.

5.7.1 Per-Class Accuracy Once All Classes Are Added

Once every task has been added to the stream, the final overall accuracy evaluated across the complete label space reaches 86.6%, 72.5%, and 90.2% under Diag-Cov for CIFAR-100, ImageNet-R, and Caltech-256, respectively, while Full-Cov attains 88.9%, 74.7%, and 94.7% on these same benchmarks. It is important to note that these values represent the classification accuracy over all classes at the final task checkpoint ($S_{T,T}$), rather than the average incremental accuracy (A_T) computed across all intermediate learning sessions. Figures 5.2–5.4 break these overall numbers down by class, illustrating how performance is distributed across individual categories once the full label space is populated.

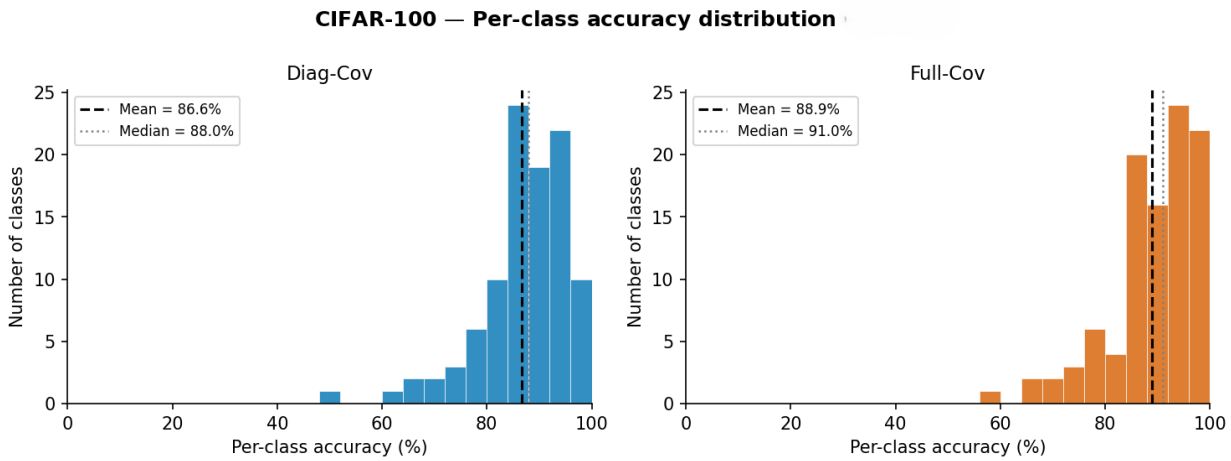


Figure 5.2: Per-class accuracy once all classes have been added, for CIFAR-100 ($N = 100$ classes). Left: Diag-Cov. Right: Full-Cov.

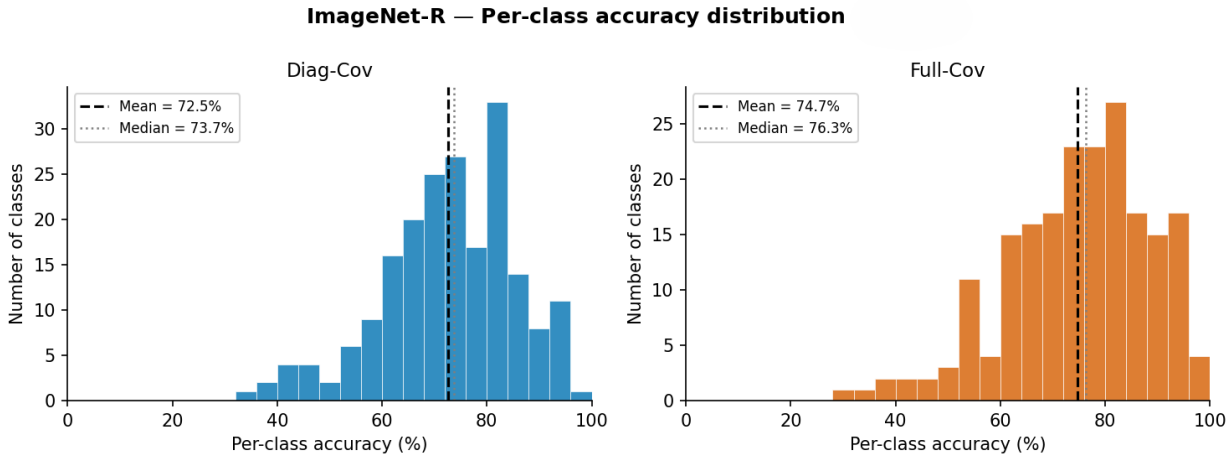


Figure 5.3: Per-class accuracy once all classes have been added, for ImageNet-R ($N = 200$ classes).

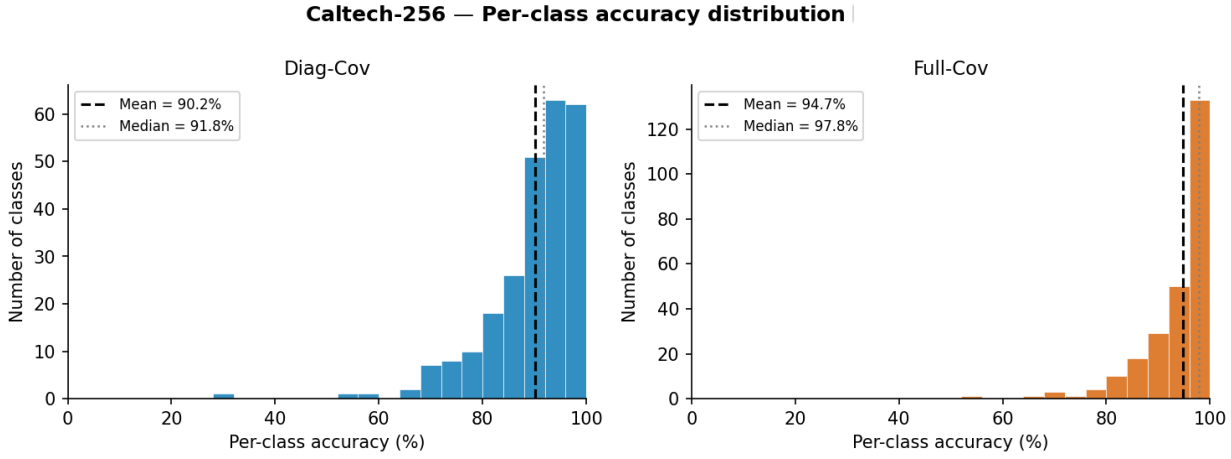


Figure 5.4: Per-class accuracy once all classes have been added, for Caltech-256 ($N = 250$ classes).

The shape of these histograms reveals clear structural differences across benchmarks:

On Split CIFAR-100 (Figure 5.2), the per-class accuracy distributions for both Diag-Cov and Full-Cov are unimodal and skewed toward the highest accuracy bins, with medians of 88.0% and 91.0%, respectively. Most categories form representations in the feature space that single-component Gaussians capture adequately, leaving only a small tail of confusable categories below the 70% mark.

Split ImageNet-R (Figure 5.3), by contrast, exhibits a dispersed, flat distribution with a heavy lower tail extending below 40% accuracy. Because the dataset consists of non-photographic renditions (e.g., stylized art, sketches, and cartoons), the frozen foundation encoder maps instances into broader, more diffuse distributions. This elevated intra-class variance limits the capacity of a single unimodal Gaussian to fit the data tightly, resulting in broader confusion across neighboring categories.

Split Caltech-256 (Figure 5.4) displays an even sharper concentration at the upper bound. Under Full-Cov estimation, the median accuracy shifts to 97.8%, placing the majority of classes in a near-perfect recognition regime. Because most categories exhibit high visual consistency, a single Gaussian fits their distribution well, resulting in the vast majority of classes achieving over 70% accuracy.

As explained in the incremental analysis, this gap between classes cannot come from forgetting in the usual sense: each class-conditional Gaussian is estimated once and frozen the moment its class is added, so nothing about how well an old class is modelled can change afterwards. If some classes still end up much harder than others once every task has been seen, the cause has to be something else, namely how close different classes actually sit to each other in the embedding space. The rest of this section looks at that directly

5.7.2 Class Overlap: Bhattacharyya and Euclidean Distances

To see how close different classes sit to each other in the embedding space, we use two complementary measures: the *Bhattacharyya distance* D_B, which accounts for differences in both class means and covariances, and the plain *Euclidean distance* $d_E = \|\boldsymbol{\mu}_i - \boldsymbol{\mu}_j\|_2$ between class mean vectors, which isolates mean separation alone.

The scaling trajectories of the pairwise Bhattacharyya distances (Figures 5.5 and 5.6) and mean Euclidean distances (Figures 5.7 and 5.8) show a consistent pattern across the foundation embedding manifolds. Rather than an overall compression of the representation space as tasks accumulate, the global layout of the features stays fairly stable, while performance is limited mainly by a small subset of highly overlapping class pairs.

One sign of this stability is that both macro-averaged distance metrics plateau quickly across all benchmarks once around 30 categories are introduced. At full class-set size, the mean pairwise D_B settles at 89.43 for CIFAR-100, 84.81 for ImageNet-R, and 220.61 for Caltech-256. This same ordering is matched by the macro-averaged mean Euclidean distance, where CIFAR-100 and ImageNet-R stay more densely packed at $\overline{d_E} = 0.749$ and $\overline{d_E} = 0.727$, respectively, while Caltech-256 maintains a wide geometric separation of $\overline{d_E} = 0.978$. Caltech-

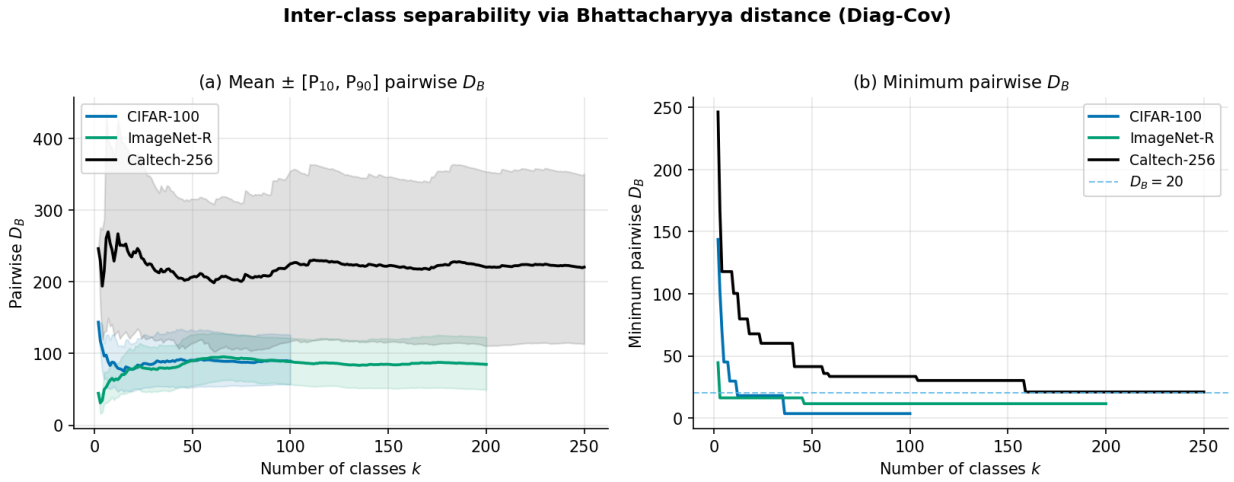


Figure 5.5: Inter-class separability under the Diag-Cov Gaussian model as the class set grows from $k = 2$ to the full benchmark size. **(a)** Mean pairwise D_B with shaded $[\mathrm{P}_{10}, \mathrm{P}_{90}]$ band. **(b)** Minimum pairwise D_B; the dashed horizontal line marks $D_B = 20$.

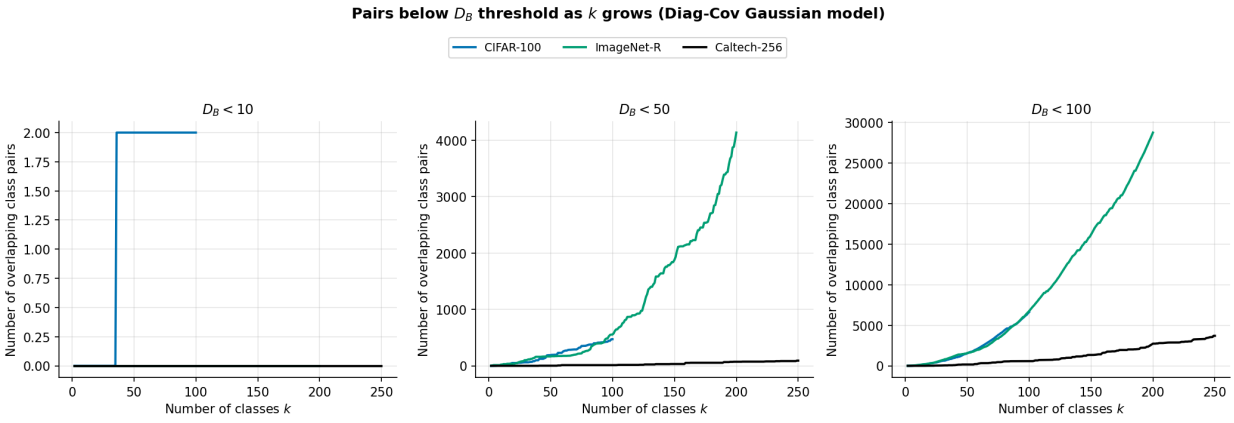


Figure 5.6: Count of class pairs below three overlap thresholds ($D_B < 10$, $D_B < 50$, $D_B < 100$) as the class set grows. Over 99.8 % of all pairs exceed $D_B = 50$ on every benchmark.

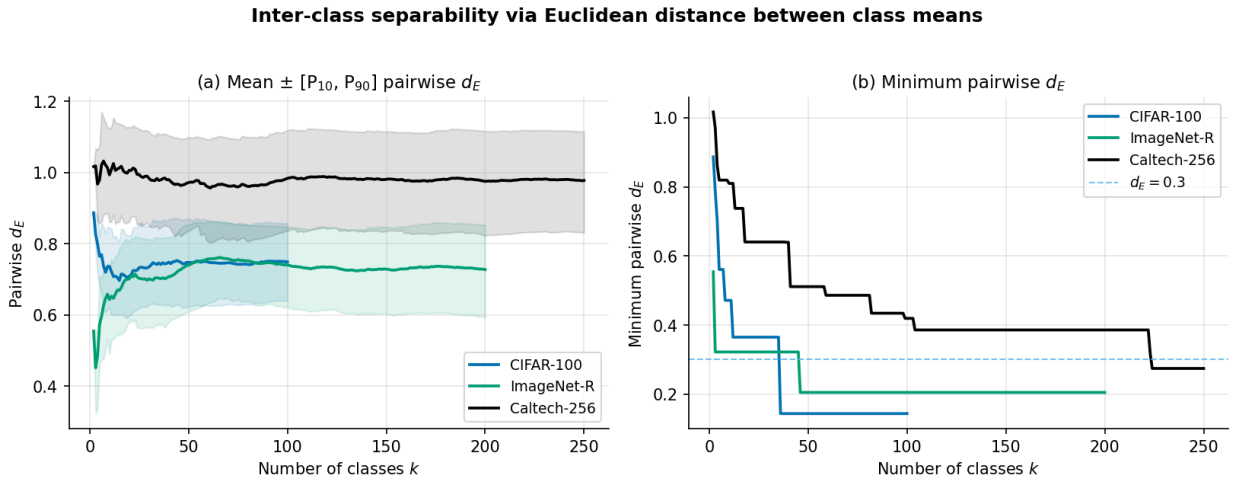


Figure 5.7: Inter-class separability in terms of the plain Euclidean distance d_E between class means, as the class set grows from $k = 2$ to the full benchmark size. **(a)** Mean pairwise d_E with shaded $[\mathrm{P}_{10}, \mathrm{P}_{90}]$ band. **(b)** Minimum pairwise d_E.

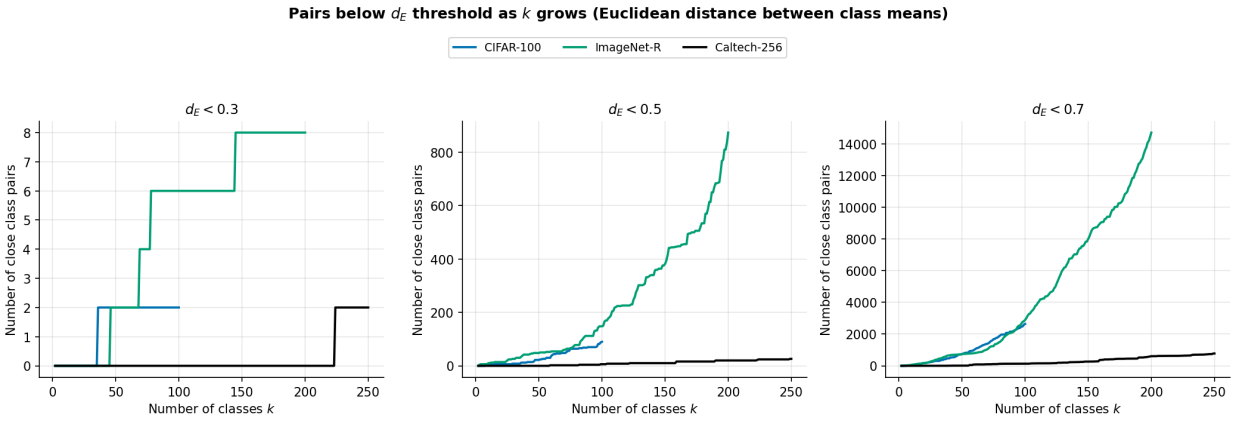


Figure 5.8: Count of class pairs below three Euclidean-distance overlap thresholds as the class set grows, complementing the Bhattacharyya-based count of Figure 5.6.

256's clear separability advantage across both metrics reflects its greater visual and semantic diversity, which keeps per-class distributions distinct.

However, as the label space grows denser, the minimum pairwise distance metrics decay steadily, and this localized crowding is the primary driver behind the per-class accuracy discrepancies documented in Section 5.7.1. Once every class has been added, the minimum pairwise D_B drops to 3.57 on CIFAR-100, 11.54 on ImageNet-R, and 20.96 on Caltech-256. This localized proximity is corroborated by the Euclidean mean separation, which reaches minimum values of 0.144, 0.205, and 0.275 for the respective datasets.

Tracking the density thresholds shows that this crowding is an isolated effect rather than a general collapse of the representation space. Over 99.8 % of all possible class combinations stay well above a threshold of $D_B = 50$ on every evaluated benchmark. Similarly, class pairs with $d_E < 0.5$ make up only a small fraction of the total, accounting for 0.91 % of pairs in CIFAR-100, 2.20 % in ImageNet-R, and just 0.04 % in Caltech-256.

Consequently, the classifier does not operate within a globally compromised feature space. Instead, performance ceilings are determined by a sparse set of highly confusable prototype pairs, directly accounting for the tail of low per-class accuracy observed on ImageNet-R in Section 5.7.1.

The two metrics serve as complementary diagnostics for these failure points. While d_E isolates pure first-order geometric separation between prototype centroids, D_B also accounts for covariance geometry and volume differences. This explains why a pair such as *boy/girl* in CIFAR-100 stands out as the single most severe point of confusion in the system: it combines an exceptionally small centroid separation ($d_E = 0.145$) with nearly identical, overlapping covariance structures, driving probabilistic separation to its minimum. To identify which specific categories are affected and examine their geometry in detail, the next subsection analyzes the worst-performing classes and their corresponding confusion sinks.

5.7.3 Worst-Performing Classes and Failure Diagnostics

Breaking macro-level performance metrics down into class-specific failure points gives a clearer view of the resolution limits of the visual foundation embedding space and the boundaries of each covariance parameterisation. To isolate these failure cases, we rank and track the five object categories with the lowest per-class accuracy scores across all benchmarks.

We analyze these failure patterns across two parameterisations: Table 5.6 characterizes the five lowest-performing classes under the independent feature assumptions of the Diagonal Covariance model, while Table 5.7 evaluates the corresponding distribution shifts and re-orderings under the regularized Full Covariance parameterisation. For both, the table records the per-class accuracy (Class-Acc%), the dominant confusion sink, the localized pairwise Bhattacharyya distance (D_B), and the Euclidean distance (d_E) to that primary target. These are compared directly against the dataset-level macro-averaged distances ($\overline{D_B}$ and $\overline{d_E}$) established in Section 5.7.2. Localized metrics falling well below these macro-averages give a direct geometric indication of significant prototype overlap.

Table 5.6: Bottom-5 accuracy classes under Diag-Cov. Conf.%: error rate to primary confusion target. D_B, d_E: target Bhattacharyya/Euclidean distances ($\overline{D_B}, \overline{d_E}$: dataset averages).

Dataset	Class	Class-Acc%	Main Confusion	Conf.%	D_B	$\overline{D_B}$	d_E	$\overline{d_E}$
Split CIFAR-100	boy	50.0	girl	30.0	3.45	89.43	0.145	0.749
	maple_tree	60.0	oak_tree	19.0	19.41	89.43	0.327	0.749
	girl	65.0	boy	23.0	3.45	89.43	0.145	0.749
	cloud	65.0	sea	8.0	30.54	89.43	0.451	0.749
	shrew	69.0	mouse	16.0	16.45	89.43	0.331	0.749
Split ImageNet-R	cabbage	35.3	vase	17.6	50.50	84.81	0.587	0.727
	saint_bernard	36.4	basset_hound	9.1	66.44	84.81	0.643	0.727
	bow_tie	39.3	scarf	21.4	16.57	84.81	0.331	0.727
	beaver	40.0	backpack	8.6	34.74	84.81	0.500	0.727
	bee	41.6	fly	15.6	20.51	84.81	0.378	0.727
Split Caltech-256	top-hat	31.2	cowboy-hat	31.2	29.75	220.61	0.319	0.978
	paperclip	52.6	drinking-straw	21.1	62.94	220.61	0.645	0.978
	canoe	57.1	people	28.6	57.87	220.61	0.641	0.978
	drinking-straw	64.7	people	11.8	27.33	220.61	0.465	0.978
	traffic-light	65.0	people	15.0	66.27	220.61	0.675	0.978

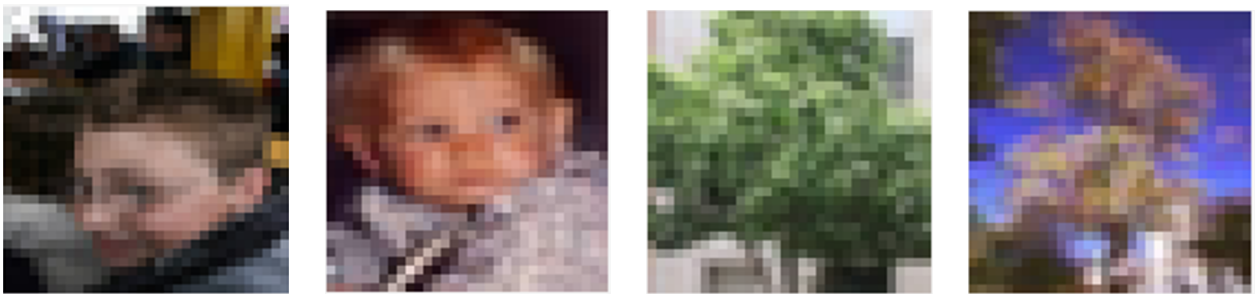
Table 5.7 details the equivalent classification landscape under the regularized Full Covariance model. On Split CIFAR-100, the most difficult classes remain largely the same between both models. On Split Caltech-256, however, the ranking changes considerably because modeling feature correlations helps separate classes with complex boundaries. The dataset averages $\overline{D_B}$ and $\overline{d_E}$ do not change, as they measure the overall spacing of the frozen features across the benchmark.

Table 5.7: Bottom-5 accuracy classes under Full-Cov. Conf.%: error rate to primary confusion target. D_B, d_E: target Bhattacharyya/Euclidean distances ($\overline{D_B}, \overline{d_E}$: dataset averages).

Dataset	Class	Class-Acc%	Main Confusion	Conf.%	D_B	$\overline{D_B}$	d_E	$\overline{d_E}$
Split CIFAR-100	boy	58.0	girl	16.0	3.45	89.43	0.145	0.749
	girl	67.0	boy	15.0	3.45	89.43	0.145	0.749
	otter	67.0	seal	8.0	16.42	89.43	0.342	0.749
	maple_tree	68.0	oak_tree	19.0	19.41	89.43	0.327	0.749
	forest	68.0	pine_tree	7.0	25.46	89.43	0.422	0.749
Split ImageNet-R	bow_tie	28.6	cowboy_hat	14.3	14.51	84.81	0.323	0.727
	bee	33.8	fly	11.7	20.51	84.81	0.378	0.727
	beaver	37.1	backpack	8.6	34.74	84.81	0.500	0.727
	cowboy_hat	39.0	hammer	6.8	21.01	84.81	0.404	0.727
	candle	40.7	stole	9.3	19.73	84.81	0.379	0.727
Split Caltech-256	drinking-straw	52.9	soda-can	11.8	51.11	220.61	0.614	0.978
	people	66.7	picnic-table	4.8	55.38	220.61	0.619	0.978
	top-hat	68.8	wine-bottle	6.2	100.90	220.61	0.802	0.978
	american-flag	70.0	speed-boat	5.0	108.32	220.61	0.823	0.978
	bowling-ball	71.4	bowling-pin	14.3	68.73	220.61	0.644	0.978

To ground these proximity numbers in something visual, we look at typical images taken directly from the benchmark data streams. Figures 5.9, 5.10, and 5.11 present the target classes side by side with their main cross-category confusion partners, the samples most frequently confused by the classifier. Examining these cross-class pairs illustrates the visual and semantic similarities that drive the lower tail of the per-class accuracy distributions.

This qualitative visual inspection confirms the reasoning behind the statistical proximity

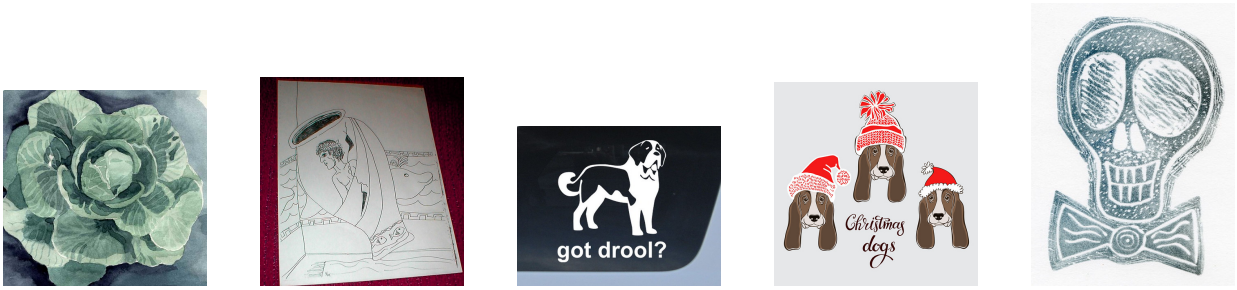


(a) Boy (b) Girl (c) Oak Tree (d) Maple Tree



(e) Cloud (f) Sea (g) Shrew (h) Mouse

Figure 5.9: Representative visual specimens of the primary low-accuracy classes and corresponding confusion sinks on the Split CIFAR-100 benchmark, mapped to the tracking trends outlined in Table 5.6 and Table 5.7.



(a) Cabbage (b) Vase (c) Saint Bernard (d) Basset Hound (e) Bow Tie



(f) Scarf (g) Beaver (h) Backpack (i) Bee (j) Fly

Figure 5.10: Representative specimens from Split ImageNet-R mapped to the tracking trends outlined in Table 5.6 and Table 5.7, showing the non-photographic style of the dataset.



(a) Top Hat



(b) Cowboy Hat



(c) Paperclip



(d) Drinking Straw



(e) Canoe



(f) Traffic Light



(g) People

Figure 5.11: Representative visual entities comprising the minimum separability boundary on the Split Caltech-256 stream mapped to the tracking trends outlined in Table 5.6 and Table 5.7.

metrics recorded in our diagnostic tables. Categories such as boy and girl (Figure 5.9a–b), captured at the limited 32×32 pixel resolution of Split CIFAR-100, share nearly identical facial structures, skin tone distributions, and foreground composition. Similarly, the overlap between cloud and sea (Figure 5.9e–f) is driven by very similar global color palettes and low-frequency texture statistics, where the boundary between a cloudy sky and an ocean horizon becomes visually continuous. Given these strong semantic and stylistic similarities, it is expected from a generative perspective that their respective class-conditional feature distributions end up in near-coincident regions of the embedding space. The resulting cross-class confusion is therefore not caused by parameter degradation or catastrophic forgetting, but is a natural consequence of modeling independent probability densities over visual representations that are genuinely ambiguous.

5.7.4 Synthesis of Inter-Class Confusion and Structural Bounds

The pairwise relationships extracted from the distributions once every class has been added (Tables 5.6 and 5.7) give a localized explanation for the separability trends seen across the benchmarks. The framework itself is not breaking down; the residual classification error concentrates in a small tail of highly confusable class pairs, and that tail is what shows where the representation's semantic boundaries actually sit.

This localized failure breaks down into three patterns:

- **Semantic and Visual Near-Synonyms:** The most notable confusion comes from pairs that share basic silhouette, texture, or composition under DINOv2's representation. Pairs such as boy↔girl ($D_B = 3.45$, $d_E = 0.145$), bow_tie↔scarf ($D_B =$

16.57), and top-hat↔cowboy-hat ($D_B = 29.75$) form overlapping distributions because a general-purpose frozen encoder prioritizes shared global semantics over subtle, fine-grained differences.

- **Contextual and Manifold Sinks:** Proximity in the embedding space often comes from shared background or composition rather than category membership. This is shown by the overlap of cloud↔sea ($D_B = 30.54$) through matching color palettes, and by broad categories like people acting as confusion sinks for unrelated objects (canoe, drinking-straw) that happen to be photographed with human subjects in frame.
- **The Mechanics of Apparent Forgetting:** This pairwise crowding is what actually explains the modest rise in forgetting recorded throughout the stream. Class-conditional models are estimated independently and frozen, so the representations themselves do not degrade over time. Apparent forgetting comes strictly from the growth of the label space, which raises the chance of inserting a new category close to one already learned.

This also shows the limits of downstream parametric adaptation. Upgrading the modeling architecture from a Diagonal to a Full covariance matrix captures useful correlational structure (recovering boy from 50.0 % to 58.0 % recall), but it cannot fully separate distributions whose means are nearly the same ($d_E \to 0$). Resolving that remaining overlap is a task for representation alignment at the backbone level. To see how much the whole class-incremental pipeline depends on these native feature distributions, the next section evaluates the sensitivity of the Gaussian framework to the choice of the upstream feature extractor itself.

5.8 Sensitivity to the Choice of Backbone

DINOv2 [12] is the default backbone for the Gaussian-based classifiers in this thesis, chosen for its self-supervised training and the close-to-Gaussian, well-conditioned class-conditional distributions shown in Section 3.2. The framework itself does not depend on this particular choice: any frozen encoder that produces fixed-dimensional embeddings can plug into the downstream Gaussian classifier unchanged. To see how much the framework actually depends on the backbone, we re-run Diag-Cov, Full-Cov, and MOG4-Shared using two supervised Vision Transformers from the `timm` library [61], **ViT-base/16** (ViT-b, $d = 768$) and **ViT-medium/16** (ViT-m, $d = 512$), both pre-trained on ImageNet-21k [60], keeping the covariance estimation, hyperparameters, and evaluation protocol identical across backbones.

Table 5.8 reports this comparison across the three benchmarks, for the two best closed-form estimators plus MOG4-Shared, evaluated with three vision backbones: DINOv2-B/14 [12] (the default used throughout this thesis), ViT-B/16, and ViT-Medium (ViT-M/16).

Table 5.8: Impact of backbone architecture on Diag-Cov, Full-Cov, and MOG4-Shared across three benchmarks.

Method	Backbone	CIFAR-100		ImageNet-R		Caltech-256	
		$A_{10}(\uparrow)$	$F_{10}(\downarrow)$	$A_{10}(\uparrow)$	$F_{10}(\downarrow)$	$A_{10}(\uparrow)$	$F_{10}(\downarrow)$
Diag-Cov	DINOv2-B/14	86.07 ± 0.00	4.30 ± 0.12	$\mathbf{77.12 \pm 0.00}$	7.44 ± 0.17	90.06 ± 0.00	1.69 ± 0.36
	ViT-M/16	81.41 ± 0.00	5.28 ± 0.13	66.11 ± 0.00	$\mathbf{4.79 \pm 0.14}$	$\mathbf{91.41 \pm 0.00}$	$\mathbf{1.63 \pm 0.12}$
	ViT-B/16	81.03 ± 0.00	4.57 ± 0.41	43.08 ± 0.00	6.41 ± 0.19	87.72 ± 0.00	1.74 ± 0.50
Full-Cov	DINOv2-B/14	$\mathbf{88.38 \pm 0.00}$	$\mathbf{4.12 \pm 0.00}$	74.71 ± 0.00	5.16 ± 0.02	$\mathbf{94.93 \pm 0.01}$	1.74 ± 0.05
	ViT-M/16	82.42 ± 0.01	5.25 ± 0.01	66.44 ± 0.01	5.21 ± 0.14	92.14 ± 0.00	2.53 ± 0.12
	ViT-B/16	82.40 ± 0.00	5.27 ± 0.36	46.23 ± 0.00	6.34 ± 0.12	87.13 ± 0.00	1.92 ± 0.80
MOG4-Shared	DINOv2-B/14	$\mathbf{88.98 \pm 0.00}$	4.03 ± 0.09	$\mathbf{80.19 \pm 0.00}$	$\mathbf{4.44 \pm 0.12}$	94.61 ± 0.00	1.88 ± 0.09
	ViT-M/16	85.57 ± 0.00	5.04 ± 0.09	70.03 ± 0.00	3.67 ± 0.10	93.93 ± 0.00	2.27 ± 0.08
	ViT-B/16	84.59 ± 0.00	4.72 ± 0.06	47.20 ± 0.00	4.64 ± 0.13	89.49 ± 0.00	2.01 ± 0.140

5.8.1 Interpreting the Backbone Comparison

Table 5.8 shows that backbone choice is the single largest factor for the framework. DINOv2 beats the supervised alternatives by 5–35 accuracy points on CIFAR-100 and ImageNet-R; ViT-B/16 in particular drops to 43–47 % on ImageNet-R. Caltech-256 is the exception: all three backbones land within about four accuracy points of each other, since its categories are already easy to separate visually, which moves the bottleneck from the encoder to the downstream classifier and makes the choice of backbone matter less.

MOG4-Shared follows the same pattern as Diag-Cov and Full-Cov, and it is the best performer with every backbone tested. With DINOv2 it reaches 88.98%/80.19%/94.61% on CIFAR-100/ImageNet-R/Caltech-256, at or above the best of the other two estimators on every benchmark. Moving to ViT-M/16 or ViT-B/16 costs it roughly the same, proportionally, as it costs Diag-Cov and Full-Cov, so a weaker backbone drags down every estimator by a similar margin rather than singling one out. In other words, DINOv2 is not just the better choice for this particular estimator; it is the better choice regardless of which covariance estimator sits on top of it.

Beyond the covariance estimators and backbones evaluated above, the next section turns to the statistical structure of the embedding space itself, beginning with its spectral geometry.

5.9 Spectral Analysis of the DINOv2 Embedding Space

Chapter 3 introduced spectral decay: how quickly the eigenvalues of the feature covariance shrink tells us whether an embedding is genuinely high-dimensional or effectively lives in a much smaller subspace. Here we apply that idea to the DINOv2 embeddings used throughout this chapter, computing the empirical spectrum of the ℓ_2-normalised features, pooled across each benchmark's training split.

One notable feature of this spectrum is how similar it looks across very different image distributions. As shown in Figure 5.12, the decay curves for Split CIFAR-100, Split ImageNet-R, and Split Caltech-256 overlap almost exactly, so DINOv2 spreads variance across its coordinates in roughly the same way whether it is processing low-resolution natural images,

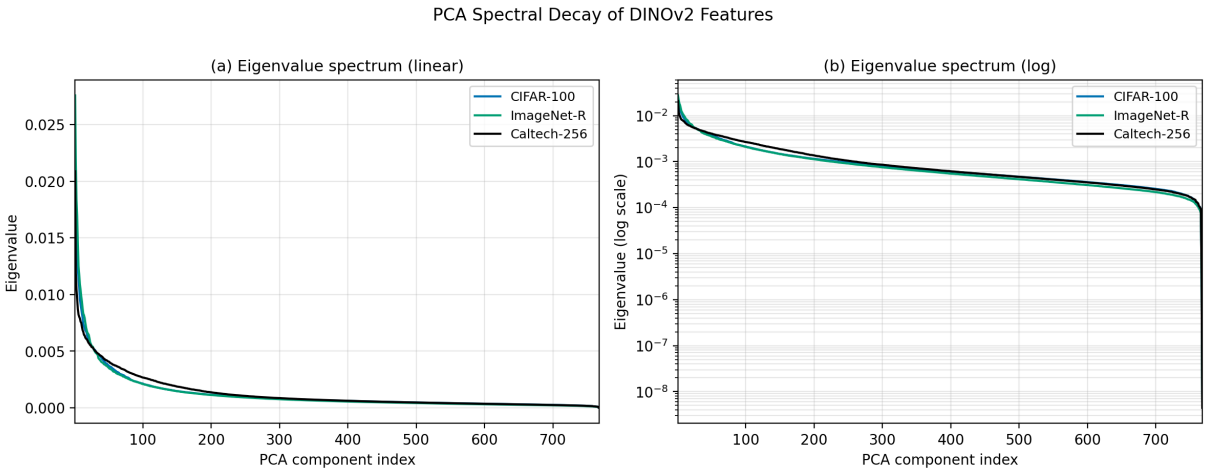


Figure 5.12: PCA spectral decay of the ℓ_2-normalised DINOv2 features. **(a)** Individual % variance explained per component: steep decay over the first ~ 50 components, near-zero contribution thereafter. **(b)** Cumulative % variance explained: dashed reference lines mark 50, 80, 90 and 95 %; annotated n values show how many components suffice to reach 90 % on each dataset. Despite the $d = 768$ embedding space, 90 % of the variance is captured by fewer than 130 components on every benchmark.

artistic renditions, or high-resolution object photography. Because of this, there is no need to tune the dimensionality separately for each dataset, and the same probabilistic modelling layer can be used across all of them.

5.9.1 Spectral Properties of the Foundation Embeddings

The spectrum in Figure 5.12 shows three consistent patterns across the DINOv2 feature space.

First, all three benchmarks allocate variance in a similar multi-stage way. The first principal component accounts for roughly 8%–10% of the total variance, followed by a sharp decay over the first ~ 50 dimensions that together capture 36%–40% of the global variance. Beyond that point, the spectrum flattens into a long tail that decays slowly across the remaining 700+ coordinates.

Second, there is no abrupt cutoff in the spectrum. Most of the discriminative signal sits in a moderate-dimensional subspace, but it is not confined to it entirely: some real variability is spread across the whole ambient space. This is why we use a full, high-dimensional ($d = 768$) generative model instead of a truncated, PCA-reduced one by default.

Third, and independently of this global spectrum, the effective rank of any per-class empirical covariance matrix is bounded above by $n_c - 1$, since at most $n_c - 1$ directions can be populated from n_c centred observations. When $n_c \leq d$, this bound alone forces rank deficiency, and it is the root cause of the numerical instability we saw earlier in unconstrained full-covariance estimation. The next subsection tests directly whether the concentration of variance shown here can be used to work around it.

5.9.2 Empirical Validation via Subspace Compression Tracking

If most of the variance really does sit in a moderate-dimensional subspace, compressing the features before classification should recover most of the rank deficiency problem discussed above, at the cost of throwing away only the low-variance tail. To test this, we project the DINOv2 embeddings down to $d = 64$ and $d = 256$ dimensions and re-run the class-incremental classifiers on the compressed features. Table 5.9 reports the average accuracy once every class has been added, across three independent class-ordering runs. As with the uncompressed results, the class-conditional statistics come from a fixed training partition, so the standard deviation is zero across all cells: which order the classes arrive in does not change where the estimator converges.

Table 5.9: Final average accuracy (%) of the Gaussian continual-learning classifier across dimensionality settings, comparing PCA projections ($d \in \{64, 256\}$) against the full feature space ($d = 768$). *Diag*: diagonal covariance; *Full*: full covariance with adaptive ridge regularisation. Bold indicates the highest accuracy within each dimensionality setting. Higher is better.

	PCA $d = 64$			**PCA** $d = 256$			**Original** $d = 768$		
Benchmark	Diag	Full	MOG4-Shared	Diag	Full	MOG4-Shared	Diag	Full	MOG4-Shared
CIFAR-100	78.48	**83.05**	82.24	85.22	82.29	**88.16**	86.07	88.38	**88.98**
ImageNet-R	64.04	49.65	**66.84**	71.79	61.25	**77.06**	77.12	74.71	**80.19**
Caltech-256	87.20	43.40	**90.58**	90.01	91.09	**94.42**	90.06	**94.93**	94.61

The behavior across dimensionalities maps directly onto the shape of the spectrum described above. Scaling the PCA dimensionality from $d = 64$ to $d = 256$ lifts Diag-Cov's accuracy on every benchmark (+6.7 pp on Split CIFAR-100, +7.8 pp on Split ImageNet-R, +2.8 pp on Split Caltech-256), confirming that although fewer than 130 components already capture 90% of the variance, the remaining low-variance tail still holds fine-grained information that helps resolve close class boundaries.

MOG4-Shared benefits from this compression even more than Diag-Cov does. At $d = 256$ it is the best of the three methods on every benchmark (88.16% on CIFAR-100, 77.06% on ImageNet-R, 94.42% on Caltech-256), and at $d = 64$ it already leads on ImageNet-R and Caltech-256, trailing Full-Cov by less than a point on CIFAR-100 (82.24% vs. 83.05%). This follows from the same mechanism discussed in Section 5.4.4: MOG4-Shared estimates a single covariance from the pooled residuals of every active class, so compressing the features still leaves it with plenty of data to estimate that one covariance well. Full-Cov, by contrast, has to fit a separate covariance per class from a much smaller sample, which is exactly what causes it to struggle under compression, as the next subsection shows in more detail.

5.9.3 Covariance Parameterisations and Subspace Interplay

The gap between Diag-Cov and Full-Cov shows two sides of the same regularisation problem under data scarcity. On Split CIFAR-100 at $d = 64$, compression actually helps Full-Cov: the sample budget ($n_c = 400$) comfortably exceeds the projection dimension, so it can exploit

stable off-diagonal correlations and beat Diag-Cov by +4.6 pp. Expanding to $d = 256$ reverses this into a −2.9 pp deficit, since even within the leading part of the spectrum, the number of free parameters starts to outpace how much data is available.

This same effect causes a sharp drop for Full-Cov on the sample-starved streams of ImageNet-R ($n_c = 40$) and Caltech-256 ($n_c = 64$) under heavy compression. At $d = 64$, Full-Cov's unconstrained covariance becomes rank-deficient, losing 14.4 pp on ImageNet-R and 43.8 pp on Caltech-256 relative to Diag-Cov. These gaps narrow as dimensionality expands to $d = 256$, where Full-Cov eventually reaches the peak accuracy on Caltech-256 (91.09%), but the pattern still shows that a linear dimensionality reduction can distort per-class correlations whenever the sample budget stays close to the projection size. MOG4-Shared avoids this failure mode altogether, for the reason given above: it never has to estimate an independent covariance from a small, per-class sample in the first place.

5.10 Validating the DINOv2 Gaussianity Assumption

Chapter 3 checked Gaussianity at two levels: per-dimension, whether each coordinate on its own looks Gaussian, and per-class, whether the full, high-dimensional feature vector of each class does. Here we apply both checks to the actual DINOv2 embeddings used throughout this thesis, using the training split of each benchmark.

5.10.1 Per-Dimension Gaussianity

Shape moments. We compute the sample skewness and excess kurtosis defined in Chapter 3 for every (class c, dimension j) pair.

Table 5.10: Per-dimension Gaussianity statistics over all $(C \times d)$ (class, dimension) pairs per dataset ($d = 768$). $|\text{Skew}|_\mu$ and $|\text{Kurt}|_\mu$ are mean absolute values; $\mathbb{P}[\cdot]$ is the fraction of pairs satisfying each cut-off.

Dataset	**pairs**	$\lvert\text{Skew}\rvert_\mu$	Skew_σ	$\mathbb{P}[\lvert\text{Skew}\rvert<0.5]$	$\lvert\text{Kurt}\rvert_\mu$	$\mathbb{P}[\lvert\text{Kurt}\rvert<1]$
CIFAR-100	76,800	0.181	0.236	0.961	0.264	0.982
ImageNet-R	153,600	0.269	0.348	0.859	0.432	0.924
Caltech-256	192,000	0.360	0.472	0.747	0.659	0.822

Table 5.10 reports aggregate statistics. Mean skewness is essentially zero on all benchmarks ($|\overline{\text{Skew}}| \leq 0.002$). The fraction of pairs satisfying $|\text{Skew}| < 0.5$ drops from 96.1% on CIFAR-100 to 74.7% on Caltech-256, reflecting the smaller per-class sample count and greater visual diversity of the latter.

D'Agostino–Pearson test. We apply the joint hypothesis test defined in Chapter 3, rejecting H_0 at $\alpha = 0.05$, and report the fraction of pairs passing in Table 5.11. With $\bar{n}_c \approx 400$, 120, and 90 for CIFAR-100, ImageNet-R, and Caltech-256 respectively, the test gets stricter as the sample grows, so even small, harmless departures from normality get

flagged on the larger datasets; even so, 70–81% of all pairs pass it, and the relaxed criterion from Chapter 3 ($|\text{Skew}| < 1$ and $|\text{Kurt}| < 1$) is satisfied by 82–98%.

Table 5.11: Per-feature normality tests on the training split (fractions of $(C \times d)$ pairs). *normaltest*: D'Agostino–Pearson χ^2_2 at α=0.05. *relaxed*: $|\text{Skew}| < 1$ and $|\text{Kurt}| < 1$ simultaneously.

Dataset	Normaltest (%)	Relaxed (%)	Both (%)
CIFAR-100	70.0	97.8	70.0
ImageNet-R	80.8	91.6	79.7
Caltech-256	73.4	82.3	71.3

Henze–Zirkler test per dimension. As a stricter check, we also apply the univariate Henze–Zirkler (HZ) test defined in Chapter 3 independently to every (c, j) pair, using $d = 1$ and the same finite-sample expectation and bandwidth given there.

Table 5.12: Summary results from computing the univariate Henze–Zirkler test statistic over all $(C \times d)$ class-dimension pairs on the training split. $\mathbb{E}_{H_0}[T_{\text{HZ}}]$ is the theoretical expectation under perfect normality, as defined in Chapter 3, evaluated at the per-class sample size n_c.

Dataset	$\tilde{n}_c$	$\mathbb{E}_{H_0}[T_{\text{HZ}}]$	T_{HZ} mean $\pm$ std
CIFAR-100	500	≈ 0.04	0.00052 ± 0.00095
ImageNet-R	137	≈ 0.06	0.00134 ± 0.00210
Caltech-256	100	≈ 0.07	0.00234 ± 0.00379

Taken together, these three diagnostics agree: the individual coordinates of the foundation feature vectors have symmetric shapes, near-zero skewness, and little excess kurtosis, and the observed HZ scores sit orders of magnitude below their theoretical H_0 expectation. Every marginal, taken on its own, looks close to Gaussian.

5.10.2 Per-Class Gaussianity

Per-dimension normality is necessary but not sufficient: a set of coordinates can each look Gaussian on their own while still not being jointly Gaussian, so we now test each class's full $d = 768$-dimensional feature vector at once, using the same HZ statistic as before but with d set to the full embedding dimension. As explained in Chapter 3, this runs into the same problem as everywhere else in this thesis: since $n_c \leq d$ for every benchmark, the full covariance matrix needed for the standard version of the test cannot be inverted.

We work around this the same way: by using a diagonal version of the covariance, so only the d per-coordinate variances need to be inverted instead of the full matrix. This changes what to expect under H_0 as well. In the univariate case above, a Gaussian coordinate gives T_{HZ} close to 0; here, because d is large, the same expectation formula pushes the reference value close to 1 instead. Table 5.13 shows that all three benchmarks concentrate tightly around $T_{\text{HZ}} = 1$, right where a Gaussian class should land under this diagonal test.

As already noted in Chapter 3, restricting this test to a diagonal covariance means it cannot guarantee full, unconstrained multivariate Gaussianity, where every pair of dimensions

Table 5.13: Summary results from computing the diagonal HZ test statistic over all classes in the ambient foundation feature space ($d = 768$), using all available training samples. Values concentrated near $T_{\mathrm{HZ}} \approx 1.00$ indicate convergence toward empirical multivariate Gaussianity under the feature-independence assumption.

Dataset	C	T_{HZ} mean $\pm$ std
CIFAR-100	100	1.0002 ± 0.0020
ImageNet-R	200	1.0033 ± 0.0073
Caltech-256	250	1.0016 ± 0.0089

is allowed to correlate freely. But given how little data is available per class relative to the embedding dimension, this diagonal version is the only one we can actually compute, and it works as a reasonable stand-in for the stronger, unconstrained test. The consistent convergence to $T_{\mathrm{HZ}} \approx 1.00$ across all three benchmarks is what supports using independent-feature probabilistic classifiers on this embedding space.

5.10.3 Empirical Validation of the Gaussian Assumption

The Gaussianity tests above confirm that the per-coordinate normality assumption is well supported across all three benchmarks: 96.1 % of CIFAR-100 coordinates, 85.9 % of ImageNet-R, and 74.7 % of Caltech-256 satisfy the informal symmetry cut-off $|\mathrm{Skew}| < 0.5$. The key distinction between the two covariance estimators follows directly from the $n_c < d$ regime. The **Diag-Cov** model estimates only d parameters per class and is therefore always well-conditioned; its squared Mahalanobis distances are broadly distributed near the theoretical $\chi^2_{d=768}$ mode, with dispersion growing from CIFAR-100 to Caltech-256 in line with the heavy-tail ordering of Table 5.10. The **Full-Cov** model, by contrast, requires estimating a $d \times d$ matrix from $n_c < d$ samples: the resulting covariance is rank-deficient, the eigenvalues outside the spanned subspace collapse to the regularisation floor, and the quadratic form becomes degenerate, producing a sharp spike at $\overline{n_c} - 1$ (approximately 498, 168, and 103 on CIFAR-100, ImageNet-R, and Caltech-256 respectively) rather than spreading across the χ^2_{768} support. This problem comes from the *estimator*, not from the data: the per-coordinate distributions are near-Gaussian, as shown above. Keeping inter-feature correlations while avoiding rank deficiency is still an open challenge; the hybrid variant is one practical way to balance the two.

For **MOG4-Shared**, the same $n_c < d$ regime explains why it avoids this trade-off in the first place. Because MOG4-Shared estimates a single covariance from the pooled residuals of every active class rather than one covariance per class, it is never forced to invert a $d \times d$ matrix built from a single class's small sample, so it does not run into the rank-deficiency spike that hurts Full-Cov. On top of that, letting each class use up to four centroids means a class does not need its own feature distribution to be a single, unimodal Gaussian to be modelled well: a mixture of a few Gaussians can still fit a class whose marginals depart from normality more than the ones tested here. This is consistent with MOG4-Shared's results throughout this chapter, and it is one more reason it holds up better than either single-covariance model on its own.

5.11 Computational Cost and Practical Considerations

Weighing accuracy against computational overhead matters for judging whether a lifelong learning framework is actually practical to run. Table 5.14 reports the inference cost of the Gaussian classification layer, in floating-point operations (FLOPs), needed to score a single pre-embedded test query against every active category once all tasks have been added.

Each covariance parameterisation has a different, predictable cost:

- **Diag-Cov** scores each class with an element-wise weighted squared Euclidean distance, so the cost grows linearly, $\mathcal{O}(C \cdot d)$, where C is the number of active classes and $d = 768$ is the feature dimension.
- **Full-Cov** (and, by extension, Hybrid-Cov, whose Mahalanobis term uses the same Cholesky-based whitening) needs a full matrix–vector multiplication per class to account for the off-diagonal correlations, so its cost grows quadratically, $\mathcal{O}(C \cdot d^2)$.
- **MOG4-Shared** scores each class's up-to-four centroids against the one shared covariance, so its cost is $\mathcal{O}(C \cdot \bar{K} \cdot d^2)$, where $\bar{K}$ is the average number of centroids per class. This puts it between Diag-Cov and a hypothetical unshared, full-mixture model in cost, closer to Full-Cov in practice since both are dominated by the same d^2 term.

Table 5.14: Inference cost of the Gaussian classification stage once every class has been added, in floating-point operations (FLOPs) per query, for the two dominant covariance parameterisations and for MOG4-Shared. For reference, a single forward pass of the DINOv2 ViT-B/14 feature extractor requires approximately 17.56 GFLOPs.

Benchmark	**Diag-Cov**	**Full-Cov**	**MOG4-Shared**
Split CIFAR-100	0.46 MFLOPs	0.12 GFLOPs	0.42 GFLOPs
Split ImageNet-R	0.92 MFLOPs	0.24 GFLOPs	0.31 GFLOPs
Split Caltech-256	1.15 MFLOPs	0.29 GFLOPs	0.58 GFLOPs

Even the most demanding case in the table, MOG4-Shared on the 250 classes of Split Caltech-256, needs only 0.58 GFLOPs. That is still under 4% of the cost of a single forward pass through the DINOv2 ViT-B/14 feature extractor itself ($\approx$ 17.56 GFLOPs). So while MOG4-Shared is consistently more expensive than Diag-Cov and, on two of the three benchmarks, than Full-Cov too, the frozen backbone is still what dominates the total cost by a wide margin: whichever covariance estimator is used, it adds essentially no overhead at inference time.

5.11.1 Computational Footprint and Practical Scalability

These FLOPs numbers also show up as a real advantage in wall-clock time. On Split CIFAR-100, adding a newly encountered task to our framework takes about two minutes on a standard consumer-grade GPU, and that time is spent entirely on a single, deterministic pass of feature extraction over the raw images, needed to compute the class statistics.

Prompt-tuning methods reported in the literature typically need 10–30 minutes of training per task on similar hardware, since they backpropagate through a pool of learned prompts over several epochs. Because our framework never backpropagates and never needs a rehearsal buffer, it trains 5–15× faster, which makes it a reasonable option for low-resource or decentralized edge deployment.

5.11.2 Limitations and Architectural Open Questions

The empirical results support the efficiency and calibration of this frozen generative approach, but three limitations are worth being explicit about.

First, the current evaluation pipeline assumes that exact task boundaries are given during training to partition incoming batches. Relaxing this constraint to support task-free continual learning is an important next step, to test whether the decoupled Gaussian updates stay robust when categorical boundaries are blurred or interleaved.

Second, our experimental validation has focused only on visual classification streams. Although the underlying generative mechanics are in principle agnostic to the input modality, running a parallel validation on transformer-encoded text or speech embeddings is needed to check whether the clean Gaussian behavior observed here generalizes to other representation spaces.

Third, an interesting extension is using the well-behaved calibration profiles of our independent configurations for open-world discovery. The strong overconfidence of the Full covariance model makes its predictive metrics unreliable outside its closed label space. However, the well-calibrated Predictive Entropy and Probability Margin of the Diagonal model could be used directly as natural out-of-distribution scores. Investigating this could show whether the framework can flag novel, unlearned semantic categories while keeping its existing performance.

5.12 Summary of Findings and Practical Recommendations

The results in this chapter give an overall picture of the behavior, computational cost, and limitations of closed-form generative classifiers built on frozen foundation embeddings. Taken together, they show that class-incremental learning, in this setting, can be treated as a density-estimation problem in a stationary representation space rather than as a fight against forgetting. The DINOv2 feature manifold agrees well with the marginal normality assumption, passing coordinate-level Gaussianity thresholds for 96 % of dimensions in Split CIFAR-100, 86 % in Split ImageNet-R, and 75 % in Split Caltech-256, and because the representation stays stable as tasks accumulate, the resulting decision boundaries compete with optimization-heavy, prompt-based architectures while staying deterministic, closed-form, and gradient-free.

The classification ceiling of this framework is set mainly by the upstream feature extractor, not by the downstream classifier, as shown by the large performance gaps across different visual backbones in Section 5.8. Separately from the choice of backbone, classification error is not spread evenly across classes: it concentrates in a small, heavy-tailed set of pairs, under 0.2 % of all class combinations, where the pairwise Bhattacharyya distance drops below a critical threshold ($D_B < 20$). These are cases of near-synonymous or visually continuous categories, such as boy/girl at low resolution, or top-hat/cowboy-hat and bow_tie/scarf under domain shift, where second-order statistics alone cannot pull the classes apart. What looks like forgetting in a frozen stream is really just newly introduced categories landing close to ones already learned.

Which covariance estimator to use depends on the sample budget and how homogeneous the incoming stream is. MOG4-Shared is the strongest general-purpose default: pooling one covariance across every active class keeps it numerically stable even when classes come from very different, unevenly sampled sources, and letting each class use up to four centroids still captures visual variety that a single Gaussian would miss, which is why it comes out ahead across every benchmark in Table 5.3. When the per-class sample size comfortably exceeds the feature dimensionality and the stream is homogeneous, Full-Cov (or Hybrid-Cov under tighter sample budgets) can match or edge it out at a lower computational cost, since it captures off-diagonal correlations directly without needing the shared-covariance machinery. But once the stream mixes sources with very different sample sizes, as in the Joint benchmark, unconstrained per-class covariances break down, and a shared covariance, ideally paired with MOG4-Shared's extra centroids, is the only reliable choice.

These results support the central idea of this thesis: that lifelong learning, in this setting, can be handled as static statistical modelling instead of ongoing parameter optimization. Because the framework only ever computes frozen sufficient statistics, representation drift is not possible by construction, and the practical decision comes down to picking the covariance parameterisation, MOG4-Shared by default, one of the others when its sample budget and computational cost make more sense, that fits the sample density and semantic granularity of the deployment scenario. Chapter 6 builds on this to discuss the system-level conclusions and the open questions around representation alignment raised here.

Chapter 6

Conclusion

This thesis revisited class-incremental learning (CIL) in the context of modern visual foundation models, arguing that once representations are frozen and sufficiently discriminative, the central challenge of CIL shifts from protecting network parameters against interference to properly modelling the statistical structure of a stationary embedding space. We formalised this shift by formulating incremental learning as the recursive, gradient-free estimation of class-conditional Gaussian densities in the embedding space of a frozen DINOv2 backbone. Each class is represented entirely by its sufficient statistics—a mean vector and a covariance matrix—which are updated in closed form as new samples arrive, without backpropagation, replay buffers, exemplar storage, or prompt parameters, and without ever modifying the statistics of a previously learned class. This design eliminates catastrophic forgetting by construction rather than by mitigation, and it is the structural reason why the forgetting scores reported throughout Chapter 5 reflect increasing classification difficulty as the class set grows rather than any degradation of previously acquired knowledge.

Empirically, this conceptually simple formulation is competitive with the strongest prompt-based state of the art, and one specific configuration stands out once every benchmark is weighed together. Our Full-Cov and Hybrid-Cov estimators match ConvPrompt on Split CIFAR-100 (88.87 %) without any trainable parameters or optimisation during incremental learning, and outperform every prompt-based baseline reported on Split Caltech-256 (94.93 % versus the best prompt-based result of 92.50 %). MOG4-Shared, which pairs a BIC-selected number of per-class centroids with a single covariance shared across all active classes, rarely wins any single column of Table 5.3 outright, but it stays close to the top on CIFAR-100 (88.98 %), ImageNet-R (80.19 %) and Caltech-256 (94.61 %), and it is clearly the strongest closed-form estimator on the harder 30-task Joint benchmark, reaching 84.49 % against 81.49 % for the next-best Shared-Full-Cov model. This combination of consistency and accuracy is what makes it our recommended default. All of this is achieved at a classification-stage cost that stays under 0.6 GFLOPs even for MOG4-Shared's heavier scoring, close to thirty times smaller than the ≈ 17.56 GFLOPs required for a single forward pass of the DINOv2 backbone, so the probabilistic classification layer introduces only marginal overhead relative to feature extraction and requires none of the gradient updates that prompt-based competitors perform at every incremental task.

Beyond the accuracy figures, this thesis provided a rigorous statistical validation of the assumption that underlies the entire framework. Combining moment-based diagnostics with formal hypothesis tests—the D'Agostino–Pearson omnibus test for univariate normality and a diagonal adaptation of the Henze–Zirkler test for multivariate normality under severe rank deficiency—confirmed that the marginal feature distributions of a self-supervised foundation encoder are consistently, if not perfectly, Gaussian across three visually heterogeneous benchmarks: 70–81 % of all (class, dimension) pairs pass the strict D'Agostino–Pearson test, and the diagonal multivariate Henze–Zirkler statistic converges tightly to its Gaussian reference value of 1 on every dataset. The complementary Bhattacharyya-distance analysis established a direct, quantitative link between inter-class separability and the achievable classification accuracy: over 99.8 % of all class pairs on every benchmark exceed a separability threshold of $D_B = 50$, while the small minority of pairs that fall below $D_B = 20$—semantic or visual near-synonyms such as boy/girl, bow_tie/scarf, and top-hat/cowboy-hat—account for essentially the entirety of the residual classification error identified in the confusion-pattern analysis of Chapter 5. This is, to our knowledge, one of the first attempts in the class-incremental learning literature to connect a formal probabilistic error bound directly to the specific, named class pairs that a deployed classifier will actually confuse.

This thesis further showed that the reliability of full-rank covariance modelling is itself a matter of numerical care, not only of statistical assumption. Computing the precision matrix and log-determinant of the Full-Cov and Hybrid-Cov estimators via a Cholesky-based decomposition, rather than via an eigendecomposition with a hard eigenvalue floor, avoids a substantial artefact that would otherwise depress the Full-Cov accuracy on Split Caltech-256 by nearly five accuracy points and produce misleadingly optimistic results on the 30-task Joint benchmark. The results present a clear and interpretable picture: full-rank covariance estimation is highly beneficial whenever the per-class sample budget is adequate and homogeneous across classes, but the Joint benchmark's collapse of Full-Cov and Hybrid-Cov to 59.35 % is not a weakness of the estimator—it is a confirmation that covariance parameterisation must be matched to the sample-size regime of the deployment scenario, precisely as the Shared-covariance variants, which pool statistics across classes of heterogeneous rank, demonstrate by becoming the strongest estimators in that same setting: Shared-Full-Cov reaches 81.49 %, and MOG4-Shared, which adds per-class centroids on top of the same shared covariance, goes further still to 84.49 %.

These results, taken together, support the view that in the visual foundation-model era, the central challenge of class-incremental learning is no longer protecting network parameters from catastrophic interference but correctly modelling and exploiting the statistical geometry of a stable representation space. The success of a classical, decades-old statistical tool—the multivariate Gaussian classifier—in this modern setting is not a coincidence: it is made possible by the qualitative change that large-scale self-supervised pretraining has produced in the topology of visual embedding spaces. By decoupling representational plasticity from semantic categorisation, stable and well-behaved foundation features eliminate representation drift by construction, allowing lifelong visual recognition to be reframed as an ongoing, closed-form problem of probability density estimation rather than an ongoing problem of parameter optimisation.

Limitations. A key limitation of the present work follows directly from the discussion of Chapter 5: the framework's residual error is structurally bounded by the semantic resolution of the upstream encoder. No choice of covariance estimator or mixture model evaluated in this thesis can fully resolve a class pair whose Bhattacharyya distance and Euclidean mean separation are both near zero, since such a pair reflects a genuine ambiguity in the frozen feature space rather than a deficiency of the downstream classifier. A second, more practical limitation concerns MOG4-Shared itself: fitting up to four centroids per class and refitting the shared covariance at every task makes it noticeably more expensive to train than Diag-Cov or a single shared covariance, so choosing it over a cheaper estimator is a trade-off that should be made deliberately, based on how much the deployment can afford, rather than by default.

6.1 Future Work

Several natural extensions of the framework deserve further investigation, building on the longer-term research directions identified in this thesis.

Resolving Persistent Class Confusions. Our failure diagnostics isolated a minute, semantically coherent subset of class pairs (e.g., boy/girl or top-hat/cowboy-hat) that account for almost the entire residual error of the system due to their near-zero mean separation ($d_E \to 0$). Because this crowding is an inherent property of the upstream embedding space, it cannot be resolved by downstream parametric tuning. Future research should investigate lightweight, targeted representation alignment. Instead of re-training the entire deep backbone, a contrastive or metric-learning objective can be applied exclusively to the low-D_B class pairs flagged by our statistical diagnostics. This targeted adjustment can local-stretch the decision surface for highly ambiguous categories, maximizing class separability without disrupting the global stationarity of the remaining representation space.

Covariance Estimation Under Unbalanced Sample Sizes. The Hybrid-Cov estimator successfully decouples the Mahalanobis shape component from the log-normalisation volume term, providing a stable baseline on homogeneous streams. However, as demonstrated by the Joint benchmark collapse, unconstrained second-order parameters break down when forced to compare data-rich classes directly against sample-starved categories. To alleviate this sensitivity, future work should develop structured precision matrices that dynamically adapt their complexity to individual sample counts.

Probabilistic Uncertainty Calibration. Our calibration metrics reveal that full-covariance parameterisations suffer from structural over-confidence, as the estimated distribution volumes tend to collapse around scarce sample clusters. This miscalibration reduces the trustworthiness of downstream probability scores in safety-critical deployment environments. Future iterations should implement post-hoc calibration layers designed for closed-form streams, such as multi-class temperature scaling or conformal prediction frameworks. By computing distribution-free prediction intervals based on historical validation residuals, the system can provide mathematically guaranteed error bounds alongside its standard maximum-a-posteriori assignments.

Gradient-Free Spatial Object Detection. A natural extension of this architecture is moving from global scene classification to localized object detection. Traditional multi-stage object detectors rely on backpropagation to optimize region proposal networks (RPNs) and bounding-box regressors, which are costly to train. Future work should bypass this by exploiting the localized patch tokens that DINOv2 already produces in a single forward pass. Applying a simple decomposition, such as localized principal component analysis or spatial attention routing, directly to these frozen patch features could isolate and classify regions of interest in one gradient-free pass, keeping the lightweight, deterministic approach of this thesis intact.

Autonomous Open-World Discovery. Because this framework computes closed-form sufficient statistics natively, it provides a well-behaved predictive entropy and Log-Gap score for every incoming sample. This capability can be leveraged to shift the system from a closed-set classifier toward an open-world discovery framework. Since the uncertainty boundaries of the Diag-Cov branch remain structurally stable when new categories are introduced, establishing an adaptive out-of-distribution (OOD) threshold would allow the model to autonomously isolate unknown entities. Once flagged, these unfamiliar clusters can be accumulated and parameterized as new Gaussian classes in closed form, expanding the label space without requiring external supervision or manual task boundaries.

Bibliography

[1] A. Thakur and A. Konde, "Fundamentals of neural networks," *International Journal for Research in Applied Science and Engineering Technology*, vol. 9, pp. 407–426, 08 2021.

[2] M. Masana, X. Liu, B. Twardowski, M. Menta, A. D. Bagdanov, and J. van de Weijer, "Class-incremental learning: Survey and performance evaluation on image classification," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 5, pp. 5513–5533, 2023. doi: 10.1109/TPAMI.2022.3213473.

[3] A. Dosovitskiy, L. Beyer, A. Kolesnikov, , *et al.*, "An image is worth 16x16 words: Transformers for image recognition at scale," in *International Conference on Learning Representations (ICLR)*, 2021.

[4] J. M. Joyce, "Kurtosis," in *International Encyclopedia of Statistical Science*, pp. 722–723, Springer, 2011.

[5] K. Pearson, "Contributions to the mathematical theory of evolution.II. skew variation in homogeneous material," *Philosophical Transactions of the Royal Society of London. A*, vol. 186, pp. 343–414, 1895.

[6] R. French, "Catastrophic forgetting in connectionist networks," *Trends in cognitive sciences*, vol. 3, pp. 128–135, 05 1999. doi: 10.1016/S1364-6613(99)01294-2.

[7] G. M. van de Ven, Z. Li, and A. S. Tolias, "Class-incremental learning with generative classifiers," in *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, pp. 3606–3615, 2021. doi: 10.1109/CVPRW53098.2021.00400.

[8] R. Chitale, A. Vaidya, A. Kane, and A. Ghotkar, "Task arithmetic with lora for continual learning," in *Workshop on Advancing Neural Network Training: Computational Efficiency, Scalability, and Resource Optimization (WANT@NeurIPS)*, 2023.

[9] Y.-S. Liang and W.-J. Li, "Adaptive plasticity improvement for continual learning," in *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 7816–7825, 2023. doi: 10.1109/CVPR52729.2023.00755.

[10] L. Wang, X. Zhang, H. Su, and J. Zhu, "A comprehensive survey of continual learning: Theory, method and application," *IEEE transactions on pattern analysis and machine intelligence*, vol. PP, 02 2024. doi: 10.1109/TPAMI.2024.3367329.

[11] R. V. Sluyters, J. Atkinson, M. Banks, R. Held, K.-P. Hoffmann, and C. Shatz, *Visual Perception: The Neurophysiological Foundations*, ch. The Development of Vision and Visual Perception. San Diego, CA: Academic Press, 1990.

[12] M. Oquab, T. Darcet, T. Moutakanni, *et al.*, "DINOv2: Learning robust visual features without supervision," *Transactions on Machine Learning Research*, 2024. Featured Certification. Code available at: `https://github.com/facebookresearch/dinov2`. Accessed: 2026-08-11.

[13] R. B. D'Agostino, "An omnibus test of normality for moderate and large size samples," *Biometrika*, vol. 58, no. 2, pp. 341–348, 1971.

[14] R. B. D'Agostino and E. S. Pearson, "Tests for departure from normality. empirical results for the distributions of b_2 and $\sqrt{b_1}$," *Biometrika*, vol. 60, no. 3, pp. 613–622, 1973.

[15] N. Henze and B. Zirkler, "A class of invariant consistent tests for multivariate normality," *Communications in Statistics – Theory and Methods*, vol. 19, no. 10, pp. 3595–3617, 1990.

[16] S. J. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*. Upper Saddle River, NJ: Pearson, fourth ed., 2020. Global Edition.

[17] C. M. Bishop, *Pattern Recognition and Machine Learning*. Information Science and Statistics, New York, NY: Springer, 2006.

[18] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. Cambridge, MA: The MIT Press, 2nd ed., 2018.

[19] R. Szeliski, *Computer Vision: Algorithms and Applications*. Springer, 2010.

[20] Y. LeCun, B. Boser, J. S. Denker, D. Henderson, R. E. Howard, W. Hubbard, and L. D. Jackel, "Backpropagation applied to handwritten zip code recognition," *Neural Computation*, vol. 1, no. 4, pp. 541–551, 1989.

[21] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. u. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems* (I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, eds.), vol. 30, Curran Associates, Inc., 2017.

[22] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," in *Advances in Neural Information Processing Systems* (F. Pereira, C. Burges, L. Bottou, and K. Weinberger, eds.), vol. 25, Curran Associates, Inc., 2012.

[23] M. Sazzad Hossain, P. Saha, T. Faisal Chowdhury, S. Rahman, F. Rahman, and N. Mohammed, "Rethinking task-incremental learning baselines," in *2022 26th International Conference on Pattern Recognition (ICPR)*, pp. 2771–2777, 2022.

[24] A. Chaudhry, M. Rohrbach, M. Elhoseiny, T. Ajanthan, P. K. Dokania, P. H. S. Torr, and M. Ranzato, "Continual learning with tiny episodic memories," *ArXiv*, vol. abs/1902.10486, 2019.

[25] A. Chaudhry, M. Ranzato, M. Rohrbach, and M. Elhoseiny, "Efficient lifelong learning with A-GEM," *CoRR*, vol. abs/1812.00420, 2018.

[26] P. Buzzega, M. Boschini, A. Porrello, D. Abati, and S. Calderara, "Dark experience for general continual learning: a strong, simple baseline," in *Proceedings of the 34th International Conference on Neural Information Processing Systems*, NIPS '20, (Red Hook, NY, USA), Curran Associates Inc., 2020.

[27] X. Liu, C. Wu, M. Menta, L. Herranz, B. Raducanu, A. D. Bagdanov, S. Jui, and J. van de Weijer, "Generative feature replay for class-incremental learning," in *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, pp. 915–924, 2020. doi: 10.1109/CVPRW50498.2020.00121.

[28] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, D. Hassabis, C. Clopath, D. Kumaran, and R. Hadsell, "Overcoming catastrophic forgetting in neural networks," *Proceedings of the National Academy of Sciences*, vol. 114, p. 3521–3526, Mar. 2017. doi: 10.1073/pnas.1611835114.

[29] F. Zenke, B. Poole, and S. Ganguli, "Continual learning through synaptic intelligence," in *Proceedings of the 34th International Conference on Machine Learning - Volume 70*, ICML'17, p. 3987–3995, JMLR.org, 2017.

[30] Z. Li and D. Hoiem, "Learning without forgetting," *CoRR*, vol. abs/1606.09282, 2016.

[31] S. C. Y. Hung, C.-H. Tu, C.-E. Wu, C.-H. Chen, Y.-M. Chan, and C.-S. Chen, "Compacting, picking and growing for unforgetting continual learning," in *Proceedings of the 33rd International Conference on Neural Information Processing Systems*, (Red Hook, NY, USA), pp. 1–11, Curran Associates Inc., 2019.

[32] S. Yan, J. Xie, and X. He, "Der: Dynamically expandable representation for class incremental learning," *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 3013–3022, 2021.

[33] J. Yoon, E. Yang, J. Lee, and S. J. Hwang, "Lifelong learning with dynamically expandable networks," in *6th International Conference on Learning Representations, ICLR 2018*, 2018.

[34] Z. Wang, Z. Zhang, C.-Y. Lee, H. Zhang, R. Sun, X. Ren, G. Su, V. Perot, J. Dy, and T. Pfister, "Learning to prompt for continual learning," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 139–149, 2022.

[35] Z. Wang, Z. Zhang, S. Ebrahimi, R. Sun, H. Zhang, C.-Y. Lee, X. Ren, G. Su, V. Perot, J. Dy, *et al.*, "Dualprompt: Complementary prompting for rehearsal-free continual learning," *European Conference on Computer Vision*, 2022.

[36] J. S. Smith, L. Karlinsky, V. Gutta, P. Cascante-Bonilla, D. Kim, A. Arbelle, R. Panda, R. Feris, and Z. Kira, "Coda-prompt: Continual decomposed attention-based prompting for rehearsal-free continual learning," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 11909–11919, June 2023.

[37] Y.-S. Liang and W.-J. Li, "Inflora: Interference-free low-rank adaptation for continual learning," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 23638–23647, 2024.

[38] O. Moussadek, R. Salami, and S. Calderara, "Dolfin: Balancing stability and plasticity in federated continual learning," 2025.

[39] H. Chen, P. Wang, Z. Zhou, X. Zhang, Z. Wu, and Y.-G. Jiang, "Achieving more with less: Additive prompt tuning for rehearsal-free class-incremental learning," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 340–349, October 2025.

[40] G. Zhang, L. Wang, G. Kang, L. Chen, and Y. Wei, "Slca: Slow learner with classifier alignment for continual learning on a pre-trained model," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 19148–19158, 2023.

[41] H. Zhuang, Z. Weng, H. Wei, R. Xie, K.-A. Toh, and Z. Lin, "Acil: Analytic class-incremental learning with absolute memorization and privacy protection," *Advances in Neural Information Processing Systems*, vol. 35, pp. 11602–11614, 2022.

[42] D. Goswami, Y. Liu, B. Twardowski, and J. van de Weijer, "Fecam: Exploiting the heterogeneity of class distributions in exemplar-free continual learning," *Advances in Neural Information Processing Systems*, vol. 36, pp. 6582–6595, 2024.

[43] P. Janson, W. Zhang, R. Aljundi, and M. Elhoseiny, "A simple baseline that questions the use of pretrained-models in continual learning," in *NeurIPS 2022 Workshop on Distribution Shifts*, 2022.

[44] M. D. McDonnell, D. Gong, A. Parvaneh, E. Abbasnejad, and A. van den Hengel, "Ranpac: Random projections and pre-trained models for continual learning," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[45] G. Petit, A. Popescu, H. Schindler, D. Picard, and B. Delezoide, "Fetril: Feature translation for exemplar-free class-incremental learning," in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, pp. 3911–3920, 2023.

[46] R. Bommasani, D. A. Hudson, E. Adeli, R. Altman, S. Arora, S. von Arx, M. S. Bernstein, J. Bohg, A. Bosselut, E. Brunskill, *et al.*, "On the opportunities and risks of foundation models," *arXiv preprint arXiv:2108.07258*, 2021.

[47] A. Kirillov, E. Mintun, N. Ravi, H. Mao, C. Rolland, L. Gustafson, T. Xiao, S. Whitehead, A. C. Berg, W.-Y. Lo, P. Dollár, and R. Girshick, "Segment anything," in *2023 IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.

[48] M. Caron, H. Touvron, I. Misra, H. Jégou, J. Mairal, P. Bojanowski, and A. Joulin, "Emerging properties in self-supervised vision transformers," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 9650–9660, 2021.

[49] J. Zhou, C. Wei, H. Wang, W. Shen, C. Xie, A. L. Yuille, and T. Kong, "ibot: Image BERT pre-training with online tokenizer," *CoRR*, vol. abs/2111.07832, 2021.

[50] J. F. Hair, W. C. Black, B. J. Babin, and R. E. Anderson, *Multivariate Data Analysis*. Cengage Learning, 8th ed., 2019.

[51] V. A. Marchenko and L. A. Pastur, "Distribution of eigenvalues for some sets of random matrices," *Mathematics of the USSR-Sbornik*, vol. 1, no. 4, pp. 457–483, 1967.

[52] J. H. Friedman, "Regularized discriminant analysis," *Journal of the American Statistical Association*, vol. 84, no. 405, pp. 165–175, 1989.

[53] O. Ledoit and M. Wolf, "A well-conditioned estimator for large-dimensional covariance matrices," *Journal of Multivariate Analysis*, vol. 88, no. 2, pp. 365–411, 2004.

[54] Y. Chen, A. Wiesel, Y. C. Eldar, and A. O. Hero, "Shrinkage algorithms for MMSE covariance estimation," *IEEE Transactions on Signal Processing*, vol. 58, no. 10, pp. 5016–5029, 2010.

[55] I. T. Jolliffe, *Principal Component Analysis*. New York: Springer, 2nd ed., 2002.

[56] A. Krizhevsky, "The cifar-100 dataset." `https://www.cs.toronto.edu/~kriz/cifar.html`, 2009. Accessed: 2025-10-02.

[57] G. Griffin, A. Holub, and P. Perona, "Caltech 256," 2022. doi: 10.22002/D1.20087. Accessed: 2026-08-11.

[58] D. Hendrycks, S. Basart, N. Mu, S. Kadavath, F. Wang, E. Dorundo, R. Desai, T. Zhu, S. Parajuli, M. Guo, D. Song, J. Steinhardt, and J. Gilmer, "The many faces of robustness: A critical analysis of out-of-distribution generalization," in *2021 IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 8320–8329, 2021.

[59] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "Imagenet: A large-scale hierarchical image database," in *2009 IEEE Conference on Computer Vision and Pattern Recognition*, pp. 248–255, 2009. doi: 10.1109/CVPR.2009.5206848.

[60] T. Ridnik, E. B. Baruch, A. Noy, and L. Zelnik-Manor, "Imagenet-21k pretraining for the masses," *CoRR*, vol. abs/2104.10972, 2021.

[61] R. Wightman, "Pytorch image models." `https://github.com/rwightman/pytorch-image-models`, 2019.

[62] C. Eckart and G. Young, "The approximation of one matrix by another of lower rank," *Psychometrika*, vol. 1, no. 3, pp. 211–218, 1936.

[63] S. Ren, K. He, R. Girshick, and J. Sun, "Faster r-cnn: Towards real-time object detection with region proposal networks," in *Advances in Neural Information Processing Systems* (C. Cortes, N. Lawrence, D. Lee, M. Sugiyama, and R. Garnett, eds.), vol. 28, Curran Associates, Inc., 2015.